Unit 01: Programming in C++

UVIC

CSC 115: Fundamentals of Programming II

Anthony Estey



Learning Objectives

- Write and compile a basic C++ program using the command line interface
- Read and interpret compiler error messages to identify and fix syntax errors
- Declare and initialize variables using both C-style and uniform initialization
- Identify the appropriate type for a given value (int, double, bool, char)
- Explain how a variable's type determines its memory size and range of values
- Describe how variables are stored in memory using addresses
- Apply casting to control type conversion in arithmetic expressions
- Evaluate expressions with arithmetic, relational, logical, and shorthand operators
- Predict the result of integer division and explain when truncation occurs
- Apply the rules of short-circuit evaluation to && and || expressions
- Trace programs involving variables and operators using memory diagrams
- Use std::cout and std::cin to produce output and read input from the terminal
- Construct output statements that combine text and variable values using <<
- Write if/else if/else statements in C++ and explain how they differ from Python
- Write for, range-based for, while, and do-while loops in C++
- Select the appropriate loop type for a given problem and justify your decision

Learning Objectives

- Declare and define functions with explicit return types and typed parameters
- Explain variable scope and trace when a variable is accessible throughout a program
- Apply the function design process: prototype, documentation, definition, & testing
- Write and place function prototypes correctly
- Apply snake_case naming conventions for variables and functions
- Write readable, well-documented C++ code
- Write documentation using @brief, @param, @return, and @pre
- Design test cases covering typical, edge, and boundary cases across all parameters
- Declare, initialize, and traverse arrays in C++
- Explain the key differences between C++ arrays and Python lists
- Use std::string and its core operations (.length(), .substr(), .find())
- Explain why std::string is an object and what that means
- Explain why structs are useful for grouping related data
- Declare a struct type with multiple fields of different types
- Create struct variables using uniform initialization
- Access and modify struct fields using the dot operator

01: Introduction to C++

Anthony Estey
CSC 115: Fundamentals of Programming: II

Programming courses at UVic

- As of the Fall 2026 term…

- Students start in CSC 110, where they learn Python
  - Python is great for learning programming fundamentals
  - Hides many details (like how memory is managed)

- Then students move into CSC 115 and use C++
  - Compared to Python, it's going to feel difficult at first
  - You will get a much better understanding of what's happening when a program runs

What is C++?

- C++ is a general-purpose programming language that was developed in the early 1980s by a computer scientist named Bjarne Stroustrup.
- He was working at Bell Labs and wanted to add some features to the C programming language, particularly the ability to work with objects and classes.
  - C++ started as "C with classes"
  - In C, a variable value is incremented with ++
  - C++ can be thought of as *"C plus 1"* or *"one better than C"*

Why are we using C++?

- There are a few reasons:
  - C++ is still one of the most widely used programming languages in the world, and not just in computer science!
  - It shows up in many places directly relevant to you:
    - For engineers: microcontrollers in medical devices, automotive systems, industrial sensors. It's also used for structural analysis or fluid dynamics
    - In physics, biology, and math: high-performance simulations, numerical solvers, and data analysis pipelines
    - In game development: major engines like Unreal are built in C++
    - And basically any field that uses hardware where performance really matters, like robotics, aerospace, or telecommunications

Why are we using C++?

- More importantly, C++ is going to expose you to some things that Python was hiding from you:
  - In Python, we don't really think about memory
  - We create a variable, and Python takes care of where it's stored and when it gets cleaned up
- In C++, we're going to pull back that curtain, and we will learn:
  - how memory is actually managed
  - what pointers are
  - what it means to allocate and deallocate memory
- These are concepts that come up again and again, and this is where we build that foundation.

C++ Summary

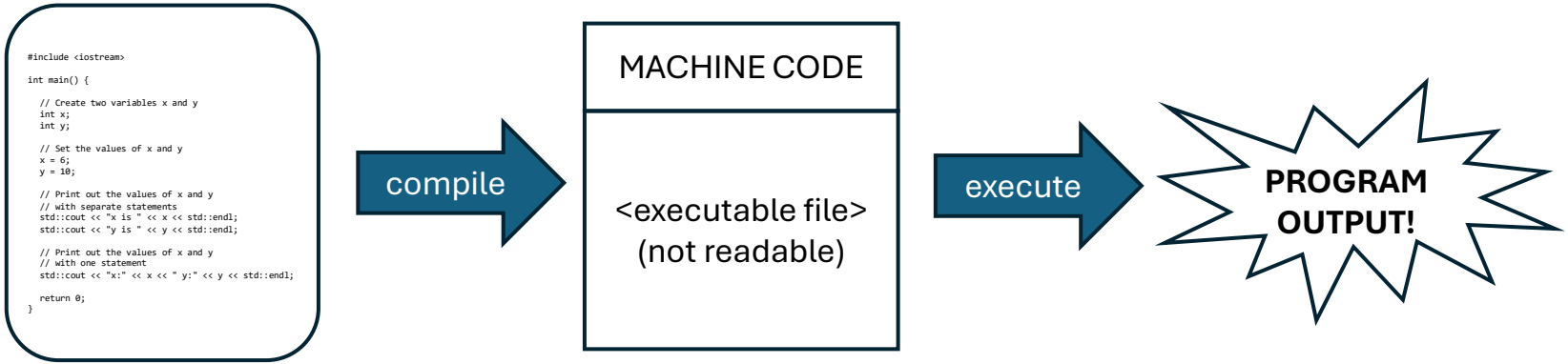
- The reasons we are using C++:
  - It is a powerful, widely-used language
  - It gives us direct access to memory management
  - It supports object-oriented programming
- Things to be aware of:
  - It's going to be more demanding than Python at first
  - We won’t spend as much time learning the language like in CSC 110, but many things will be familiar (variables, ifs, loops, etc.)
  - Building onto your current knowledge, the concepts we’ll explore this term will serve you well throughout the rest of your program

02: Our First C++ Program

Anthony Estey
CSC 115: Fundamentals of Programming: II

“Running” a C++ program

- We will be writing C++ programs in an editor
  - This code is readable and editable by us
- To run it, the first thing we're going to do is compile it
  - This generates a different file that is not readable by us
  - It's machine code that our computer can execute.



Our First C++ Program

```
/**
 * @brief Prints a welcome message to CSC 115 students.
 * @author Anthony Estey
 */

#include <iostream>

int main() {
  std::cout << "Welcome everyone!" << std::endl;

  return 0;
}
```

Our First C++ Program

```
/**
 * @brief Prints a welcome message to CSC 115 students.
 * @author Anthony Estey
 */

#include <iostream>

int main() {
  std::cout << "Welcome everyone!" << std::endl;

  return 0;
}
```

Comment

Our First C++ Program

```
/**
 * @brief Prints a welcome message to CSC 115 students.
 * @author Anthony Estey
 */

#include <iostream>

int main() {
  std::cout << "Welcome everyone!" << std::endl;

  return 0;
}
```

Compiler directive

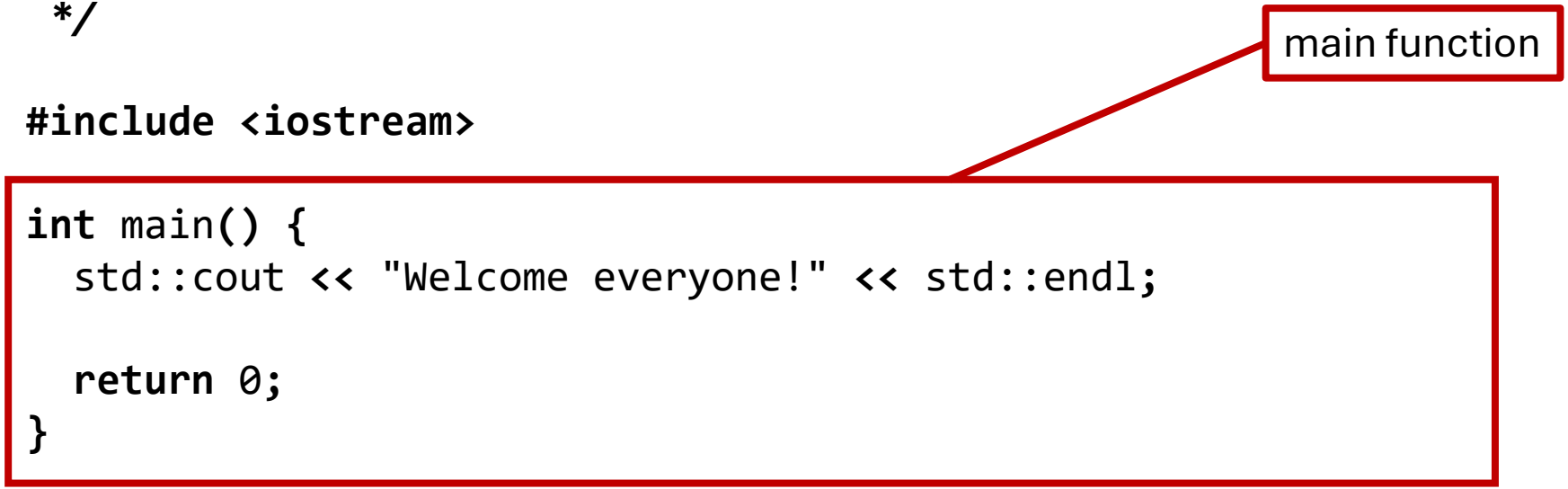
Our First C++ Program

```
/**
 * @brief Prints a welcome message to CSC 115 students.
 * @author Anthony Estey
 */

#include <iostream>

int main() {
  std::cout << "Welcome everyone!" << std::endl;

  return 0;
}
```



The **main** function

```
int main() {

  std::cout << "Welcome everyone!" << std::endl;

  return 0;

}
```

The **main** function

```
int main() {

  std::cout << "Welcome everyone!" << std::endl;

  return 0;

}
```

Curly brackets define blocks of code

The **main** function

```
int main() {

  std::cout << "Welcome everyone!" << std::endl;

  return 0;

}
```

Semicolons terminate each statement

The **main** function

```
int main() {
    std::cout << "Welcome everyone!" << std::endl;
    return 0;
}
```

Code in the main function is executed

The **main** function

```
int main() {

  std::cout << "Welcome everyone!" << std::endl;

  return 0;

}
```

`std::cout` is how we output to the terminal

The **main** function

```
int main() {

  std::cout << "Welcome everyone!" << std::endl;

  return 0;

}
```

Indicates the program completed sucessfully

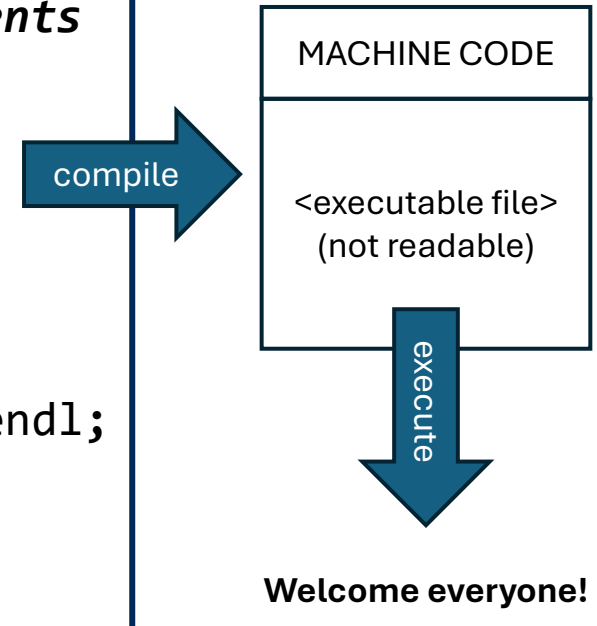
Our First C++ Program

```
/**
 * Prints a welcome message to CSC 115 students
 * @author Anthony Estey
 */

#include <iostream>

int main() {
  std::cout << "Welcome everyone!" << std::endl;

  return 0;
}
```



03: Programming Environment & Setup

Anthony Estey
CSC 115: Fundamentals of Programming: II

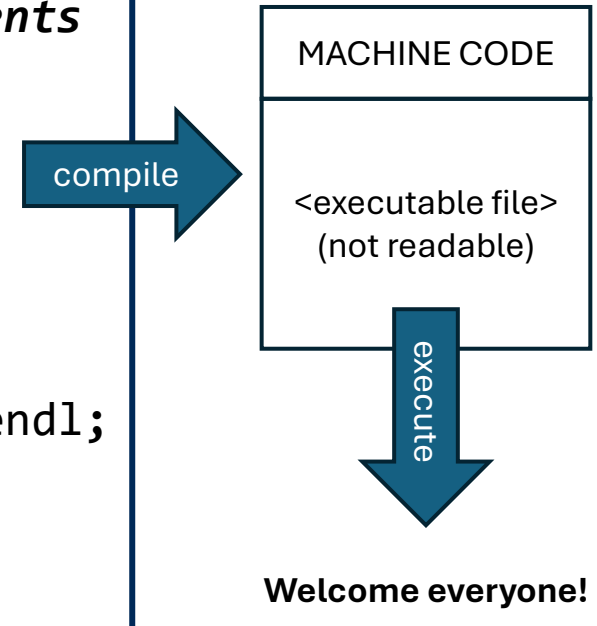
Our First C++ Program

```
/**
 * Prints a welcome message to CSC 115 students
 * @author Anthony Estey
 */

#include <iostream>

int main() {
  std::cout << "Welcome everyone!" << std::endl;

  return 0;
}
```

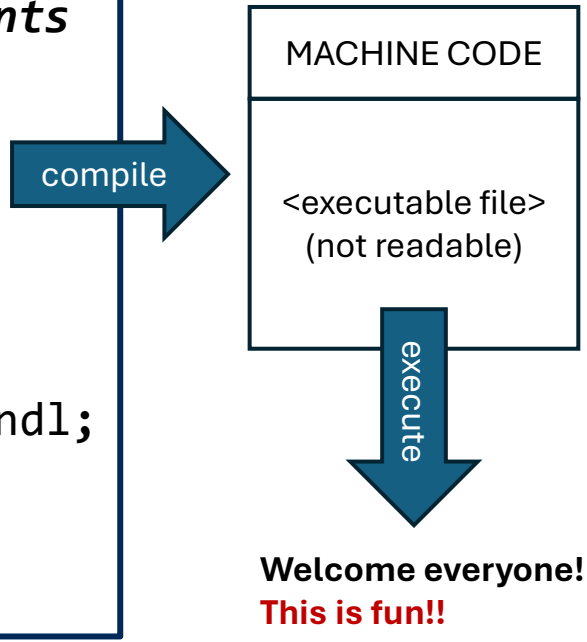


Our First C++ Program

```
/**
 * Prints a welcome message to CSC 115 students
 * @author Anthony Estey
 */

#include <iostream>

int main() {
  std::cout << "Welcome everyone!" << std::endl;
  std::cout << "This is fun!!" << std::endl;
  return 0;
}
```



Save any changes made since last compile!

Compilation

- When we compile a program, the *human-readable* C++ code written by us is transformed into executable code that a computer can run.

- There are a few steps in this process…
  - **Preprocessing:** Directives like `#include <iostream>` provide the compiler with instructions about what other files should be included in the source code
  - **Compilation:** The compiler translates the preprocessed code into assembly language. The compiler ensures our code follows the correct syntax rules.
  - **Assembly:** The assembly code is translated in binary code, resulting in an object file (notice a file with extension .o shows up in your working directory).
  - **Linking:** The object file(s) and any other required files are combined into an executable file that we will be able to run on our computer.

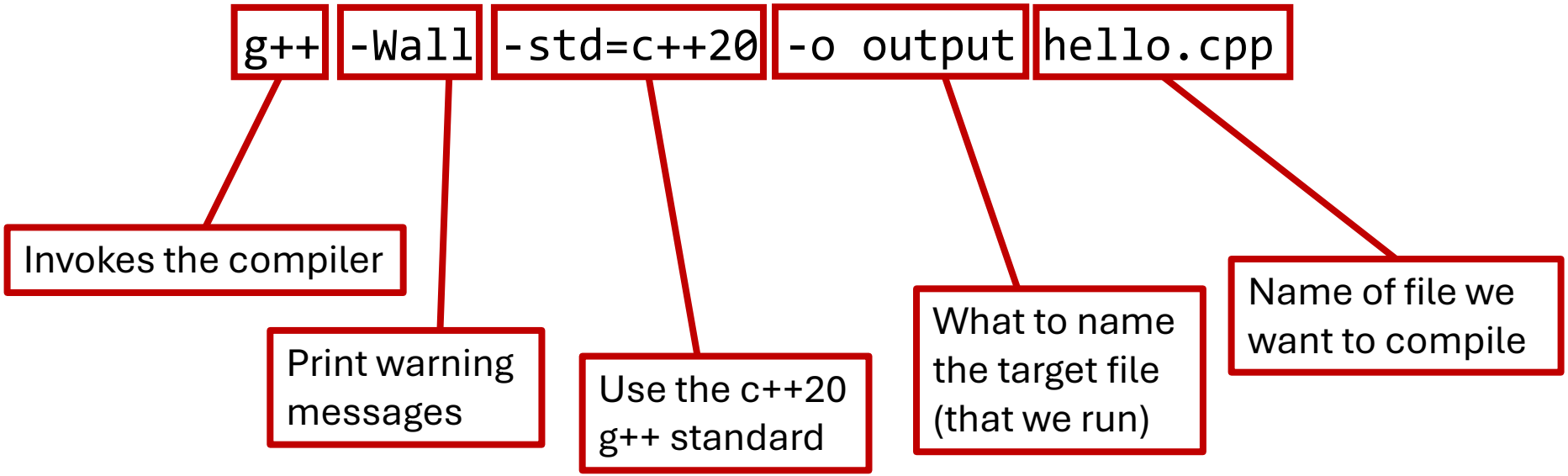
Programming Environment Setup

- Go to the course page
  - And then go to **“Programming Environment & Setup”**

- It has step-by-step instructions on writing, compiling, and running a C++ program for the first time!
  - I don’t have these specific instructions here in the video, because links and other things may change from term to term

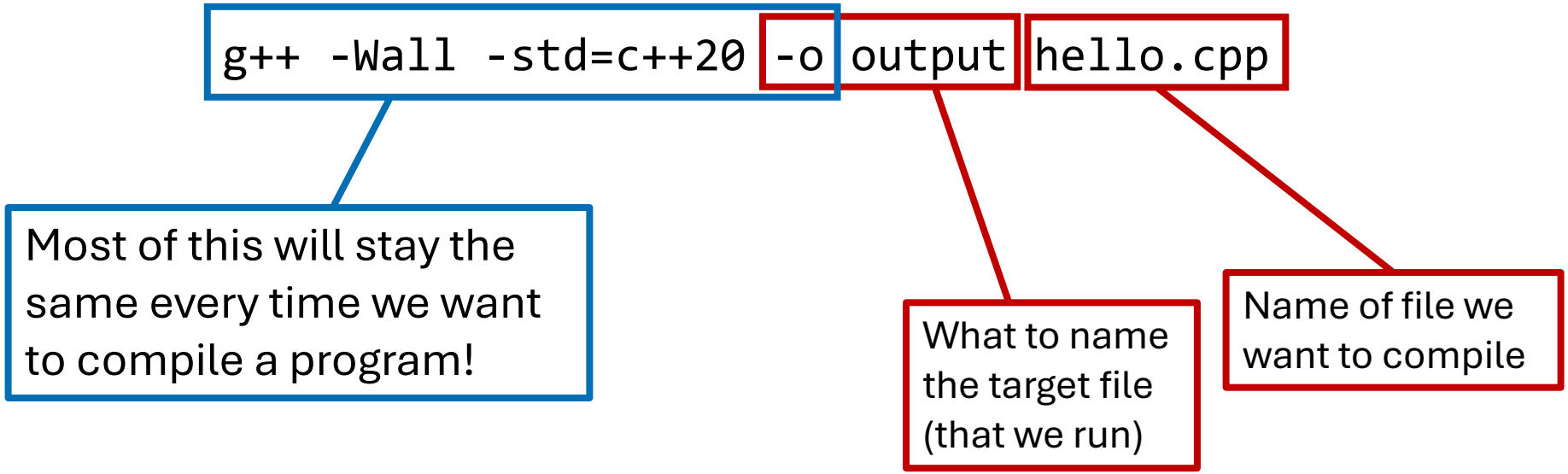
Compilation Example

- Assume we saved a C++ program and named it **hello.cpp** (remember to add the .cpp to all of our C++ programs)
- To compile it, we would type the following:



Compilation Example

- Assume we saved a C++ program and named it **hello.cpp** (remember to add the .cpp to all of our C++ programs)
- To compile it, we would type the following:



Compilation Example

- Assume we saved a C++ program and named it **hello.cpp** (remember to add the .cpp to all of our C++ programs)
- To compile it, we would type the following:

```
g++ -Wall -std=c++20 -o output hello.cpp
```



- If no errors are reported, an executable file named **output** is generated, which we can run by typing the following:

```
./output
```

04: Variables and Memory

Anthony Estey
CSC 115: Fundamentals of Programming: II

Variables

- A **variable** is a storage location for a single piece of data.

- Variables must have a **name**, a **type**, and a **value**.
  - The name and type are declared by the programmer when the variable is first created and *cannot be changed*
  - The value can be changed as needed, using an **assignment statement**

Declaration and Assignment

- General form for declaration and initialization (C-style):

```
type name = value;
```

- General form assignment:

```
name = new_value;
name = (expression result)
```

- Specific examples for declaration and initialization:

```
int fav_number = 10;
double gpa = 8.4;
```

- Specific assignment examples:

```
fav_number = 21;
gpa = 4.0 + 4.8;
gpa = gpa - 0.1;
```

Types

- There are a number of **built-in types** in C++.
- The table below shows the common variable types we will focus on in this course:

Category	Keyword	Description
Whole numbers	`int`	Stores whole numbers
Floating-point numbers	`double`, `float`	Stores numbers with decimal/fractional parts
Boolean values	`bool`	Stores either `true` or `false`
Single characters	`char`	Stores individual text symbols

Why are types important?

- Types matter because they determine how much memory a value uses (or how much memory to allocate for a variable)
  - This directly affects the range of values that can be stored

Type	Size	Description of values stored
`int`	**4 bytes**	Stores values in the range from roughly -2 billion to +2 billion
`double`	**8 bytes**	Stores values to about 15 decimal digits of precision
`bool`	**1 byte**	Stores either **`true`** or **`false`** (and nothing else)
`char`	**1 byte**	Stores a single character symbol (and always just a single character)

Variables and memory

- All variables take up space in memory
- When a variable is declared, space is allocated for it at an open **memory address**
- A **memory address** is like a home address: it uniquely identifies where the variable lives
  - the variable's name is for us
  - the address is how the computer finds it
- Addresses are usually written in hexadecimal (e.g. 0x1004)
  - you'll see this notation when we do exercises to output addresses

Memory Allocation Example

- Steps to trace memory allocation:
  - Consider the size (based on type)
  - Allocate space for the variable at the first available set of memory addresses
- For example:

```
int first = 24;
int second = 8;
double third = 3.14;
int fourth = -99;
```

Variable	Address	Value
first	0x1000	24
	0x1001	
	0x1002	
	0x1003	
second	0x1004	8
	0x1005	
	0x1006	
	0x1007	
third	0x1008	3.14
	0x1009	
	0x100a	
	0x100b	
	0x100c	
	0x100d	
	0x100e	
	0x100f	
fourth	0x1010	-99
	0x1011	
	0x1012	
	0x1013	
	0x1014	

05: Arithmetic and Shorthand Operators

Anthony Estey
CSC 115: Fundamentals of Programming: II

Operations

Arithmetic Operators (produce a numerical value)	
`+ - / *`	Basic arithmetic operations
`%`	Remainder after integer division (ex: 7%3 = 1)
Shorthand Operators (combine assignment with an arithmetic operation)	
`+= -= *= /=`	Shorthand for updating a variable (x+=1 is the same as x=x+1)
`++ --`	Increment and decrement a variable by 1
Relational Operators (produce a boolean value: true or false)	
`< > <= >=`	Compares whether a value is greater or less than another
`== !=`	Checks whether two values are equal or not equal
Logical Operators (produce a boolean value: true or false)	
`&& || !`	Logical **and**, **or**, and **not** (to combine boolean expressions)

Operations

Arithmetic Operators (produce a numerical value)	
`+ - / *`	Basic arithmetic operations
`%`	Remainder after integer division (ex: 7%3 = 1)
Shorthand Operators (combine assignment with an arithmetic operation)	
`+= -= *= /=`	Shorthand for updating a variable (x+=1 is the same as x=x+1)
`++ --`	Increment and decrement a variable by 1
Relational Operators (produce a boolean value: true or false)	
`< > <= >=`	Compares whether a value is greater or less than another
`== !=`	Checks whether two values are equal or not equal
Logical Operators (produce a boolean value: true or false)	
`&& || !`	Logical **and**, **or**, and **not** (to combine boolean expressions)

Operations

Arithmetic Operators (produce a numerical value)	
`+ - / *`	Basic arithmetic operations
`%`	Remainder after integer division (ex: 7%3 = 1)
Shorthand Operators (combine assignment with an arithmetic operation)	
`+= -= *= /=`	Shorthand for updating a variable (x+=1 is the same as x=x+1)
`++ --`	Increment and decrement a variable by 1
Relational Operators (produce a boolean value: true or false)	
`< > <= >=`	Compares whether a value is greater or less than another
`== !=`	Checks whether two values are equal or not equal
Logical Operators (produce a boolean value: true or false)	
`&& || !`	Logical **and**, **or**, and **not** (to combine boolean expressions)

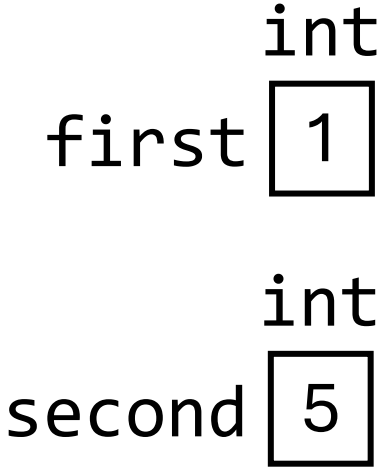
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	1
second	0x1004	5
	0x1008	

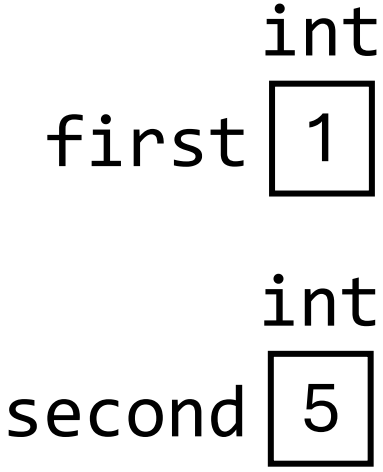
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	1
second	0x1004	5
	0x1008	

Tracing Example

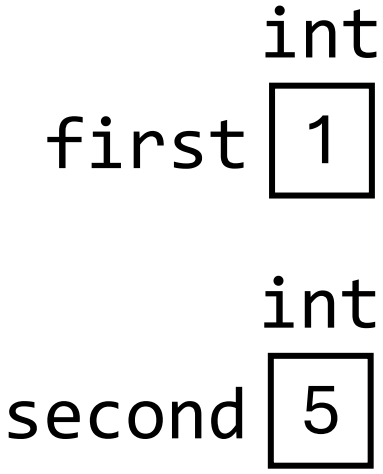
```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```

5



Memory:

Variable	Address	Value
first	0x1000	1
second	0x1004	5
	0x1008	

Tracing Example

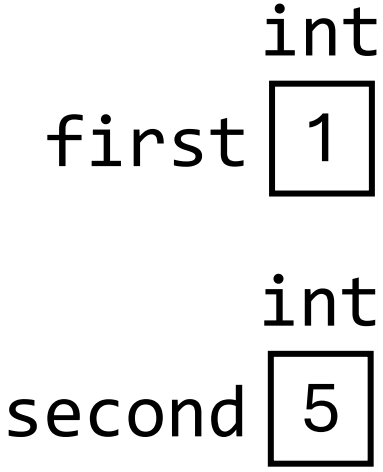
```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```

10



Memory:

Variable	Address	Value
first	0x1000	1
second	0x1004	5
	0x1008	

Tracing Example

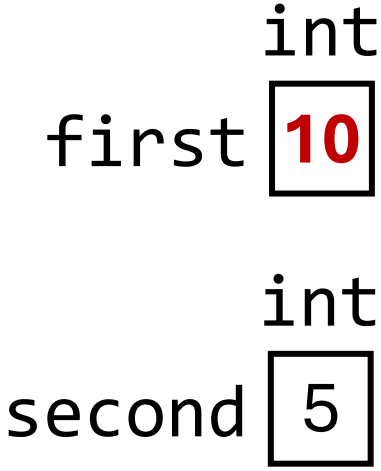
```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```

10



Memory:

Variable	Address	Value
first	0x1000	10
second	0x1004	5
	0x1008	

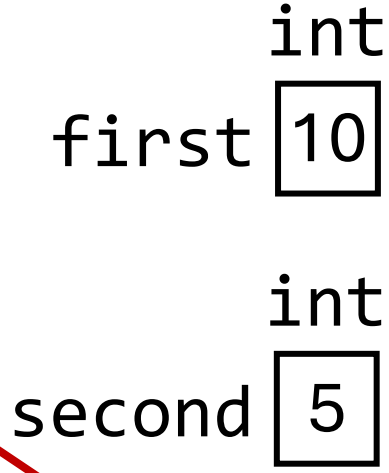
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	10
second	0x1004	5
	0x1008	

Equivalent to: `second = second - 3;`

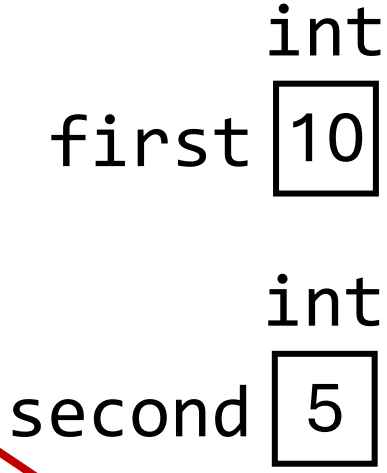
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	10
second	0x1004	5
	0x1008	

Equivalent to: `second = second - 3;`

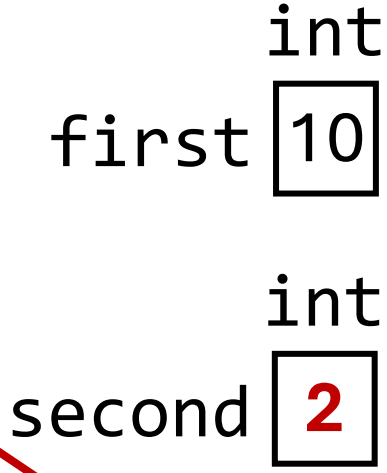
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	10
second	0x1004	2
	0x1008	

Equivalent to: `second = second - 3;`

Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```

int
first 10

int
second 2

Memory:

Variable	Address	Value
first	0x1000	10
second	0x1004	2
	0x1008	

Equivalent to: `second = second + 1;`

Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```

int
first 10

int
second 2

Memory:

Variable	Address	Value
first	0x1000	10
second	0x1004	2
	0x1008	

Equivalent to: second = second + 1;

3

Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```

int
first 10

int
second 3

Memory:

Variable	Address	Value
first	0x1000	10
second	0x1004	3
	0x1008	

Equivalent to: second = second + 1;

3

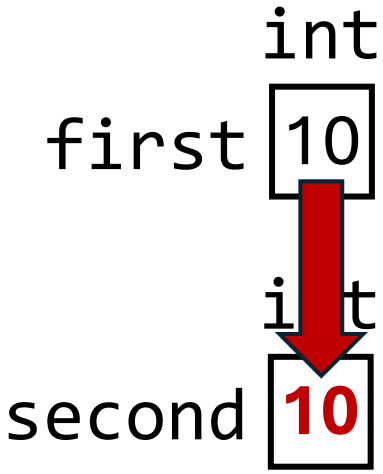
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	10
second	0x1004	**10**
	0x1008	

a = b++; first assigns the current value of **b** to **a** and *then* increments the value of **b**.

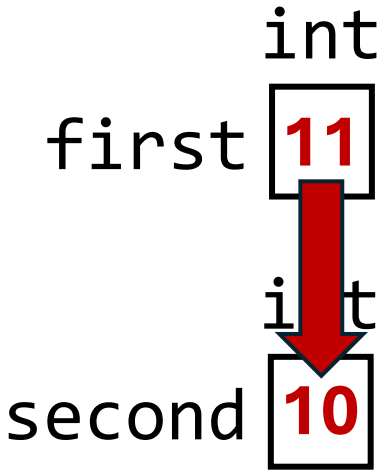
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	11
second	0x1004	10
	0x1008	

a = b++; first assigns the current value of **b** to **a** and *then* increments the value of **b**.

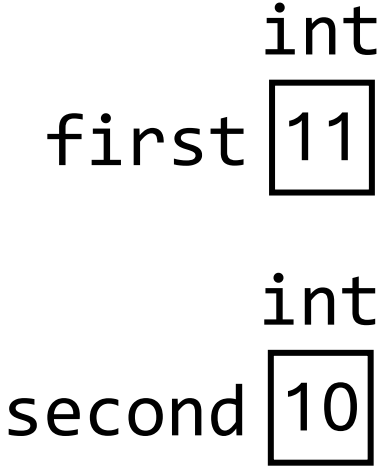
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	11
second	0x1004	10
	0x1008	

a = ++b; first increments the value of **b** and *then* assigns the updated value of **b** to **a**.

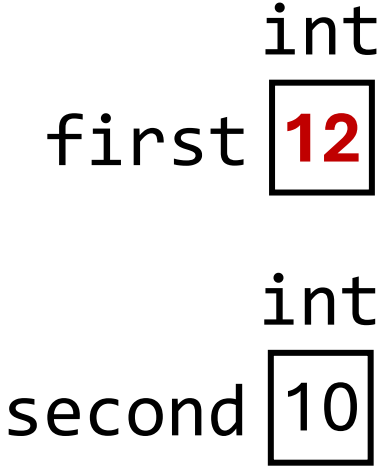
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	12
second	0x1004	10
	0x1008	

a = ++b; first increments the value of **b** and *then* assigns the updated value of **b** to **a**.

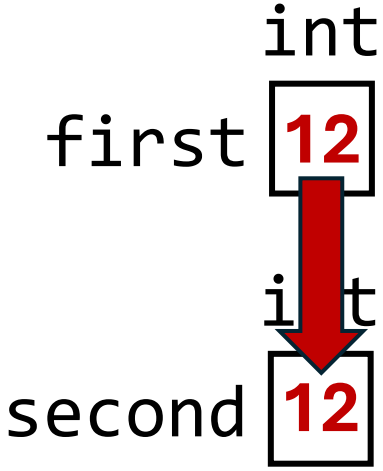
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	12
second	0x1004	12
	0x1008	

a = ++b; first increments the value of **b** and *then* assigns the updated value of **b** to **a**.

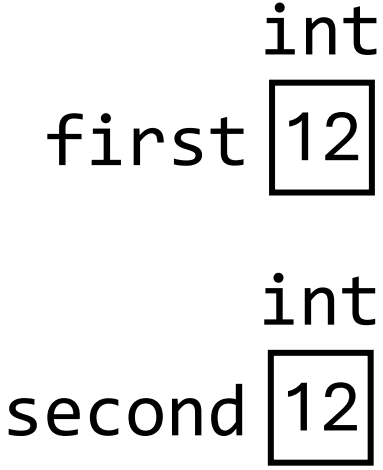
Tracing Example

```
int main() {
  int first = 1;
  int second = 5;

  first = second * 2;
  second -= 3;
  second++;

  second = first++;
  second = ++first;

  return 0;
}
```



Memory:

Variable	Address	Value
first	0x1000	12
second	0x1004	12
	0x1008	

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```

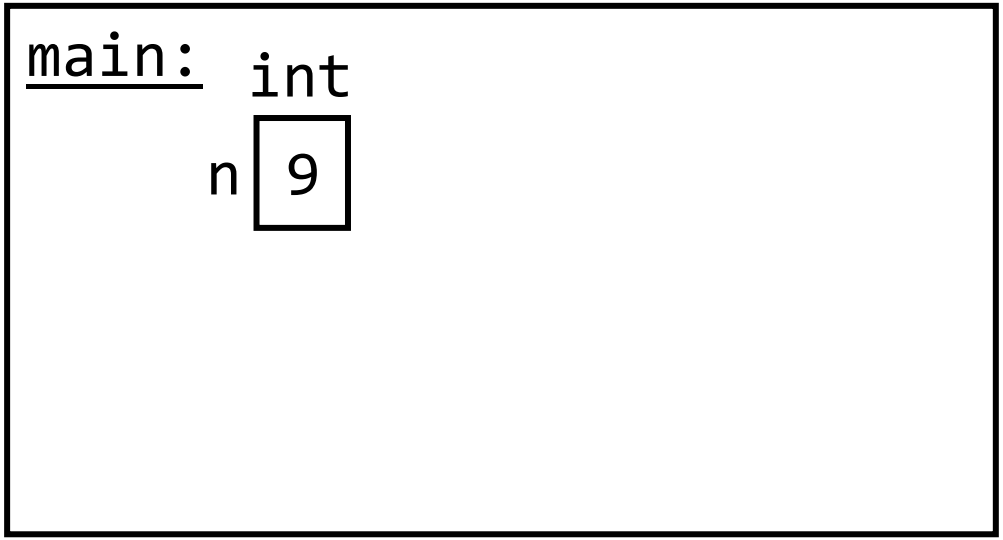
main:

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```

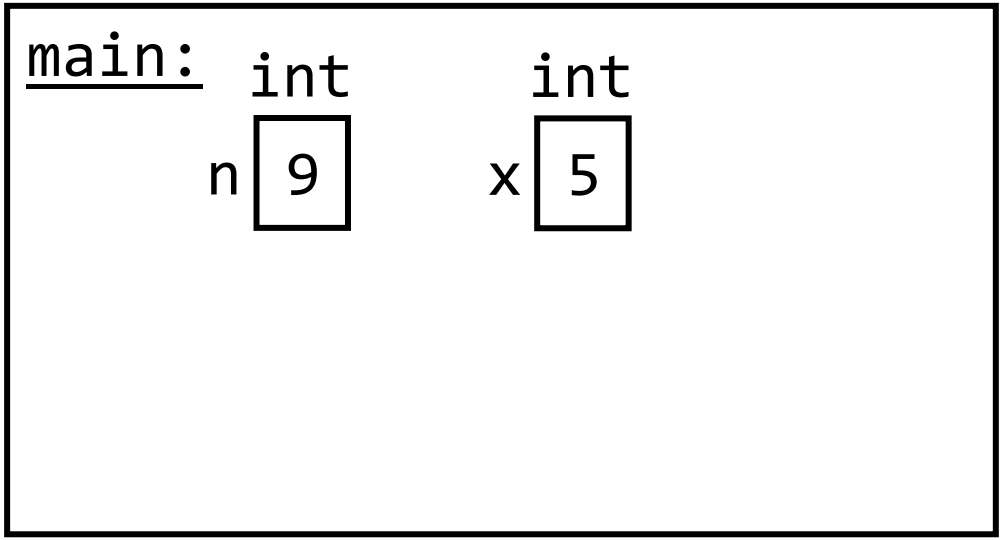


Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```

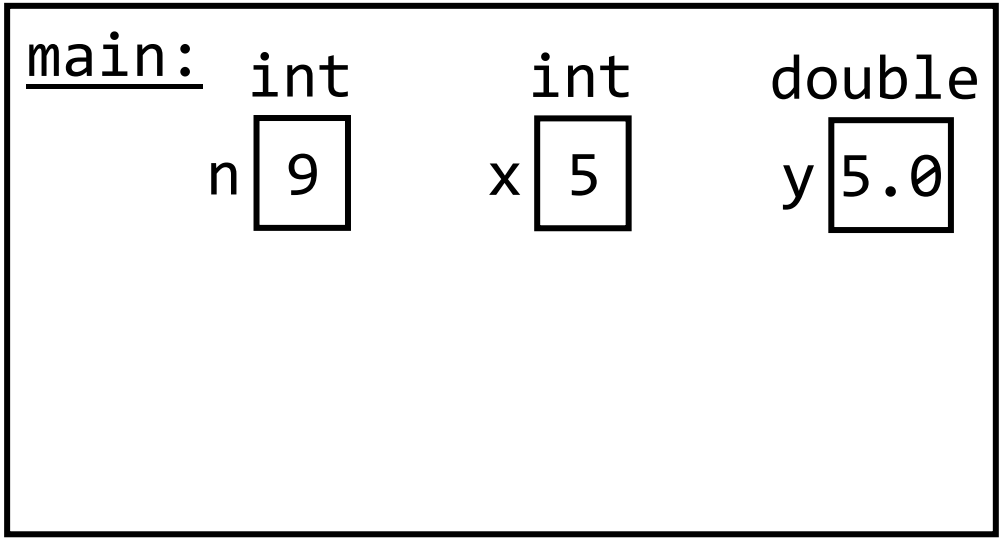


Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```

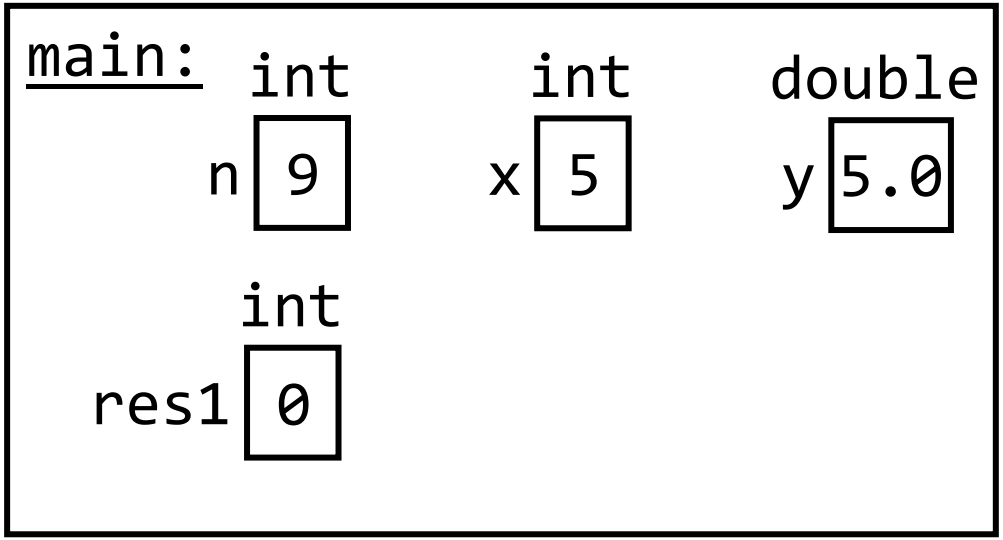


Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```

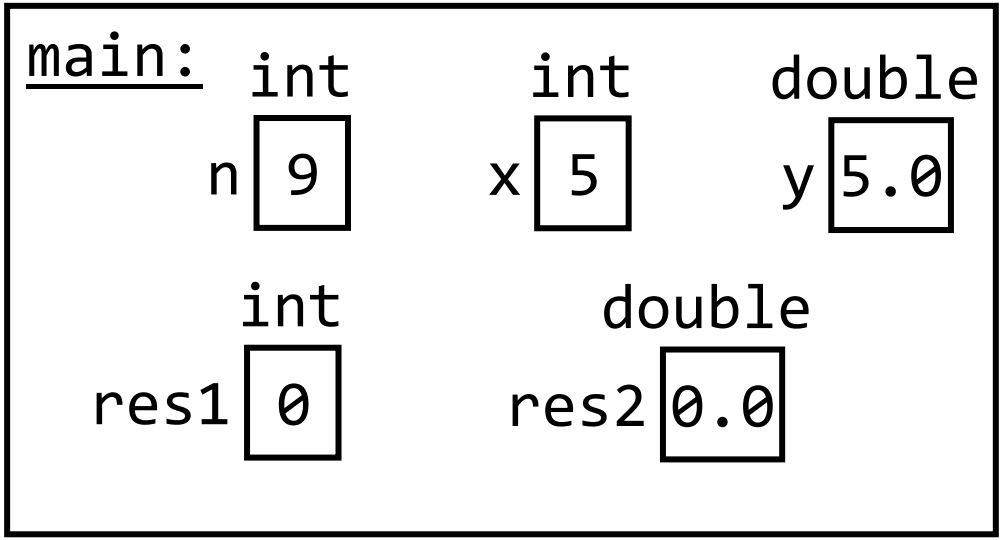


Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```

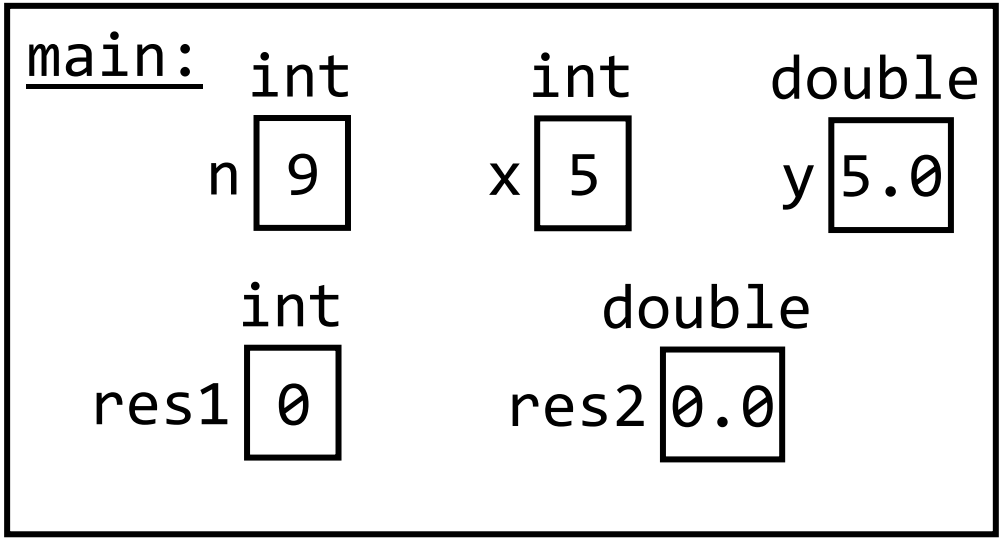


Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



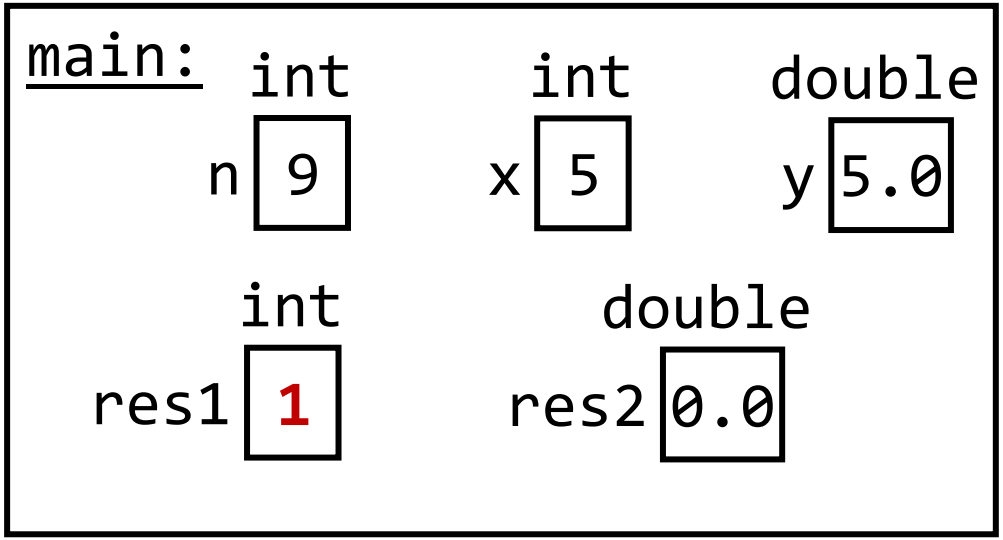
res1 = n / x = 9 / x = 9 / 5 = 1.8

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



res1 = n / x = 9 / x = 9 / 5 = 1.8

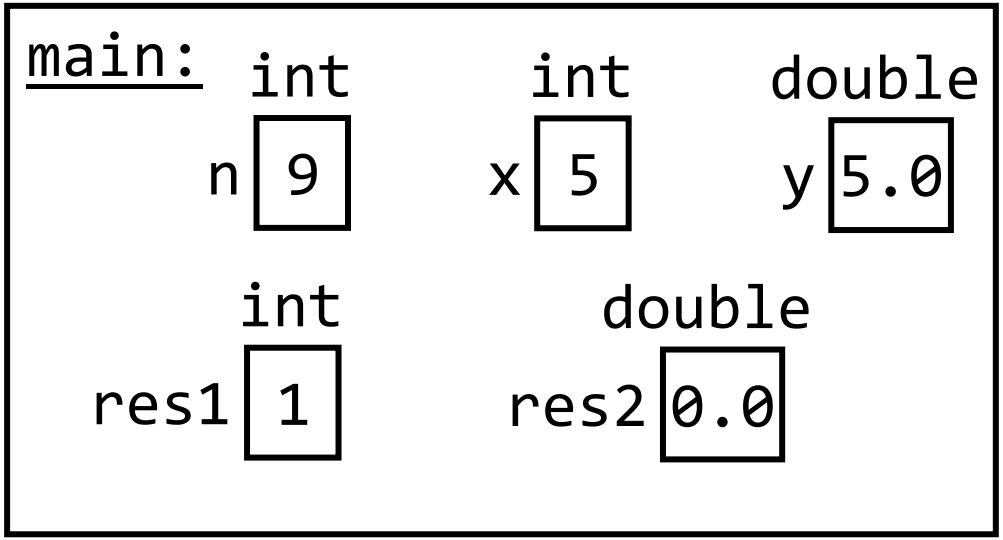
When *both* operands are integers, **integer division** is performed, and the result is truncated, not rounded.

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



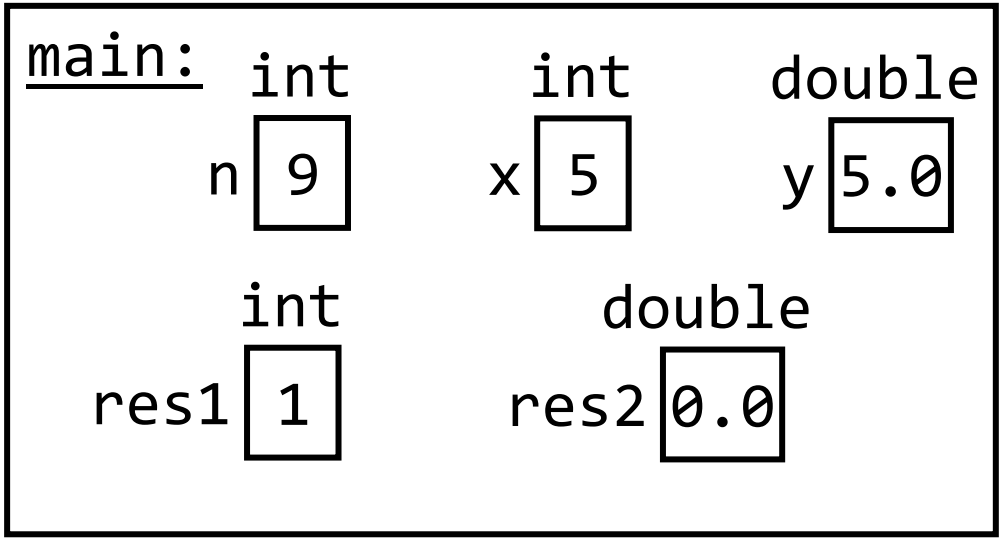
res1 = n / y = 9 / y = 9 / 5.0 = 1.8

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



`res1 = n / y = 9 / y = 9 / 5.0 = 1.8`

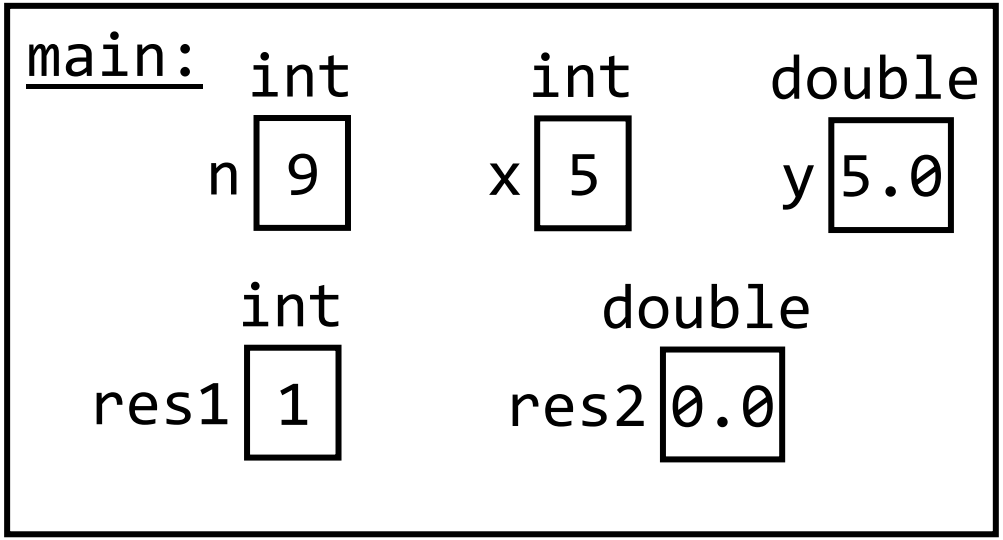
Since one operand is a double, the result is 1.8, but assigning 1.8 to an integer truncates the fractional part.

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



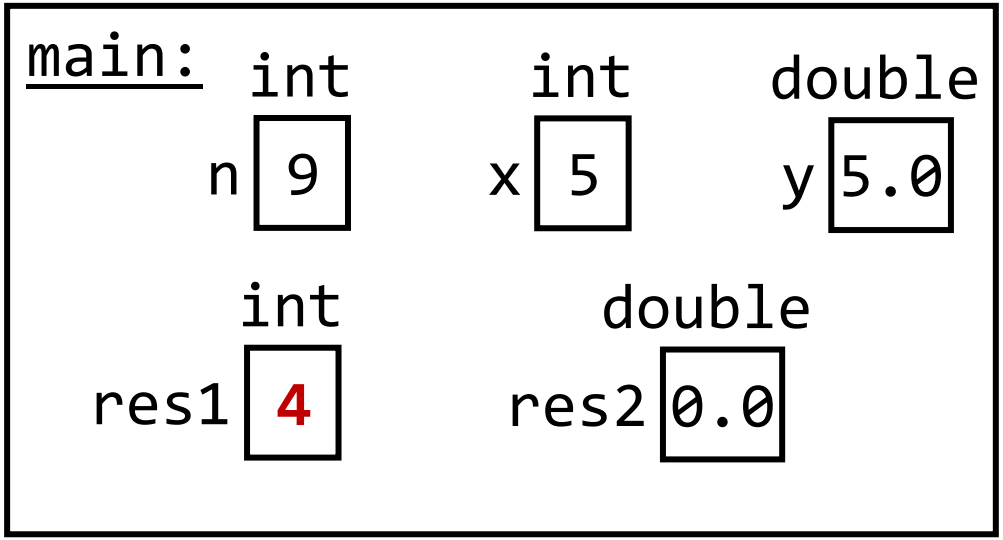
res1 = n % x = 9 % x = 9 % 5 = 4

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



`res1 = n % x = 9 % x = 9 % 5 = 4`

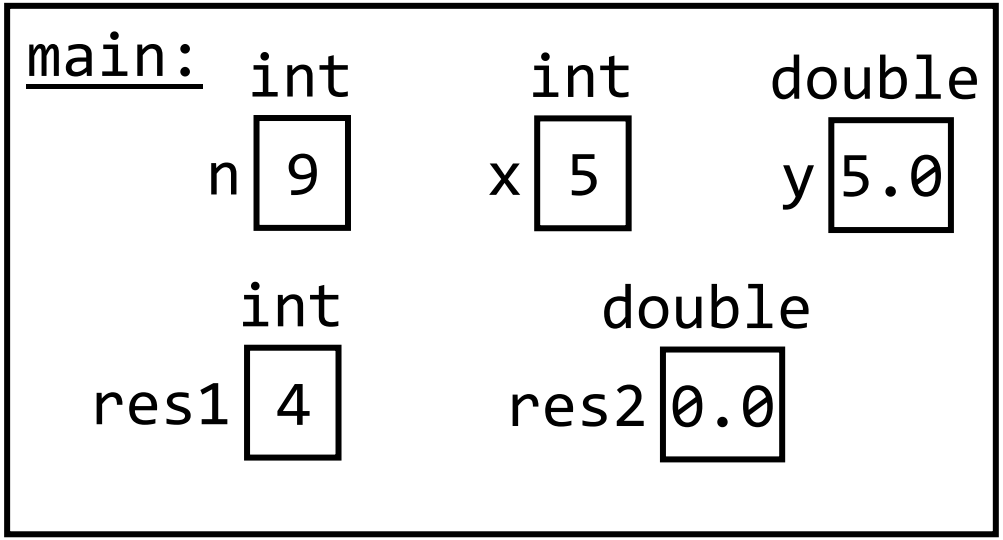
$9/5 = 1.8$, or 1 remainder 4, or $1\frac{4}{5}$. The % operator gives the **remainder** of integer division.

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



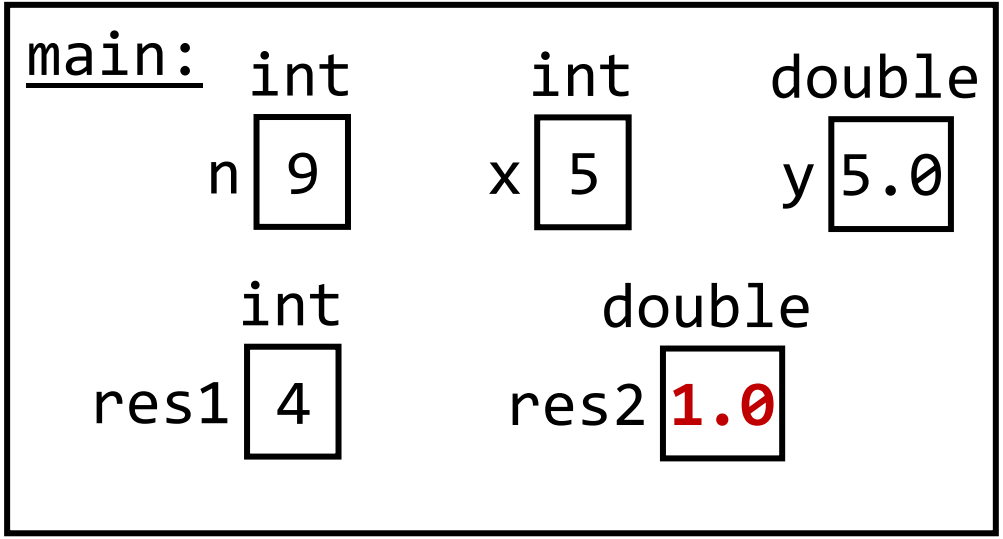
res2 = n / x = 9 / x = 9 / 5 = 1.8

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



`res2 = n / x = 9 / x = 9 / 5 = 1.8`

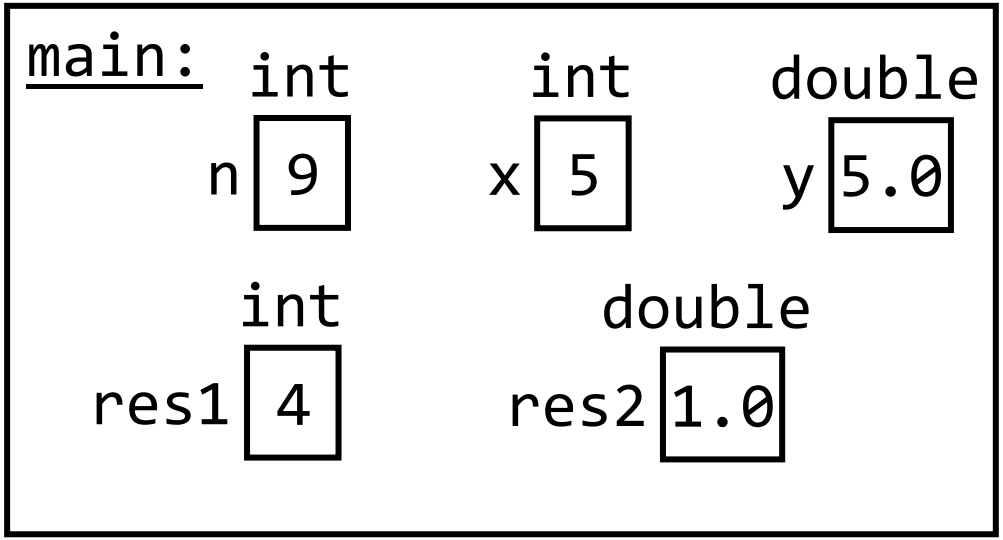
Again, *both* operands are integers, so the result is 1.
The value of `res2` will be 1.0 instead of 1.8.

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



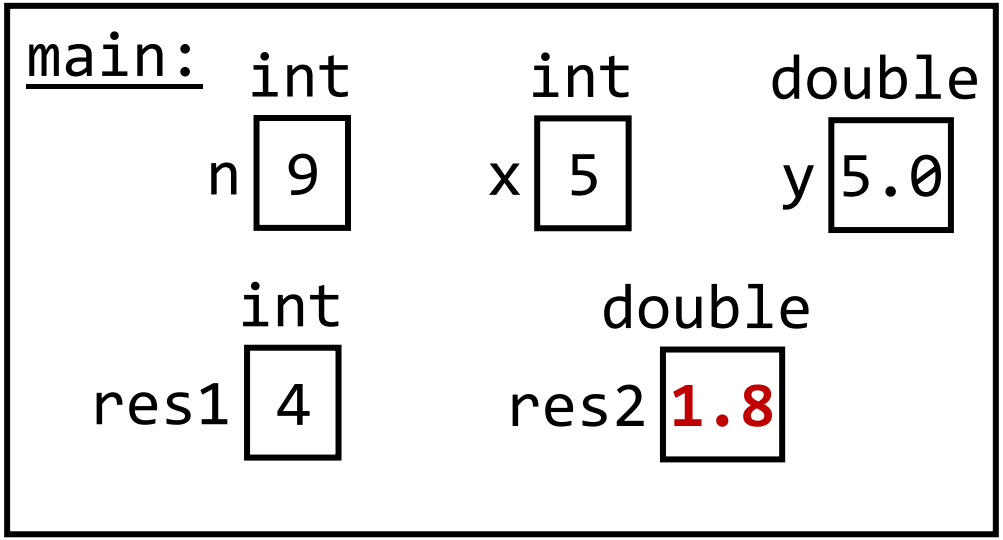
`res2 = n / y = 9 / y = 9 / 5.0 = 1.8`

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



`res2 = n / y = 9 / y = 9 / 5.0 = 1.8`

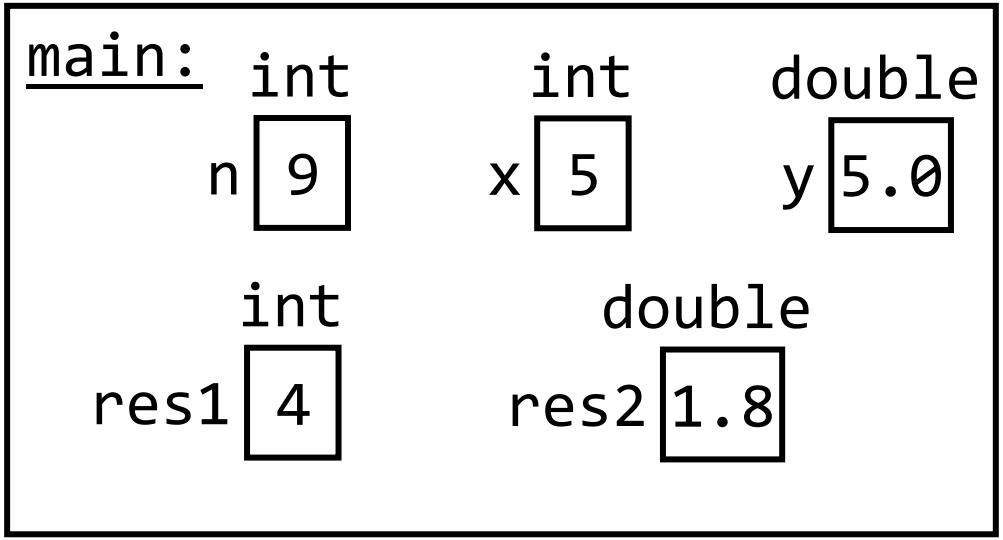
When **at least one** operand is a double, the result is a double, so the value 1.8 is assigned to `res2`.

Tracing Example: precision

```
int main() {
  int n = 9;
  int x = 5;
  double y = 5.0;
  int res1 = 0;
  double res2 = 0.0;

  res1 = n / x;
  res1 = n / y;
  res1 = n % x;
  res2 = n / x;
  res2 = n / y;

  return 0;
}
```



Key Takeaway: To avoid truncation, *at least one* operand must be a double, and the result of the operation must be assigned to a *double* variable.

06: Logical and Relational Operators

Anthony Estey
CSC 115: Fundamentals of Programming: II

Operations

Arithmetic Operators (produce a numerical value)	
`+ - / *`	Basic arithmetic operations
`%`	Remainder after integer division (ex: 7%3 = 1)
Shorthand Operators (combine assignment with an arithmetic operation)	
`+= -= *= /=`	Shorthand for updating a variable (x+=1 is the same as x=x+1)
`++ --`	Increment and decrement a variable by 1
Relational Operators (produce a boolean value: true or false)	
`< > <= >=`	Compares whether a value is greater or less than another
`== !=`	Checks whether two values are equal or not equal
Logical Operators (produce a boolean value: true or false)	
`&& || !`	Logical **and**, **or**, and **not** (to combine boolean expressions)

Operations

Arithmetic Operators (produce a numerical value)	
`+ - / *`	Basic arithmetic operations
`%`	Remainder after integer division (ex: 7%3 = 1)
Shorthand Operators (combine assignment with an arithmetic operation)	
`+= -= *= /=`	Shorthand for updating a variable (x+=1 is the same as x=x+1)
`++ --`	Increment and decrement a variable by 1
Relational Operators (produce a boolean value: true or false)	
`< > <= >=`	Compares whether a value is greater or less than another
`== !=`	Checks whether two values are equal or not equal
Logical Operators (produce a boolean value: true or false)	
`&& || !`	Logical **and**, **or**, and **not** (to combine boolean expressions)

Operations

Arithmetic Operators (produce a numerical value)	
`+ - / *`	Basic arithmetic operations
`%`	Remainder after integer division (ex: 7%3 = 1)
Shorthand Operators (combine assignment with an arithmetic operation)	
`+= -= *= /=`	Shorthand for updating a variable (x+=1 is the same as x=x+1)
`++ --`	Increment and decrement a variable by 1
Relational Operators (produce a boolean value: true or false)	
`< > <= >=`	Compares whether a value is greater or less than another
`== !=`	Checks whether two values are equal or not equal
Logical Operators (produce a boolean value: true or false)	
`&& || !`	Logical **and**, **or**, and **not** (to combine boolean expressions)

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

main:

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

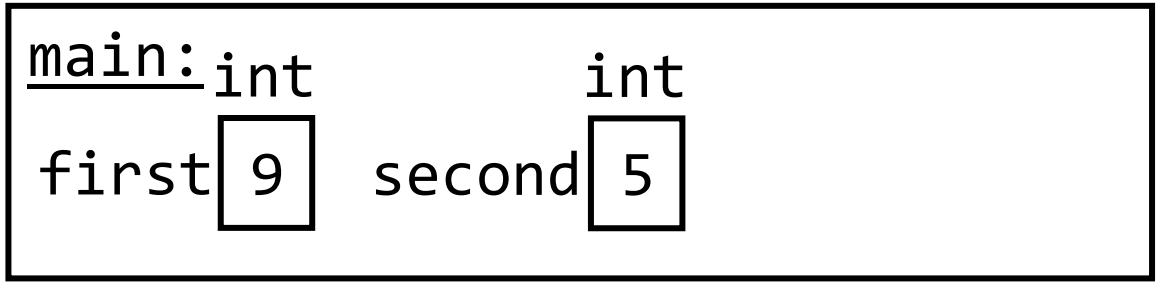


Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

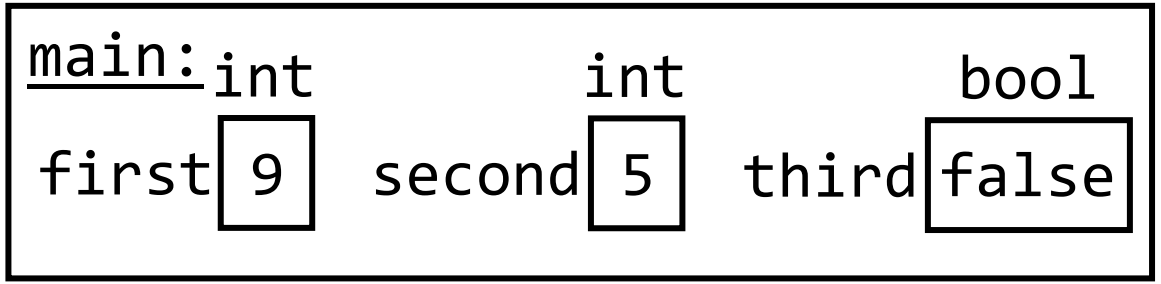


Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

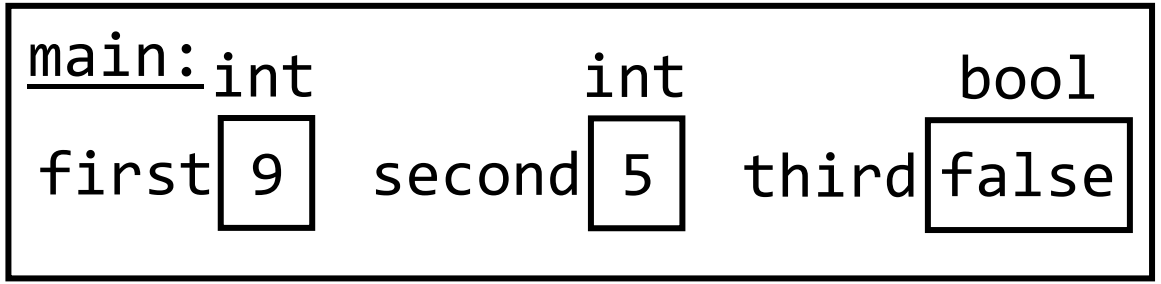


Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

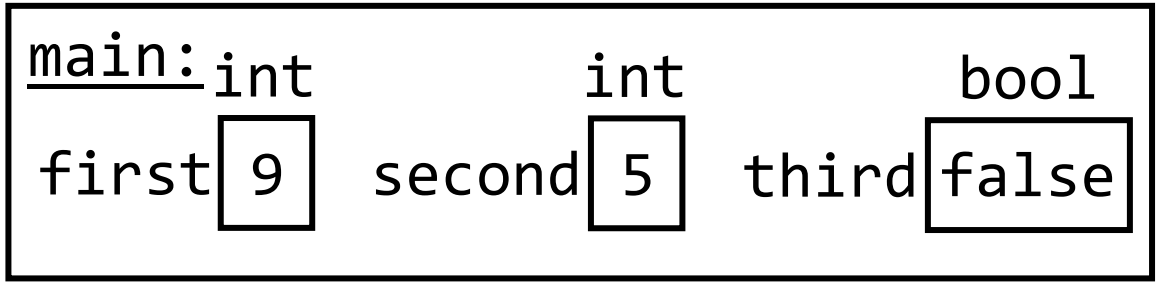


Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



first > second = 9 > second = 9 > 5 = true

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

true



main:	int	int	bool
	first 9	second 5	third true

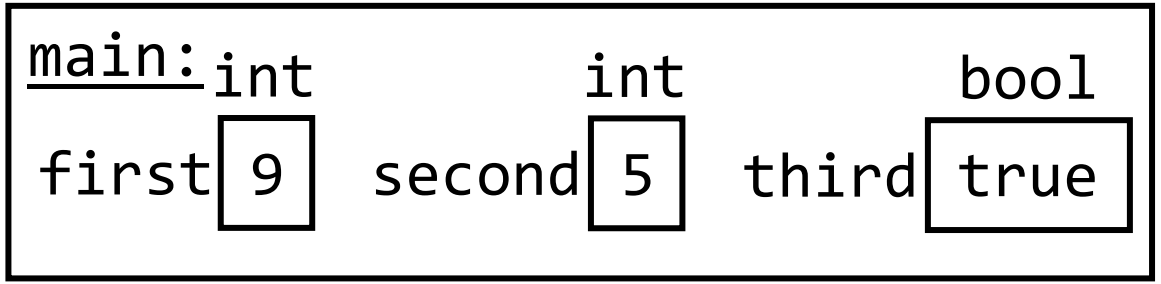
first > second = 9 > second = 9 > 5 = true

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

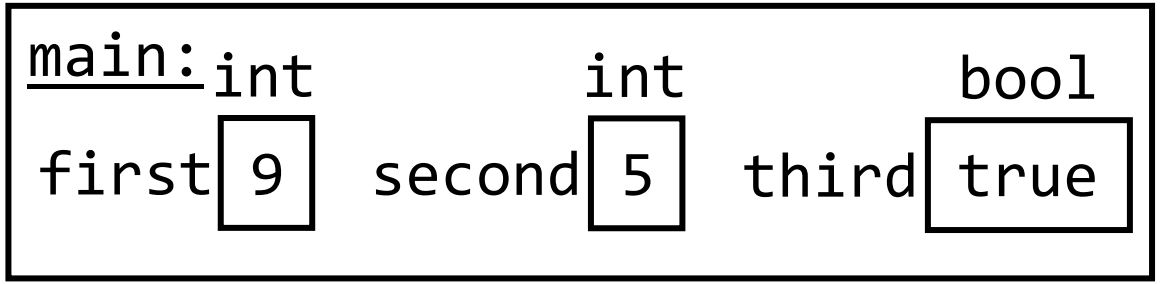


Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



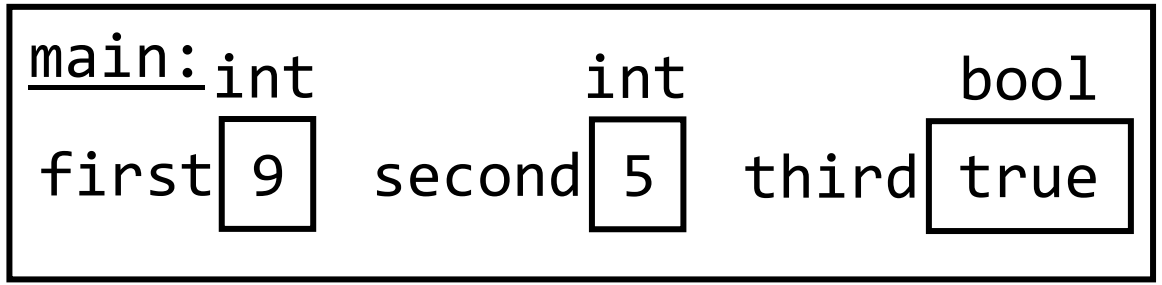
first > second && second < 0

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



true

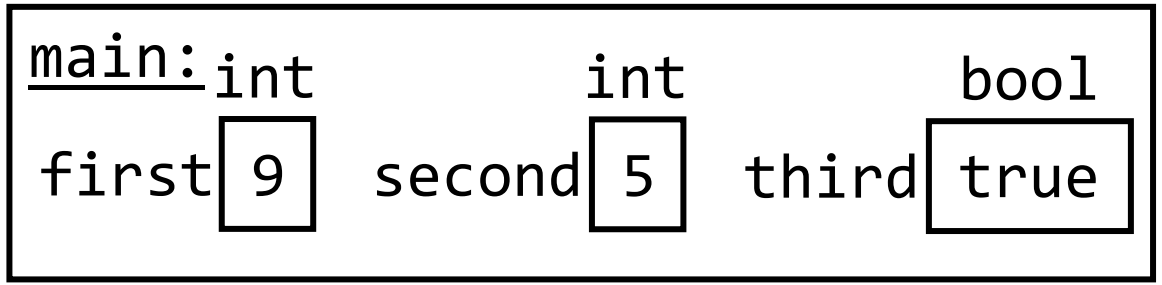
first > second && second < 0

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



true

false

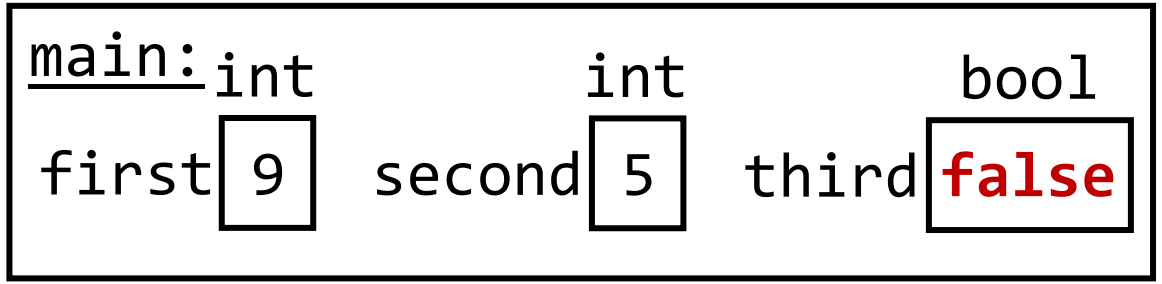
first > second && second < 0

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



false

true

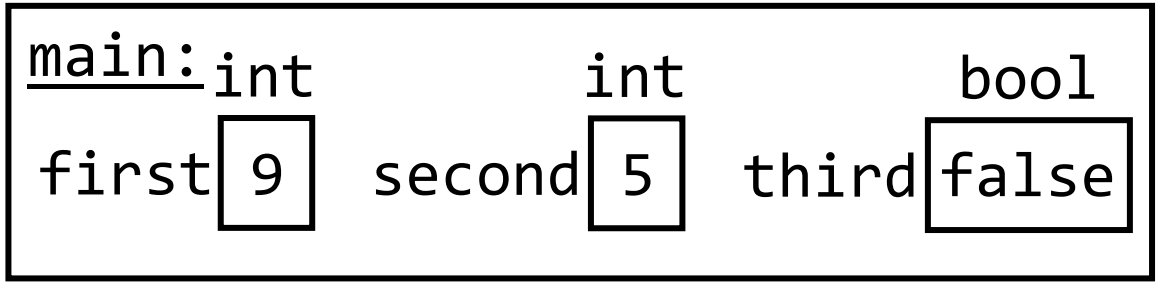
false

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

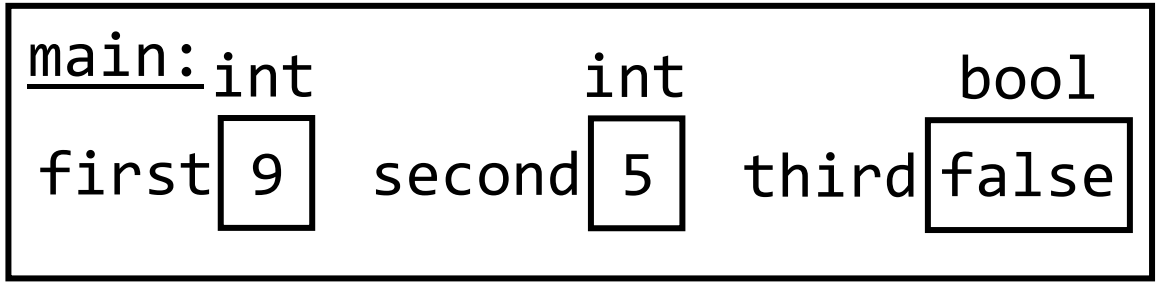


Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



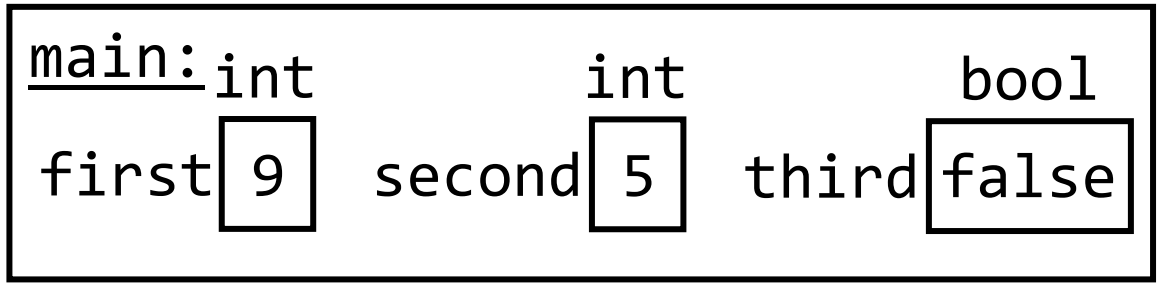
second < 0 && first > second

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



With **&&**, if any operand evaluates to **false**, no remaining operands are evaluated. This is called **short-circuit evaluation**

false

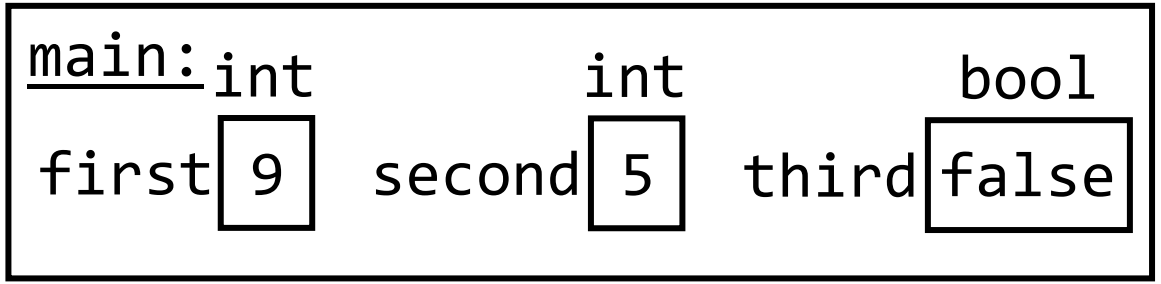
second < 0 && first > second

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



false

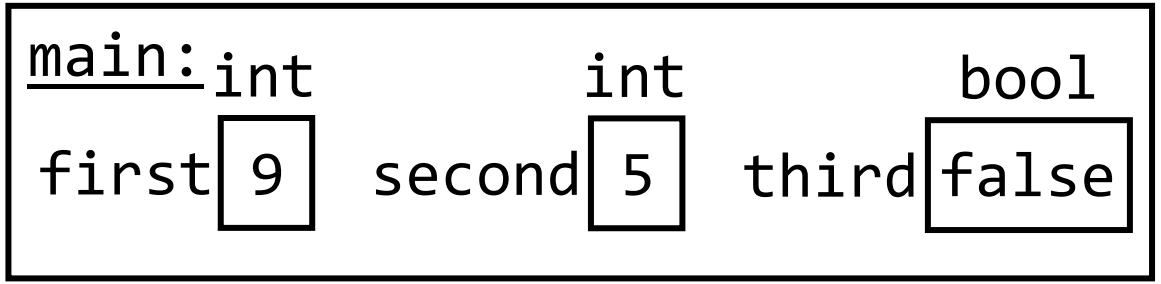
second < 0 && first > second

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

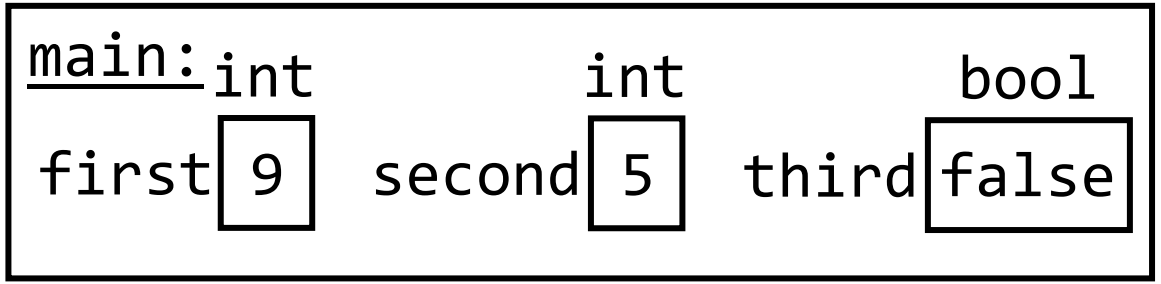


Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



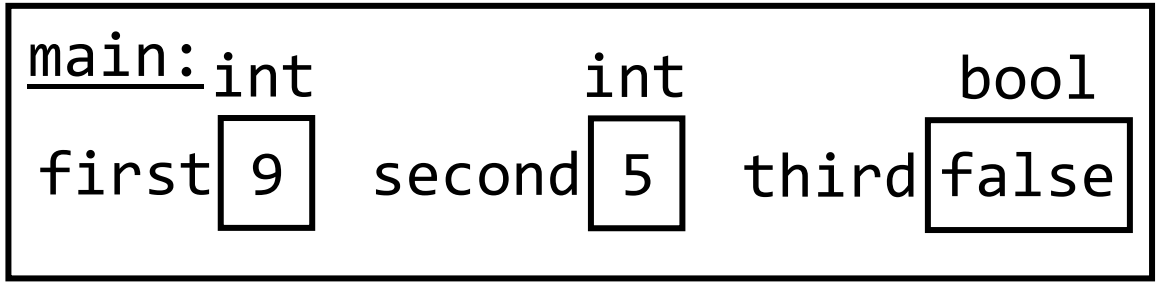
second < 0 || first > second

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



|| is the opposite: if any operand evaluates to **true**, no remaining operands are evaluated.

false

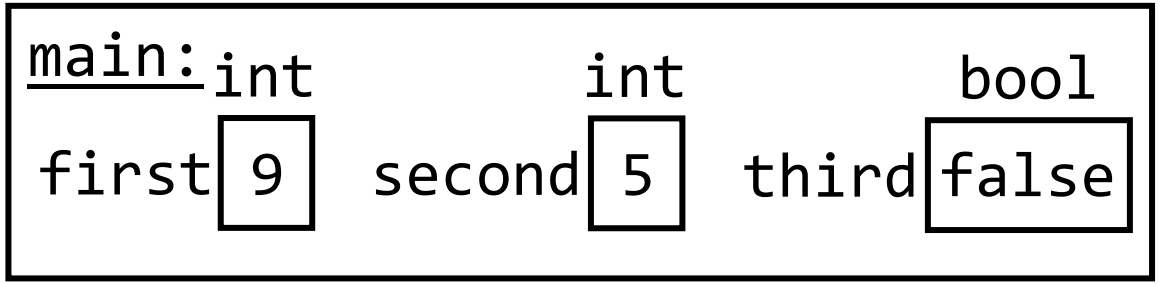
second < 0 || first > second

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



|| is the opposite: if any operand evaluates to **true**, no remaining operands are evaluated.

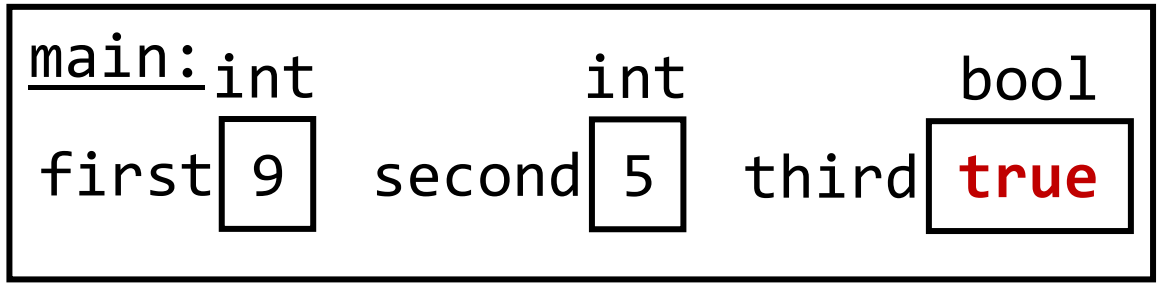
false
second < 0 || first > second
true

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



true

false

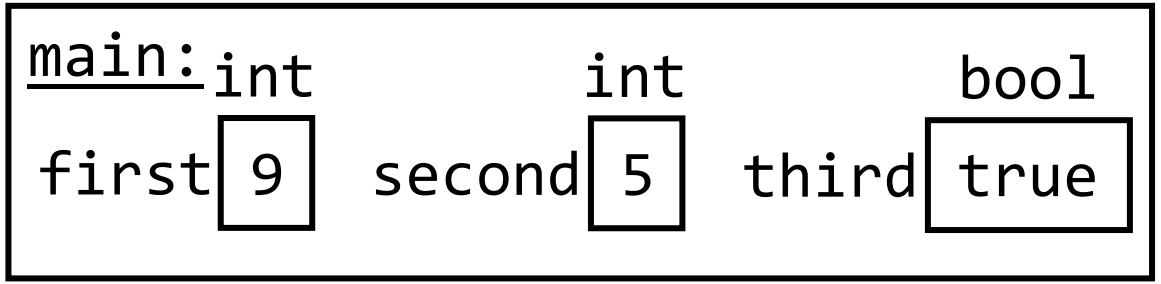
true

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

true



main:	int	int	bool
	first 9	second 5	third true

! negates a value (from **true** to **false** and vice versa). Notice the whole expression is placed in parentheses after the !

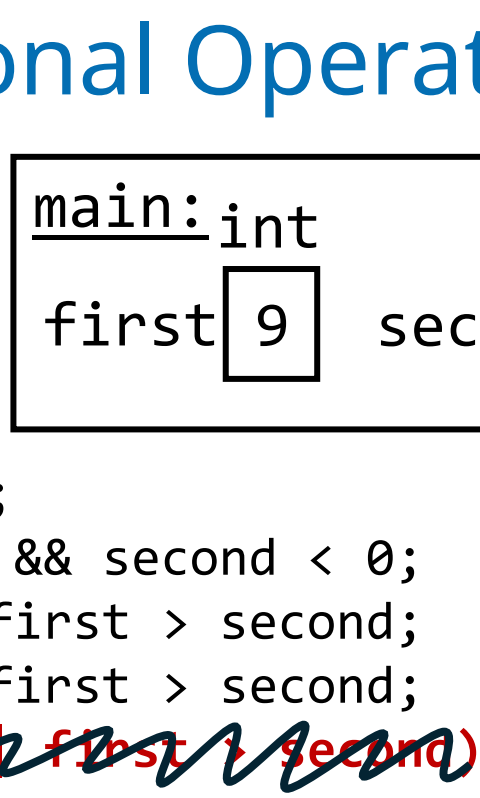
Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```

!true = false



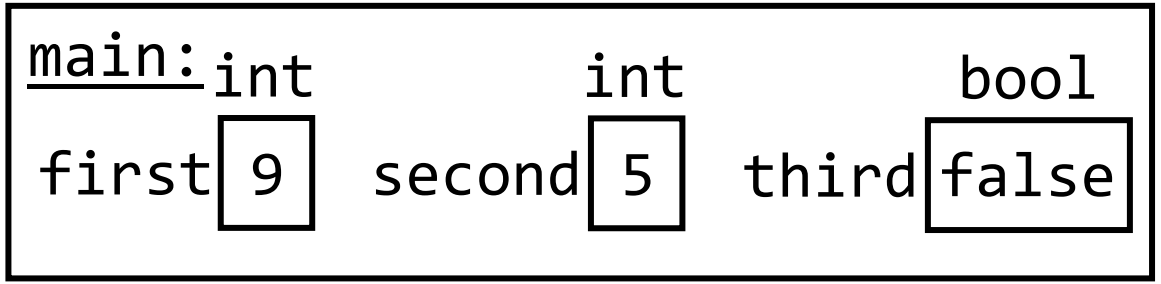
! negates a value (from **true** to **false** and vice versa). Notice the whole expression is placed in parentheses after the !

Logical & Relational Operator Trace

```
int main() {
  int first = 9;
  int second = 5;
  bool third = false;

  third = first > second;
  third = first > second && second < 0;
  third = second < 0 && first > second;
  third = second < 0 || first > second;
  third = !(second < 0 || first > second);

  return 0;
}
```



Key Takeaway: Short circuit evaluation means **&&** does not evaluate beyond the first **false**, and **||** past the first **true**.

007: Input and Output

Anthony Estey
CSC 115: Fundamentals of Programming: II

Outputting to the Terminal

- We often want to output something to the terminal
  - Sometimes called the *command-line interface*, *console* or *standard output,* other terms you may see when reading C++ documentation
- We will use **`std::cout`** to send output to the terminal
  - **cout**, short for character output, is the object that represents the terminal. When we send something to **cout**, it appears on screen.
  - **std::** is the namespace, and can be thought of as the label that tells the compiler where to find and how to use **cout**
    - The **std** specifically means the **standard library namespace**
    - The **::** is how we say to look inside this namespace

Using std::cout

```
std::cout << "The value " << x << " is too high" << std::endl;
```

- Important things to consider in the statement above:
  - << (insertion operator)
    - Sends data to the left, into **cout**, so it appears on screen
    - Chain multiple << operators to send text and variables in one statement (notice we alternate between text in quotes and variable values)
  - `std::endl` (short for end line)
    - Also sent with << as it's a special value from the standard library
    - Moves the cursor to the next line.
    - We end our statements with `std::endl`

Using std::cout

```
int x = 109;
std::cout << "The value " << x << " is too high" << std::endl;
```

- Output:

```
The value 109 is too high
```

- Notes:
  - Any text, numbers, names within double quotes (") are output as is
  - The values of any variables provided *outside* of quotes are output

Using std::cin

- Something we'll use less often in this course is **std::cin**
  - It allows us to read values we type in the terminal into our program
- For example:

```
int entry = 0;
std::cout << "Enter a value: ";
std::cin >> entry;
std::cout << "You entered: " << entry << std::endl;
```

Prompt the user to enter a value (notice no `std::endl`)

The value entered will be assigned to the `entry` variable

If the user types in 15. The output will be:
You entered: 15

08: Uniform Initialization

Anthony Estey
CSC 115: Fundamentals of Programming: II

Variable declaration and initialization

- The C++ programming language is a descendent of C
  - Many structures and styles from C have been inherited by C++

- Up until now, we have declared and initialized variables the way we would do it in C (and some other languages)
  - This works for the primitive variables we have used so far
  - The way we will initialize our variables from now on in C++ is called **uniform initialization**

Variable declaration and initialization

▶C style

```
int x = 5;
int age = 17;
double gpa = 8.2;
bool passed = true;
```

▶C++ style

```
int x{5};
int age{17};
double gpa{8.2};
bool passed{true};
```

C style: uses the assignment operator (=) to provide the initial value
C++ style: the initial value is provided within curly braces {}

Variable declaration and initialization

- **Problem**: Soon we will be working with more complex data types and built-in C++ collections as it's not easy to provide a simple initial value.
- Uniform initialization allows us to leave the braces empty, and C++ will provide a default value for us
- This value is what you might expect for simple types:
  - `int x{};`
  - `double y{};`
  - `bool z{};`

By default, variables of type `int` are initialized to 0, `double` to 0.0, and `bool` to false.

Variable declaration and initialization

- It also helps us avoid errors
- For example, we may mistakenly declare and initialize an int variable to floating point value (int x = 3.7;)
  - This isn't a compile error, the value 3.7 is truncated to 3
  - If this variable is used for calculations in our program, and we are expecting 3.7 to be used, our program will produce incorrect results
- This problem is solved with uniform initialization (i.e. int x{3.7};)
  - This does not compile correctly; the error is reported

Key Takeaways

- C-style initialization works for simple types like int or double, but we will use uniform initialization for all variable declarations in this course
  - Uniform initialization works consistently across all types, from simple types to the complex objects and collections we will see later in the course
  - The initial value is not always obvious for complex types, but empty braces {} give us a safe default value
- Uniform initialization protects us from accidental data loss
  - `int x{3.7};` produces a compile error, allowing us to find and fix the error much more quickly than if 3.7 was truncated

09: Casting

Anthony Estey
CSC 115: Fundamentals of Programming: II

Casting

- **Casting** is the process of converting a value from one data type to another.
- This is done using parentheses (), the cast operator.
- General form:

```
(type) value;
```

- Specific examples:

```
x = (int) 3.14;
```

In both examples, 3.14 is converted to the integer 3 before the assignment. This means x will always receive the value 3, even if x is declared as a double.

```
double pi = 3.14
x = (int) pi;
```

Casting

- Casting is important a number of reasons:
  - Without casting, integer division silently truncates, resulting in an incorrect final result with no warning
  - Casting gives us control over when and how conversion happens
- An example scenario that casting can solve:
  - Assume people are awarded a score between 1 and 5, and we wanted to calculate the average score.
  - We could calculate the sum of all scores with an integer
  - Dividing the integer sum by the integer number of participants is integer division, so the result is truncated!

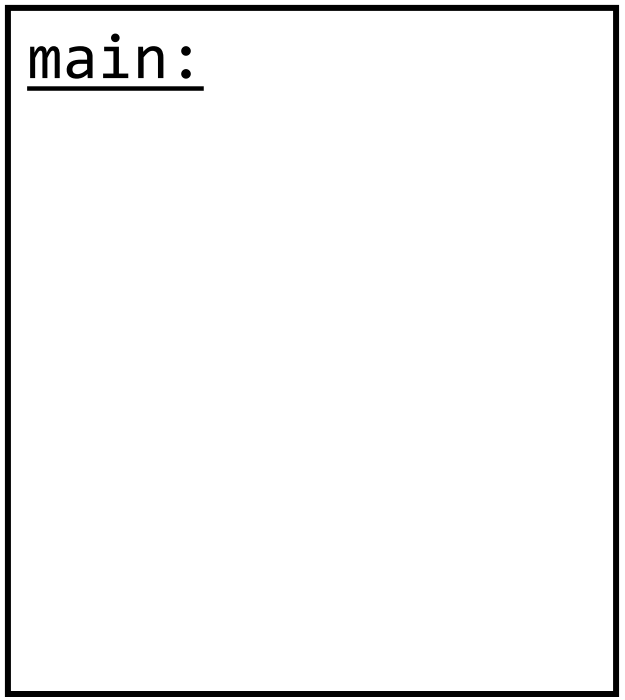
Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```



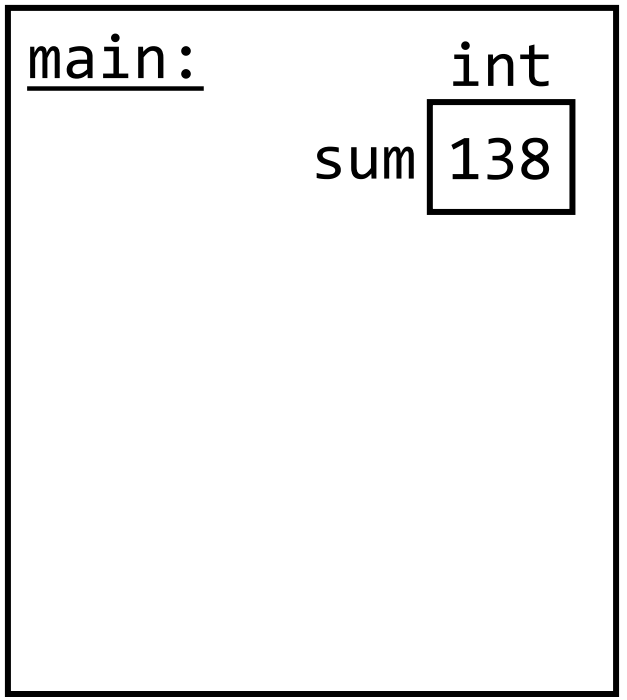
Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```



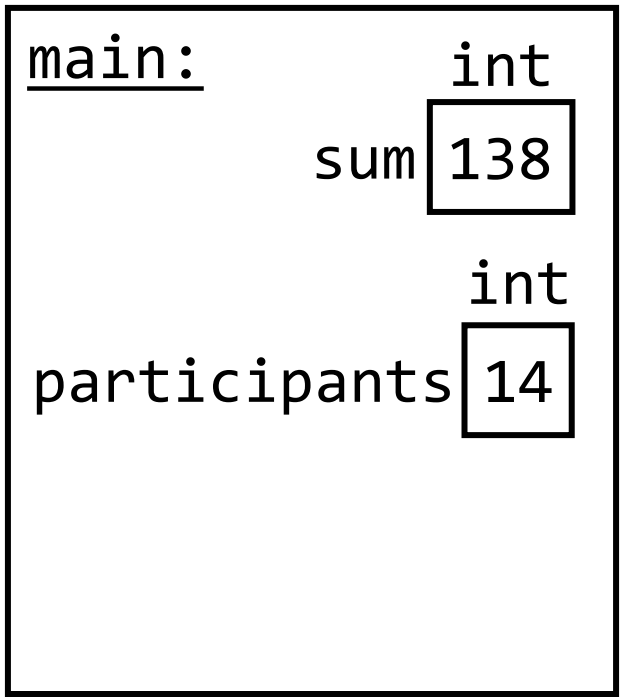
Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```



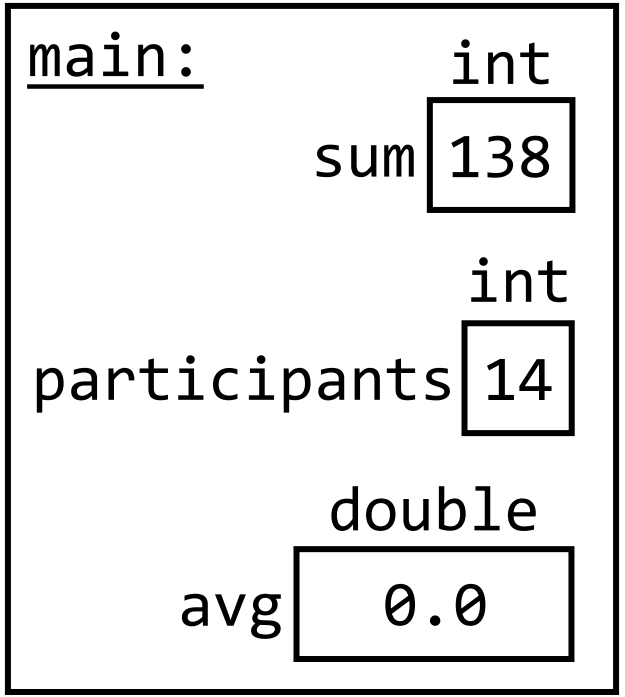
Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```



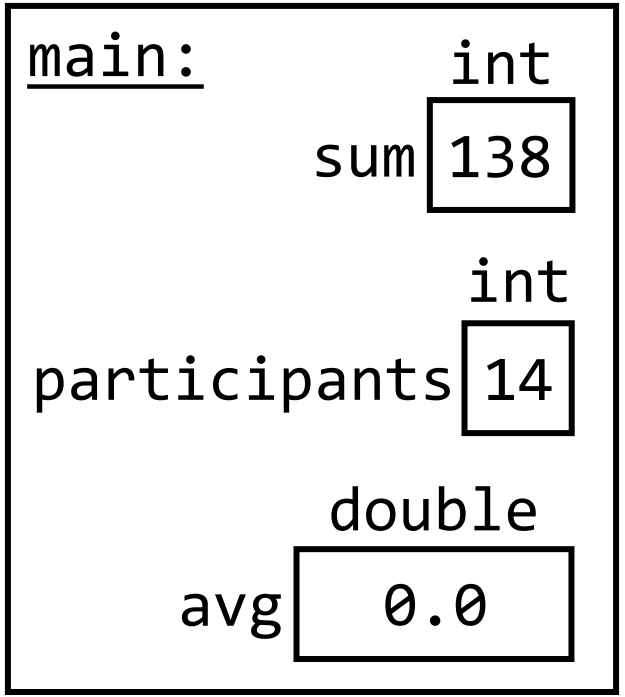
Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

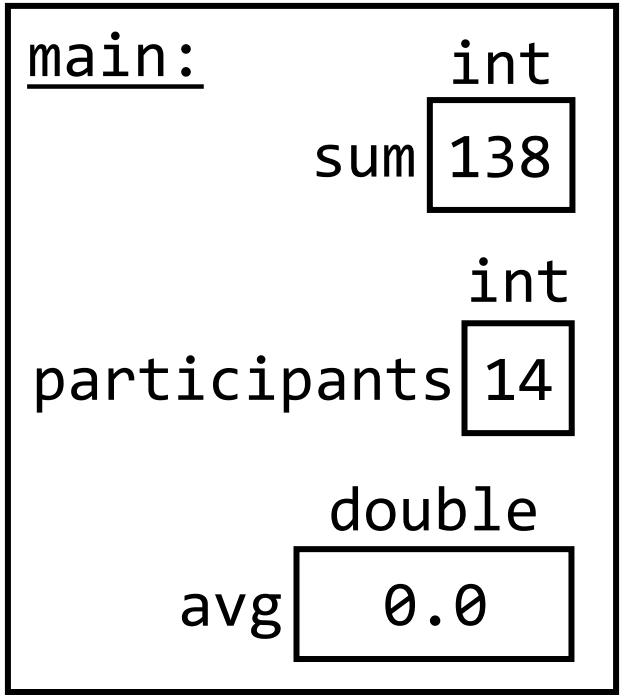


Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};
        138
  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```



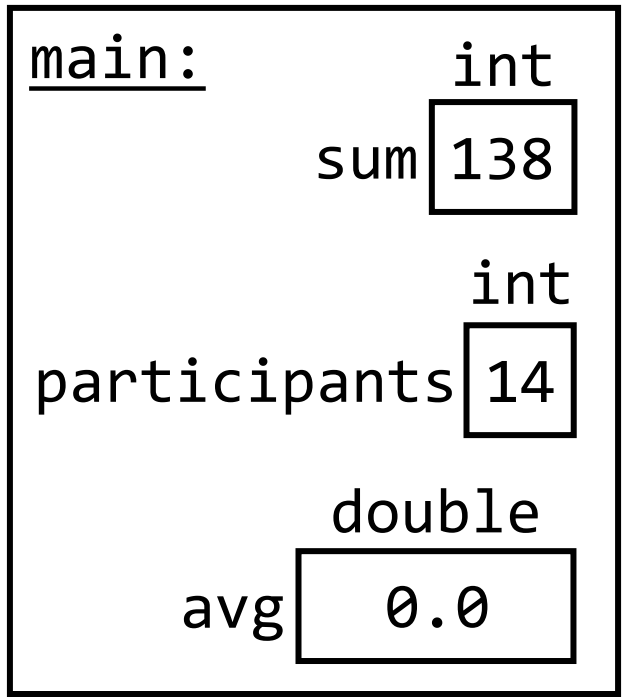
Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};
  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

138 / 14



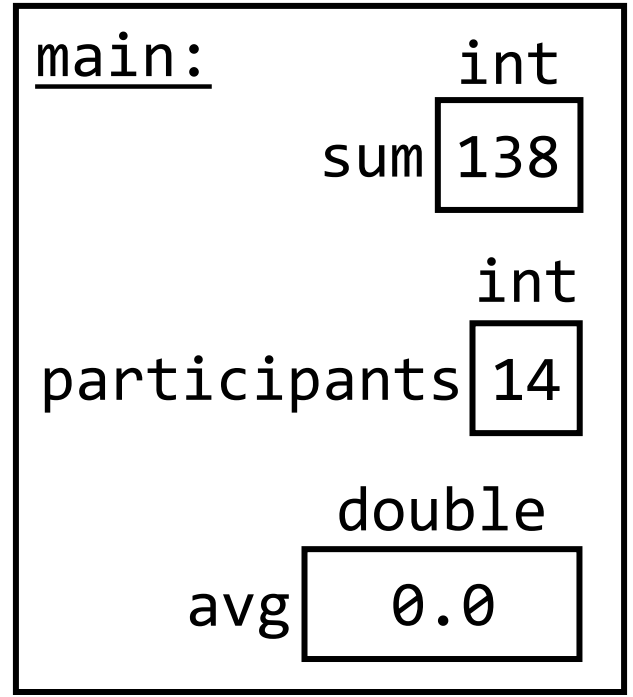
Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};
  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

138 14 = 9.85714…



Casting Example

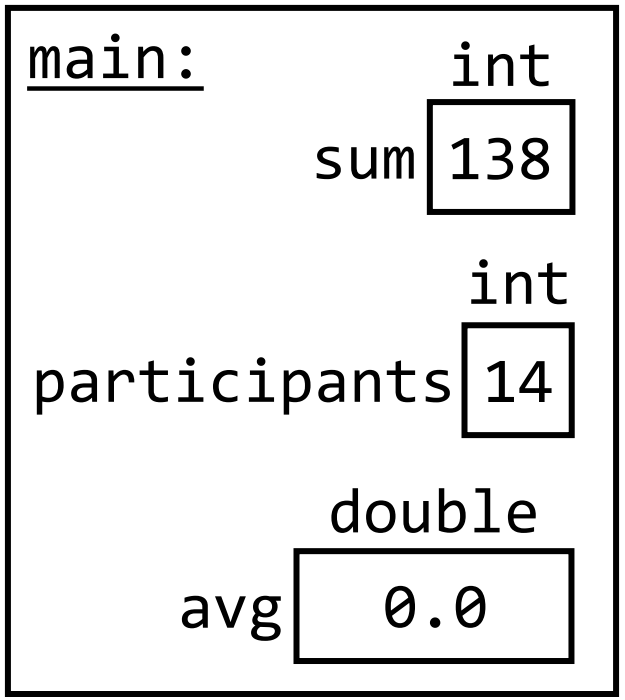
```
int main() {
  int sum{138};
  int participants{14};
  double avg{};
  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

138 14 = 9.85714...

Integer division!



Casting Example

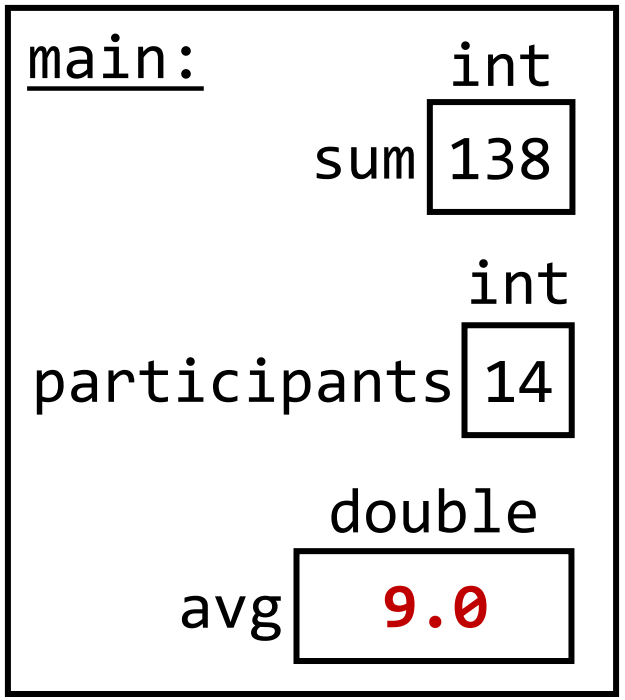
```
int main() {
  int sum{138};
  int participants{14};
  double avg{};
  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

138 14 = 9.85714...

Integer division!



Casting Example

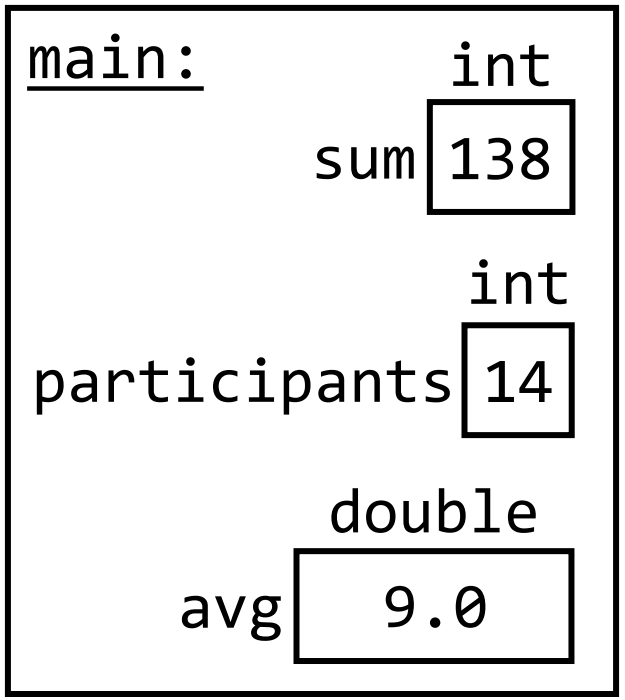
```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

Output: 9



Casting Example

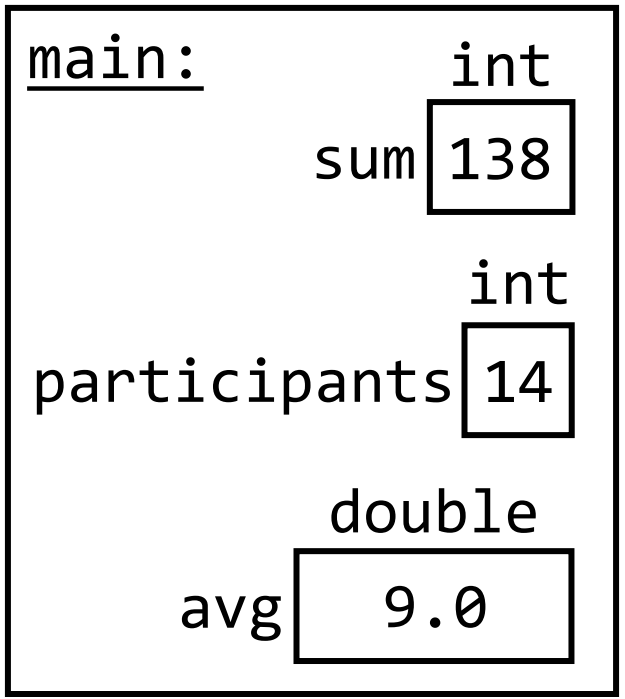
```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

Output: 9



Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

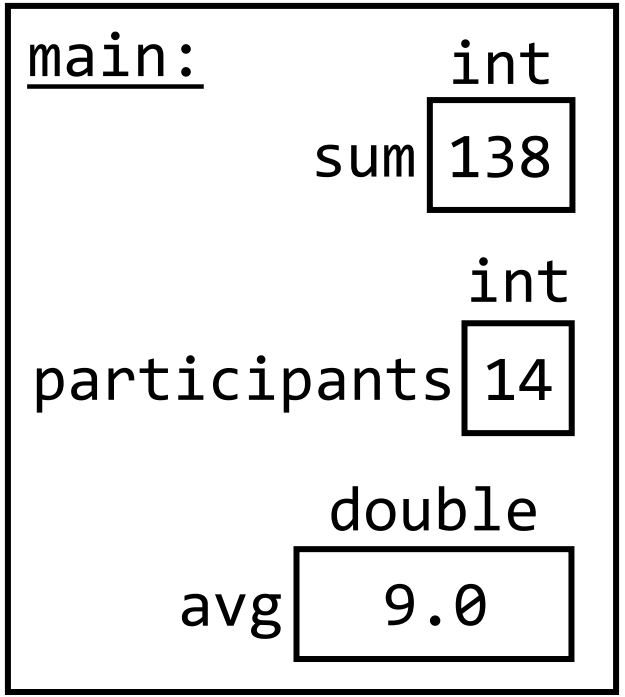
  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

138

Output: 9



Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

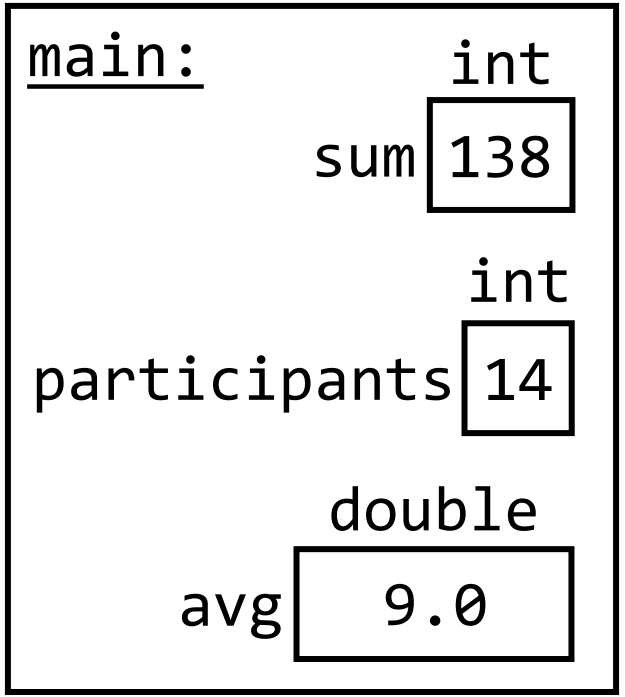
  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

138

Output: 9



Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

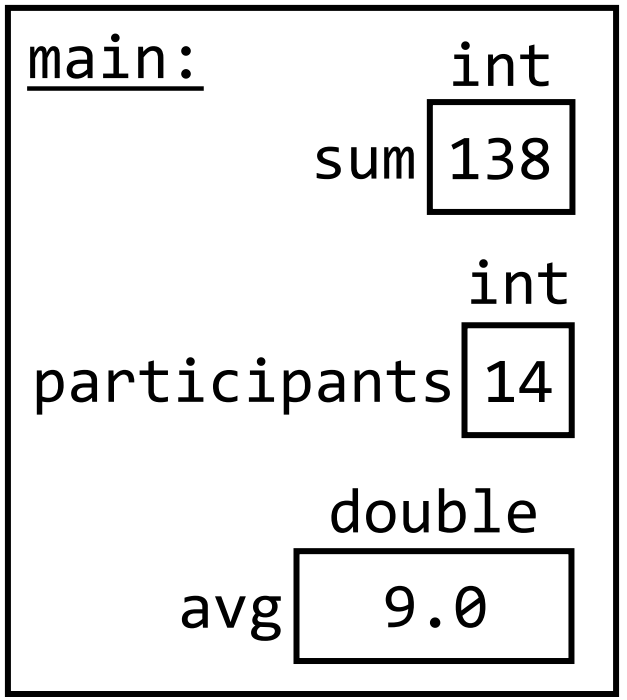
  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

138.0

Output: 9



Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

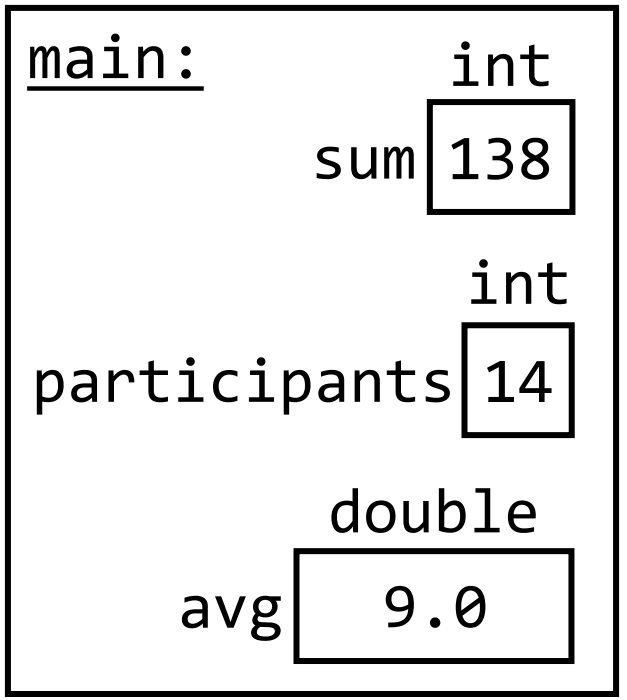
  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

138.0 / 14

Output: 9



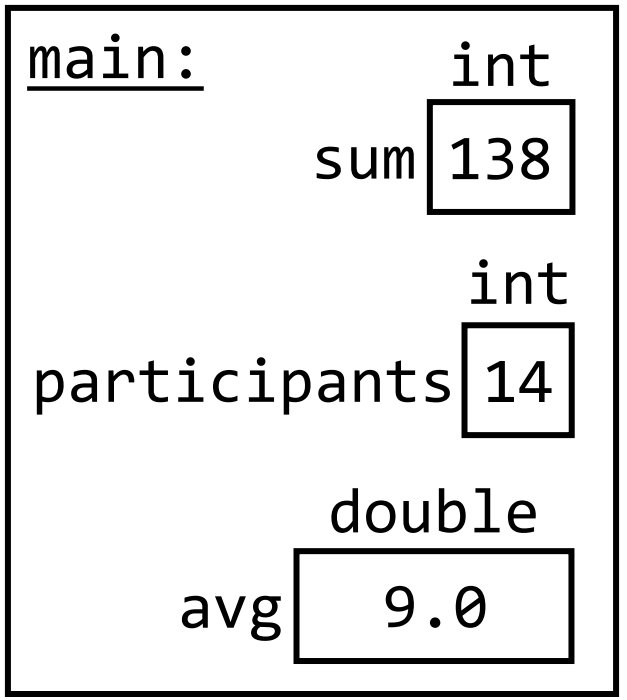
Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;
                138.0 /  14  = 9.85714…
  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

Output: 9



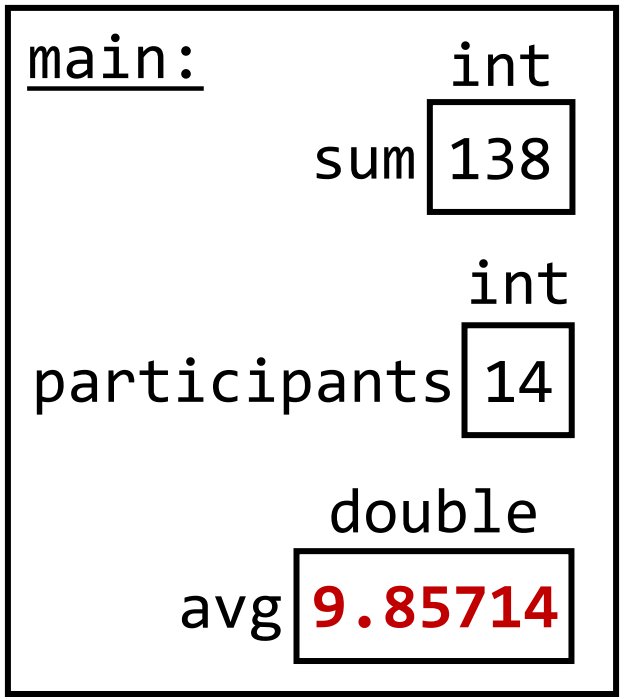
Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;
                138.0 /  14  = 9.85714…
  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

Output: 9



Casting Example

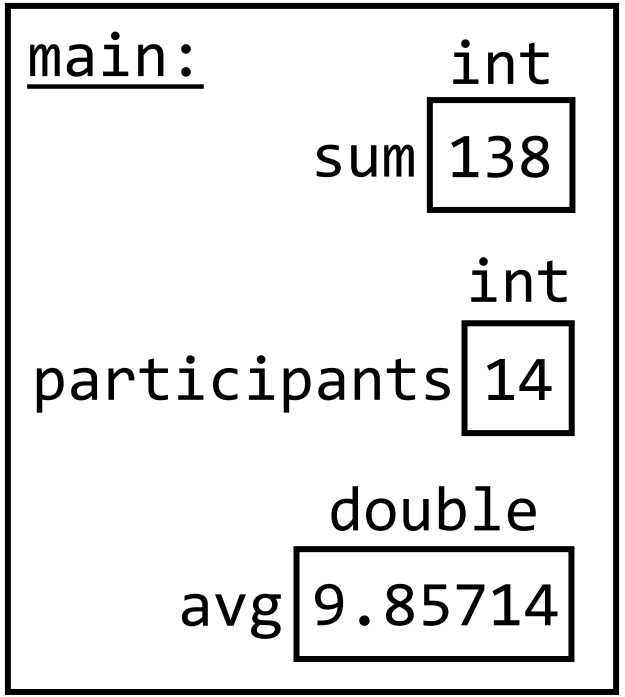
```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

Output: 9



Casting Example

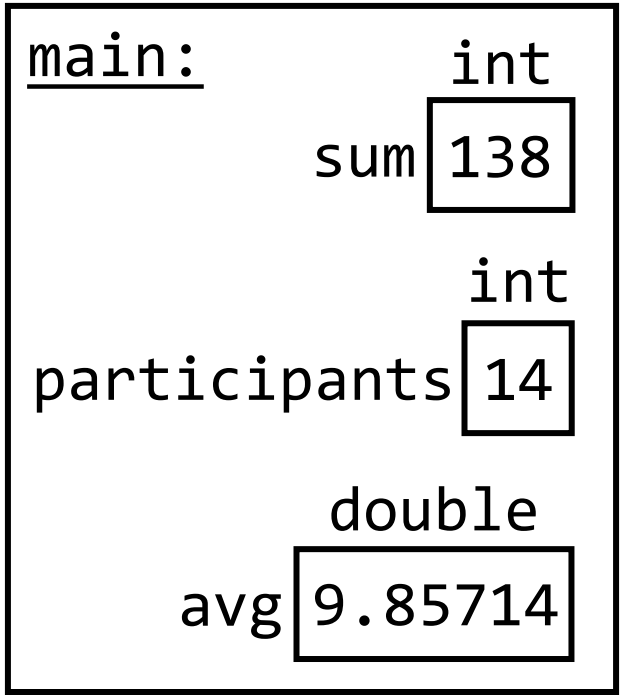
```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

Output: 9
9.85714



Casting Example

```
int main() {
  int sum{138};
  int participants{14};
  double avg{};

  avg = sum / participants;
  std::cout << avg << std::endl;

  avg = (double)sum / participants;
  std::cout << avg << std::endl;

  return 0;
}
```

Key Takeaway: Casting converts a value to a different type before the operation is performed.

Here, we used it to avoid unintended truncation.

```
Output:   9
          9.85714
```

10: If Statements

Anthony Estey
CSC 115: Fundamentals of Programming: II

If Statements

- As we know from Python, if statements allow us to specify a block of code to be executed *if* a condition is true

- General form:

```
if (condition1) {
    …
}

if (condition2) {
    …
    …
    …
}
```

If Statements

- As we know from Python, if statements allow us to specify a block of code to be executed *if* a condition is true
- Optionally, we can also include:
  - **`else if`**: test an additional condition when the previous one is false
  - **`else`**: specify a block of code to execute when all prior conditions are false
- General form:

```
if (condition1) {
    …
} else if (condition2) {
    …
} else if (condition3) {
    …
} else {
    …
}
```

Working with If Statements

- Differences between Python and C++:
  - Syntax:
    - In Python: **`if/elif/else`**. No parentheses around conditions
    - In C++: **`if/else if/else`**. Parentheses around conditions
  - Indentation vs. curly braces:
    - In Python, indentation is used to specify code blocks
    - In C++ it is the curly braces (the indentation is just a coding convention)
    - **General rule**: use curly braces and indentation. Relying on indentation alone in C++ will compile but the program may not behave as expected

Working with If Statements

- Other common mistakes:
  - Equality vs. assignment:
    - The assignment operator (=) assigns a value
    - The equality operator (==) checks for equality
- For example:

```
int x{0};

if (x = 5)  // assigns 5 to x, then evaluates to true (in
            // C++, non-zero integers are treated as true)

if (x == 5) // checks if x equals 5
```

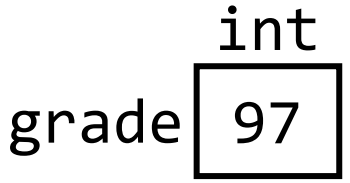
If Statements Example

```
int grade{97};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

Output:

If Statements Example

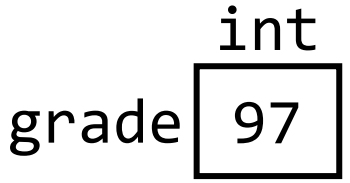
```
int grade{97};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```



Output:

If Statements Example

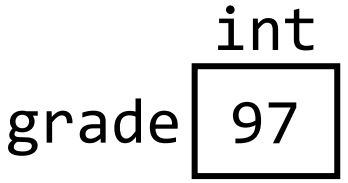
```
int grade{97};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```



Output:

If Statements Example

```
int grade{97};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

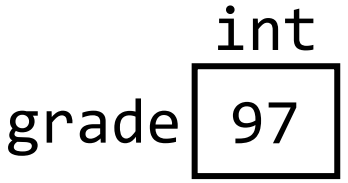


Output:

```
Outstanding!
```

If Statements Example

```
int grade{97};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

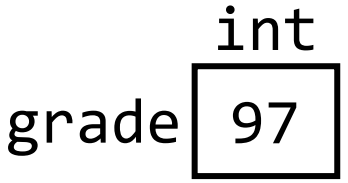


Output:

```
Outstanding!
```

If Statements Example

```
int grade{97};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

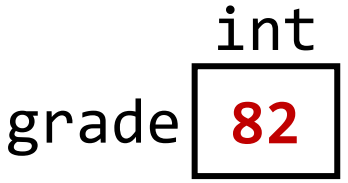


Output:

```
Outstanding!
Goodbye
```

If Statements Example

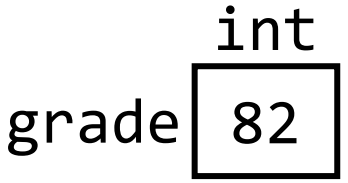
```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```



Output:

If Statements Example

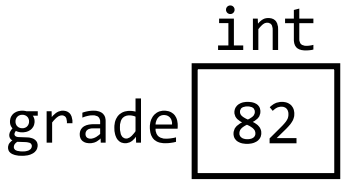
```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```



Output:

If Statements Example

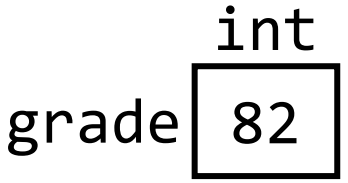
```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```



Output:

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

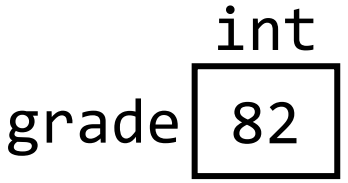


Output:

```
Good job
```

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

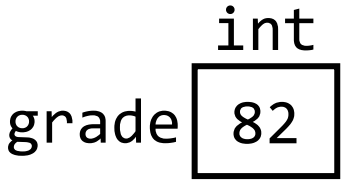


Output:

```
Good job
```

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

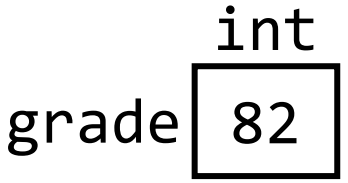


Output:

```
Good job
Goodbye
```

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

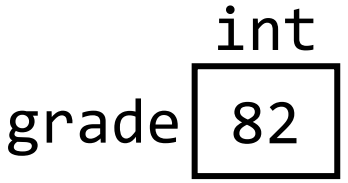


Output:

Observe: For all values of **grade**, “Goodbye” is output. “Pass” is output when $50 \leq \texttt{grade} < 75$. “Fail” is output when $\texttt{grade} < 50$.

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

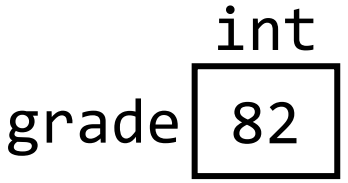


Output:

How would the output change if we swap the order of our conditions?

If Statements Example

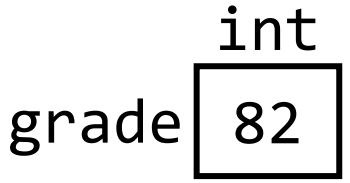
```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```



Output:

If Statements Example

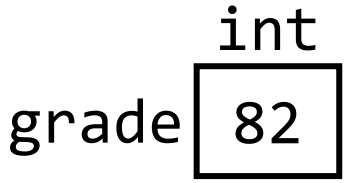
```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```



Output:

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

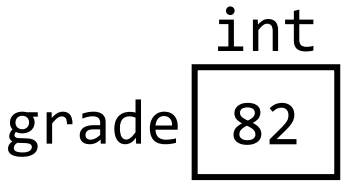


Output:

```
Pass
```

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

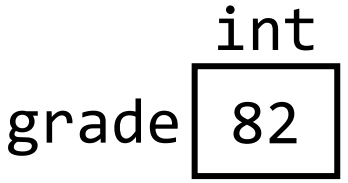


Output:

Pass

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```

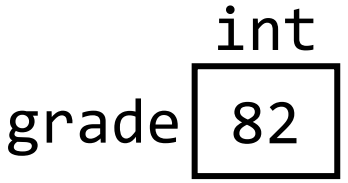


Output:

```
Pass
Goodbye
```

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```



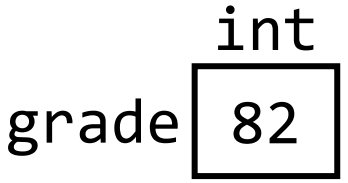
Output:

Pass
Goodbye

This probably isn't the output we were hoping for since the grade is above 75!

If Statements Example

```
int grade{82};
if (grade >= 90) {
    std::cout << "Outstanding!" << std::endl;
} else if (grade >= 50) {
    std::cout << "Pass" << std::endl;
} else if (grade >= 75) {
    std::cout << "Good job" << std::endl;
} else {
    std::cout << "Fail :(" << std::endl;
}
std::cout << "Goodbye" << std::endl;
```



Output:

```
Pass
Goodbye
```

Key Takeaway: As soon as one if condition evaluates to true, the corresponding block executes and all following conditions are skipped.

11: Loops

Anthony Estey
CSC 115: Fundamentals of Programming: II

Loops in C++

- As we know from Python, loops allow us to repeatedly execute a block of code.
- C++ has four types of loops, each suited for different situations:
  - **for loop**: repeat a block of code a *known* number of times
  - **range-based for loop**: iterate over every element in a collection
  - **while loop**: repeat a block of code *while* a condition is true
  - **do-while loop**: like a while loop, but the block always executes *at least once*

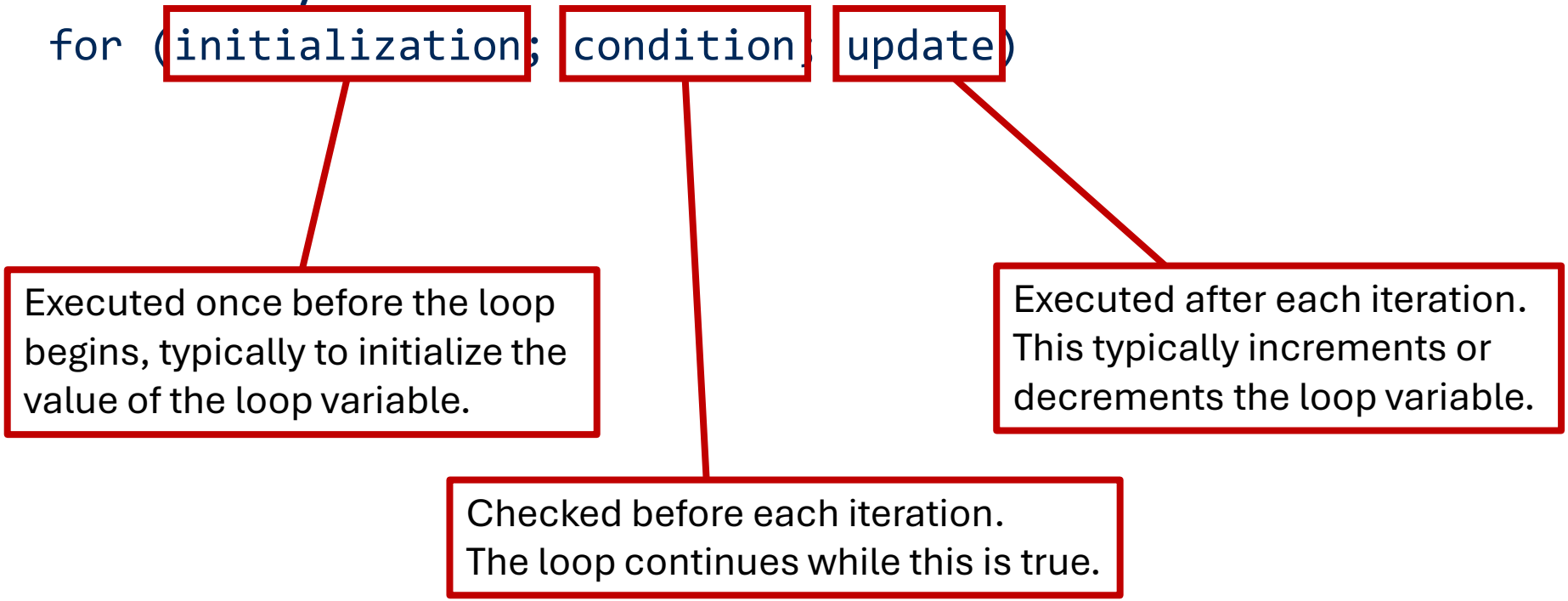
For Loops

- A **for loop** is used when we know exactly how many iterations we need.
- Some other specific use cases include:
  - When we need the index of an element in a collection (searching for the index of a specific value, updating the value at a specific position, comparing elements at different positions, etc.)
  - When we need to traverse a collection in a non-standard way (backwards, every other element, two arrays simultaneously, etc.)

For Loops

- General Syntax:

```
for (initialization; condition; update)
```



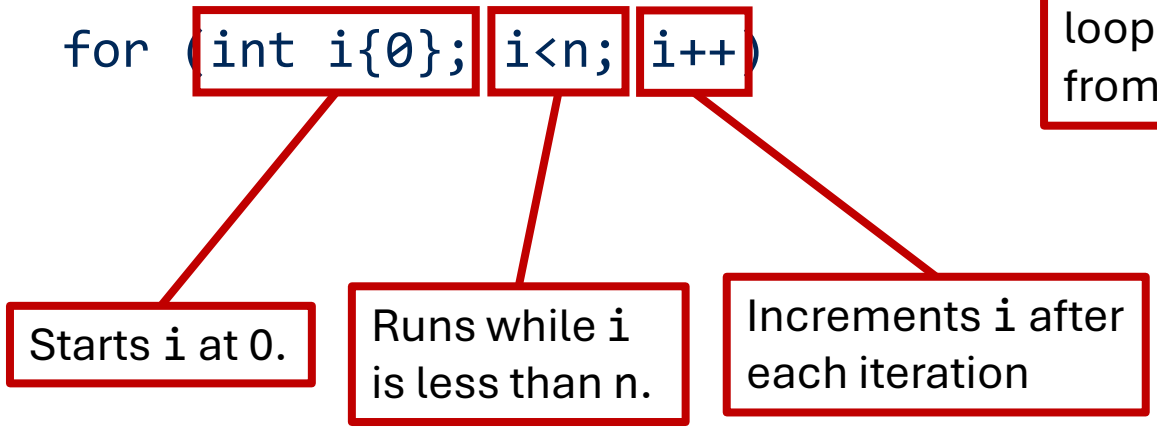
For Loops

- General Syntax:

```
for (initialization; condition; update)
```

- Specific examples:

```
for (int i{0}; i<n; i++)
```



Overall, we would say this for loop iterates through all values from 0 up to but not including n.

For Loops

- General Syntax:

```
for (initialization; condition; update)
```

- Specific examples:

```
for (int i{0}; i<n; i++)
```

```
for (int i{100}; i>=0; i-=5)
```

This for loop starts at 100, and counts down by 5 all the way to 0 (i.e. 100, 95, 90, ..., 10, 5, 0).

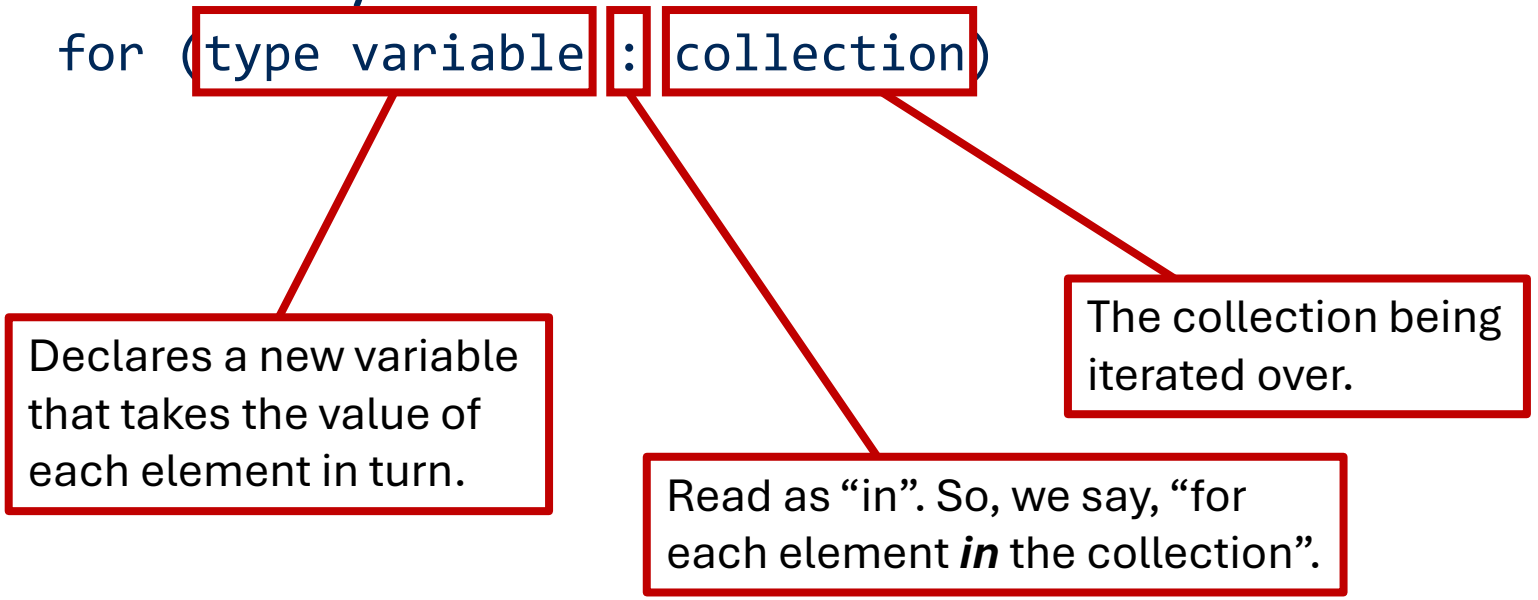
Range-Based For Loops

- A **range-based for loop** is used when we want to visit every element in a collection
- Used when we *don't* need to know the indexes of elements
- Specific use cases:
  - Summing or counting elements
  - Printing all elements
  - Checking a condition on every element

Range-Based For Loops

- General Syntax:

```
for (type variable : collection)
```



Range-Based For Loops

- General Syntax:

```
for (type variable : collection)
```

- Specific examples:

```
for (int grade : class_grades)
```

The `grades` variable gets a copy of each element in the `class_grades` collection. We can use this variable to calculate a sum, average, etc.
Note: `grade` is a copy of each element, so changes to it inside the loop do not affect the elements in the collection.

While Loops

- A **while loop** is used when the number of iterations is not known in advance
- The condition is checked *before* each iteration
  (so if is false from the start, the body never executes)
- Specific use cases:
  - Searching for an element in a collection (stop looping when found)
  - Reading input until a specific value is entered
  - Repeating until a condition changes

While Loops

- General Syntax:

```
while (condition) {
    // body
}
```

Evaluated *before* each iteration (the loop continues **while** this is true).

The body of the loop executes as long as the condition is true.

While Loops

- General Syntax:

```
while (condition) {
    // body
}
```

- Specific examples:

```
int value{};
std::cout << "Enter positive nums, -1 to stop" << std::endl;
std::cin >> value;
while (value != -1) {
    std::cout << "You entered: " << value << std::endl;
    std::cin >> value;
}
std::cout << "Done!" << std::endl;
```

This program will continue to output values entered by the user until a -1 is entered. Then, it will print “Done!”.

Note: If a -1 is entered first, the while loop is never entered and only “Done!” Is output.

Do While Loops

- A **do-while loop** is like a while loop, but the body always executes at least once
- The condition is checked *after* each iteration, not before
- Specific use cases:
  - Input validation: always ask at least once, repeat if invalid
  - Any situation where the body must run before the condition can be meaningfully checked

Do While Loops

- General Syntax:

```
do {
    // body
} while (condition);
```

The body executes first *before* any condition is ever checked.

Evaluated *after* each iteration.
The loop repeats **while** this is true.

Do While Loops

- General Syntax:

```
do {
    // body
} while (condition);
```

The loop *always* runs at least once to get the initial value for age. It continues to repeat *until* a valid positive value is entered.

- Specific examples:

```
int age{};
do {
    std::cout << "Enter a positive age: ";
    std::cin >> age;
} while (age <= 0);
```

12: Functions

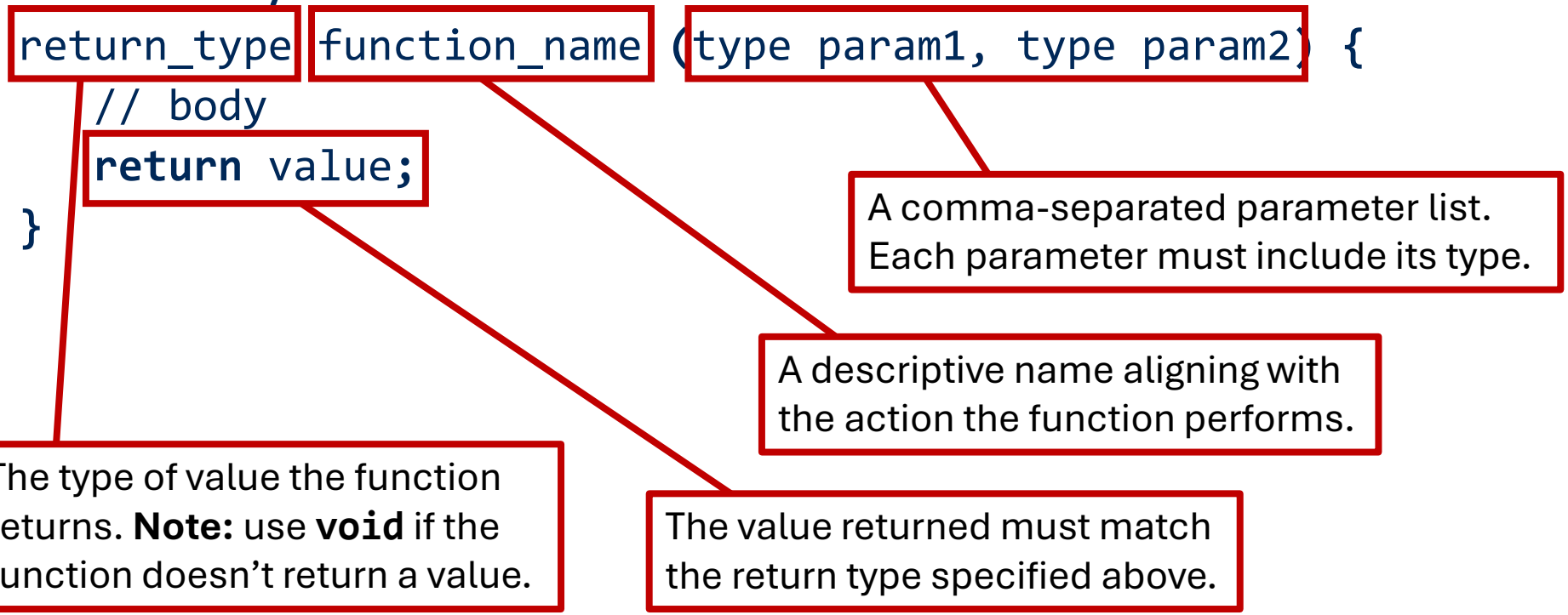
Anthony Estey
CSC 115: Fundamentals of Programming: II

Functions

- Functions are reusable blocks of code that perform a single task.
- The main differences between how we write functions in C++ when compared to Python are the following:
  - In order for C++ programs to compile correctly, every function must declare the type of value it returns and the type of each parameter.
  - The code block is defined with curly braces {} instead of indentation

Functions

- General Syntax:



Functions

- General Syntax:

```
int sum_numbers(int a, int b, int c) {
    return a + b + c;
}
```

Three integer parameters, each declared with type and name.

The function name describes its purpose

This function returns an integer (and the program will not compile otherwise).

Send the result (which must be an integer) back to the caller.

Functions

- General Syntax:

```
int sum_numbers(int a, int b, int c) {
    return a + b + c;
}
```

- Calling the function:

```
int result{};
result = sum_numbers(7, 2, 4);
```

The function call `sum_numbers(7, 2, 4)` returns 13. The function call evaluates to 13, and this value is assigned to the variable `result`.

13: Parameter Passing, Returns, and Variable Scope

Anthony Estey
CSC 115: Fundamentals of Programming: II

Variable Scope

- In C++, every variable has a **scope,** which is the region of code where it is accessible
  - A variable's scope begins where it is declared, and ends at the closing brace **}** of the block it was declared in
  - This applies to any of the code blocks we have seen so far: functions, if statements, and loops
  - Variables declared in different scopes are completely independent from one another, even if they share the same name
  - Variables declared in a for loop only exist for the duration of that loop (after the closing brace **}** the variable is no longer accessible)

Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

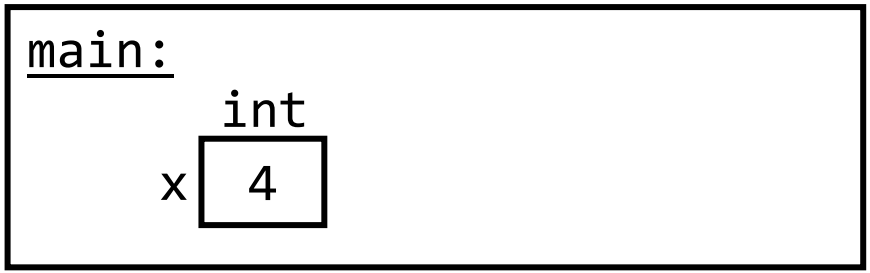
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

main:

Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

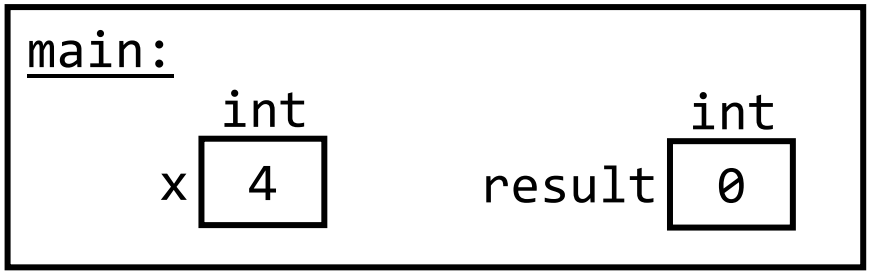
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```



Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

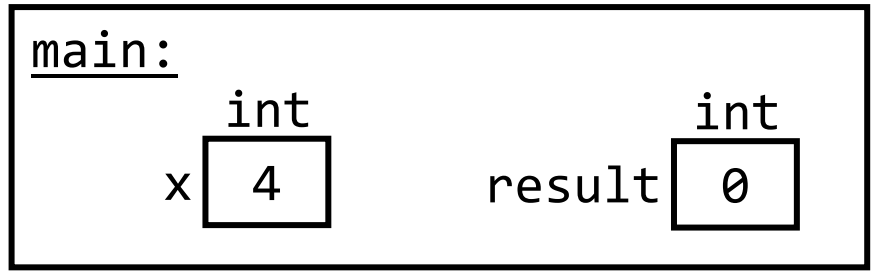


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

4

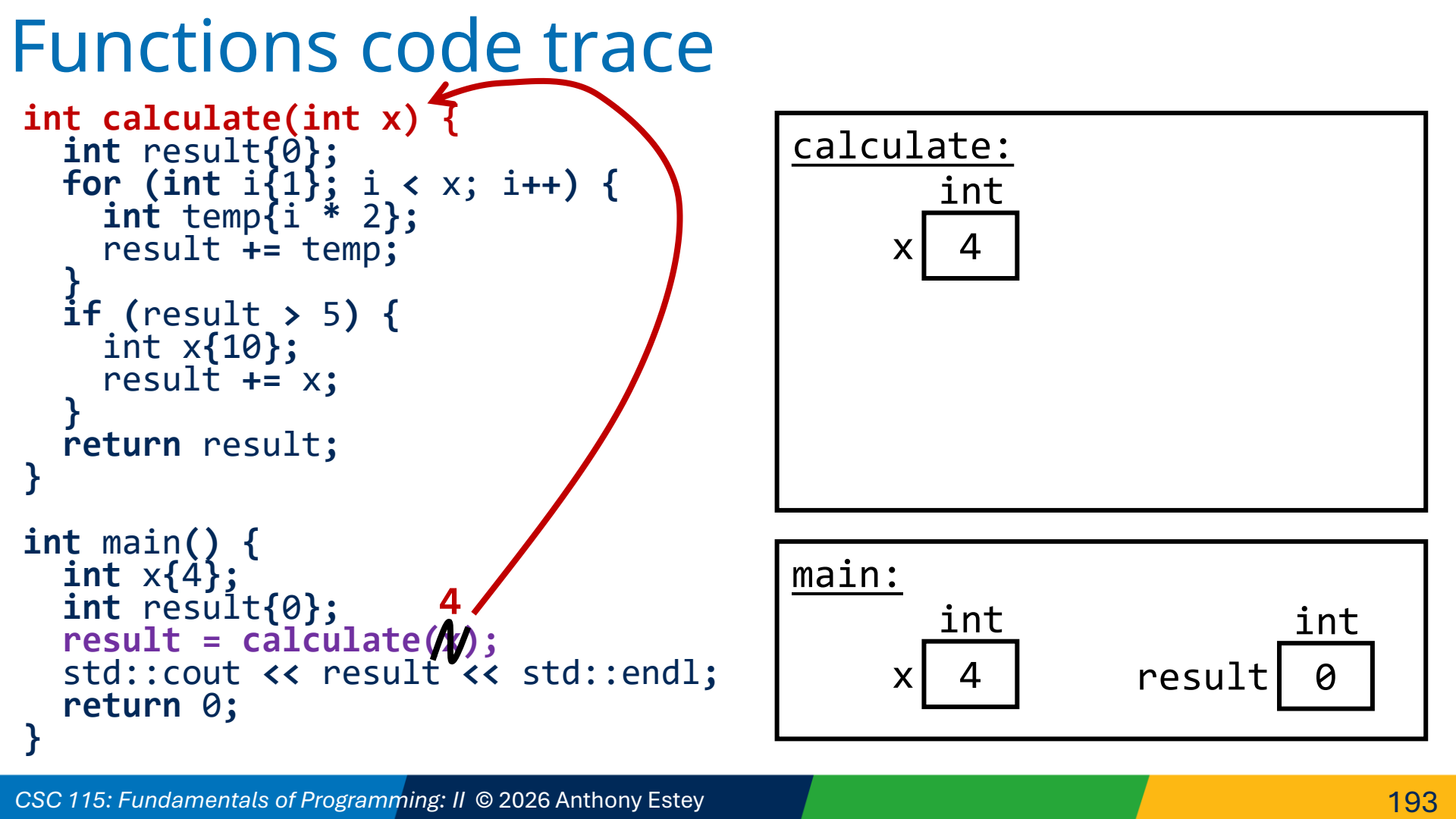


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

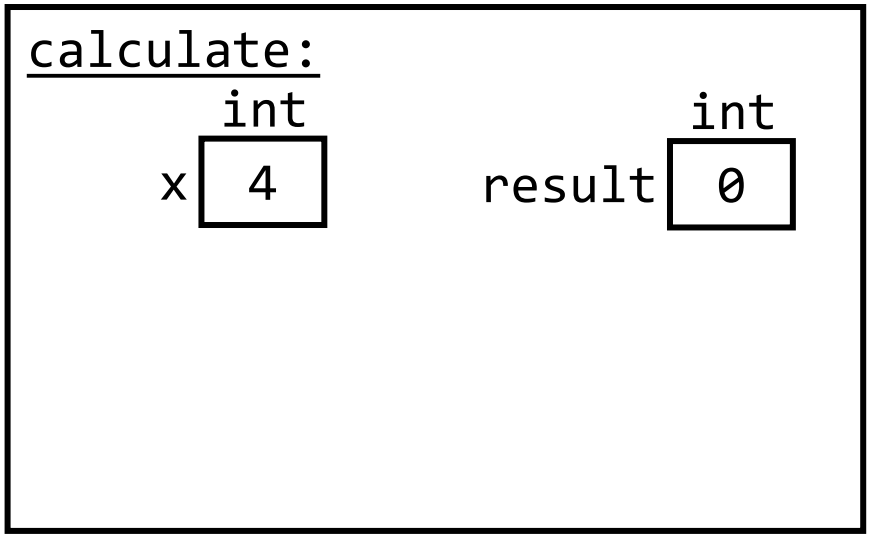
4

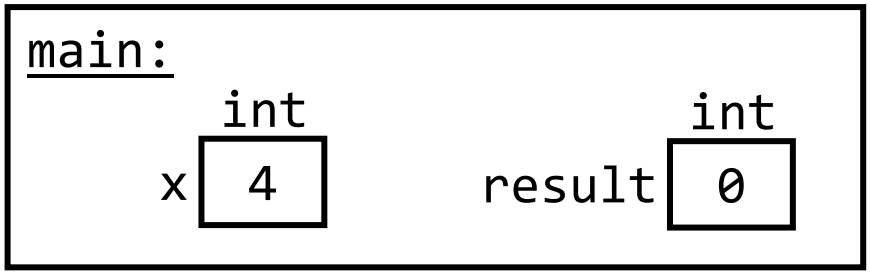


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

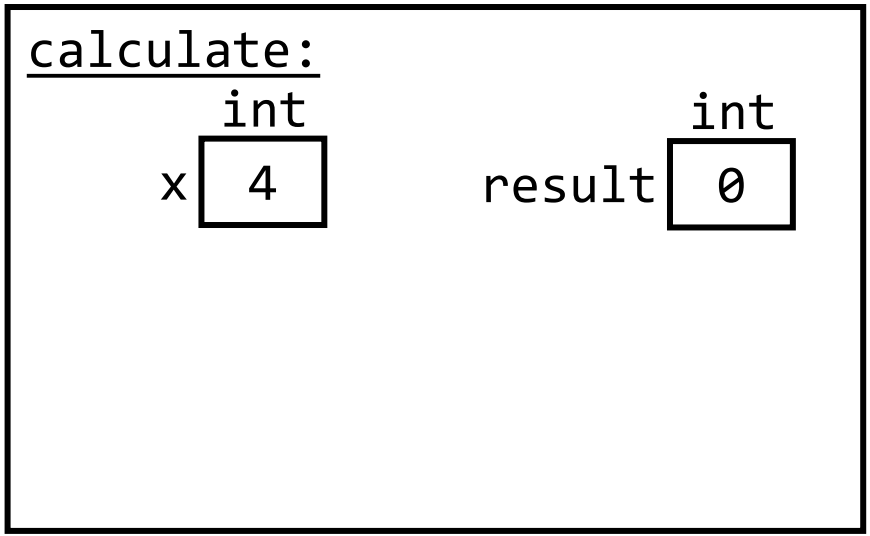


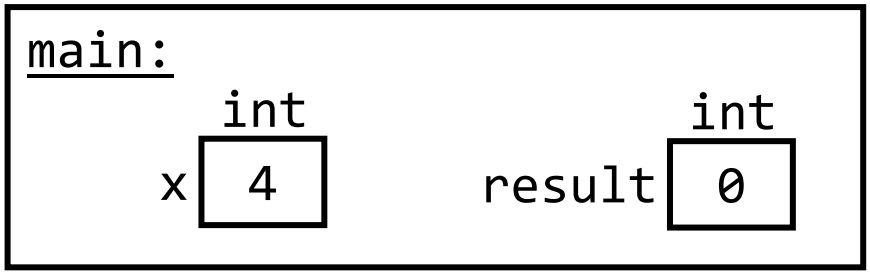


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

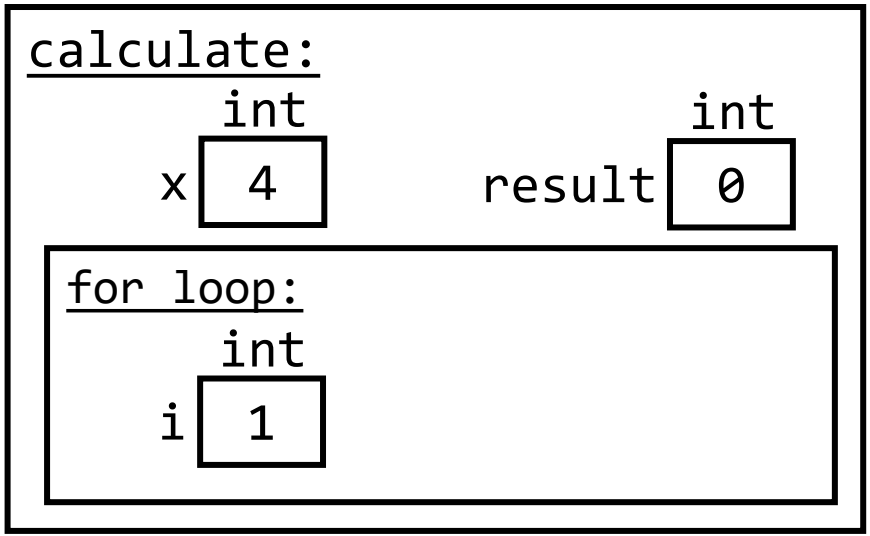


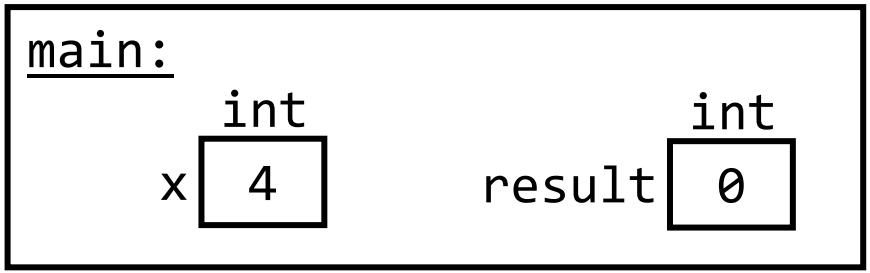


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

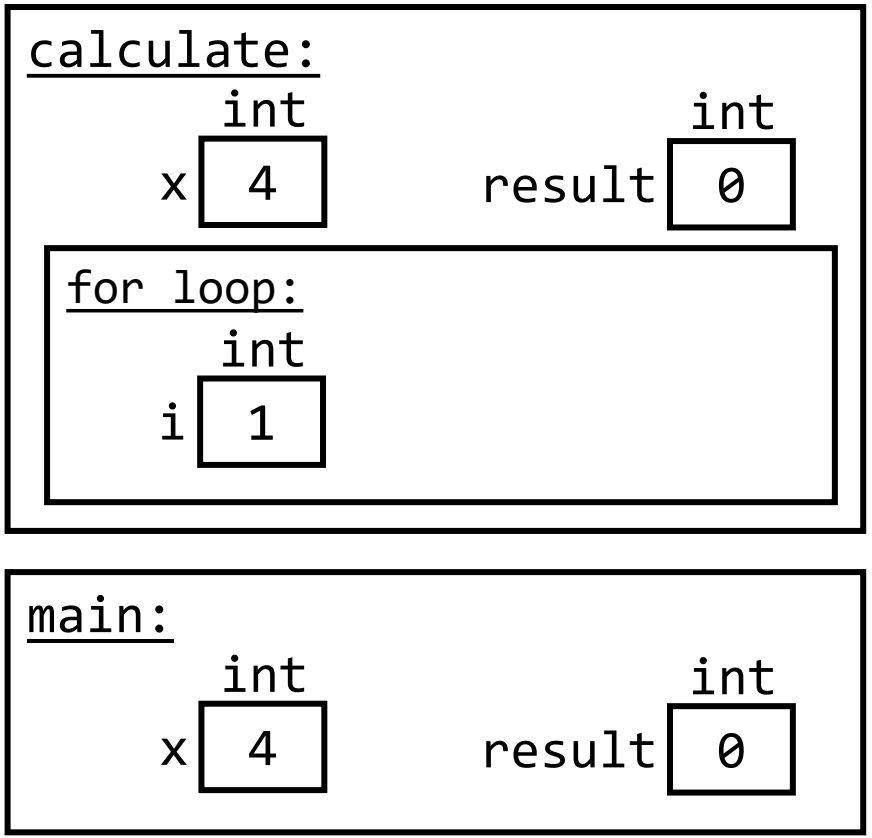




Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

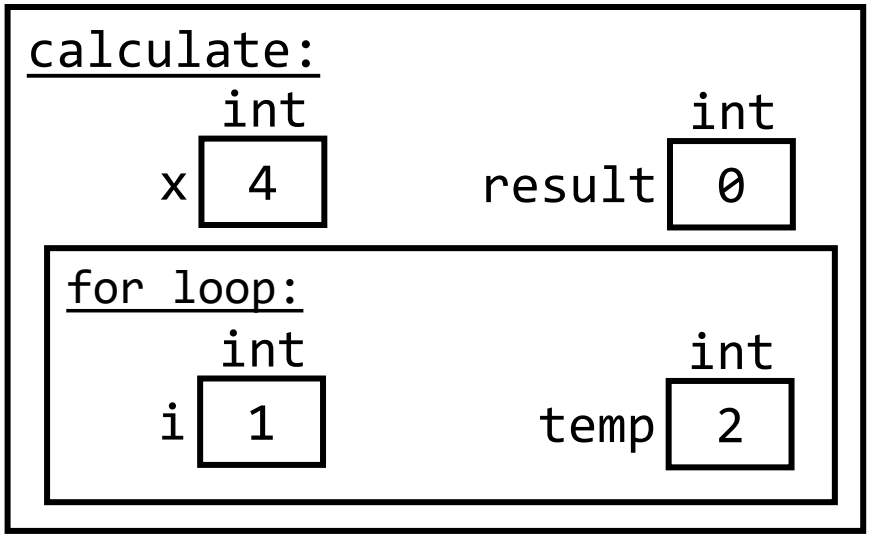
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

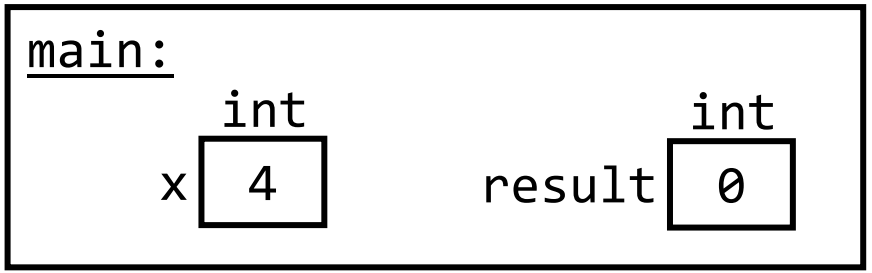


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

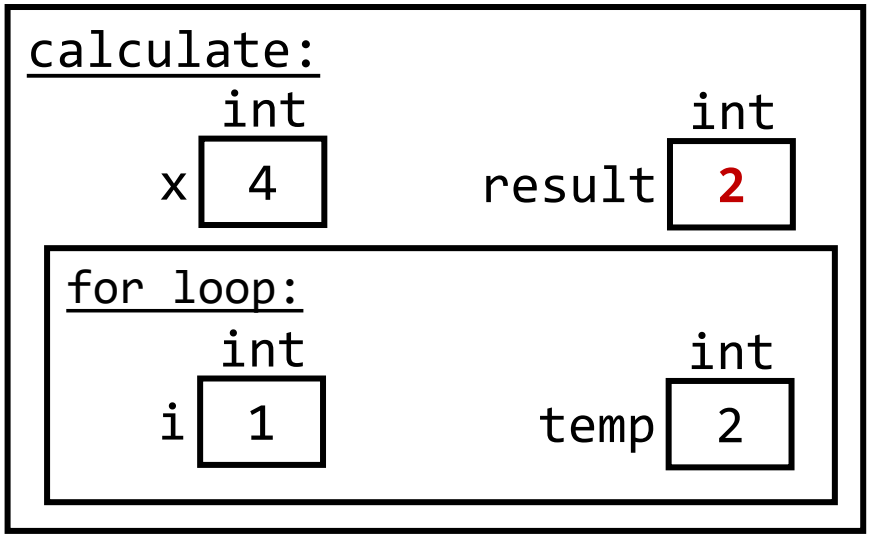


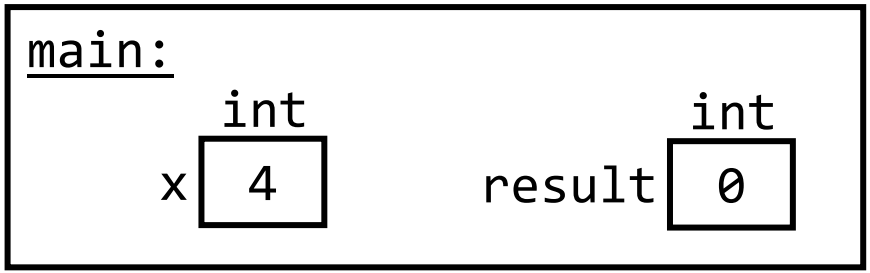


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

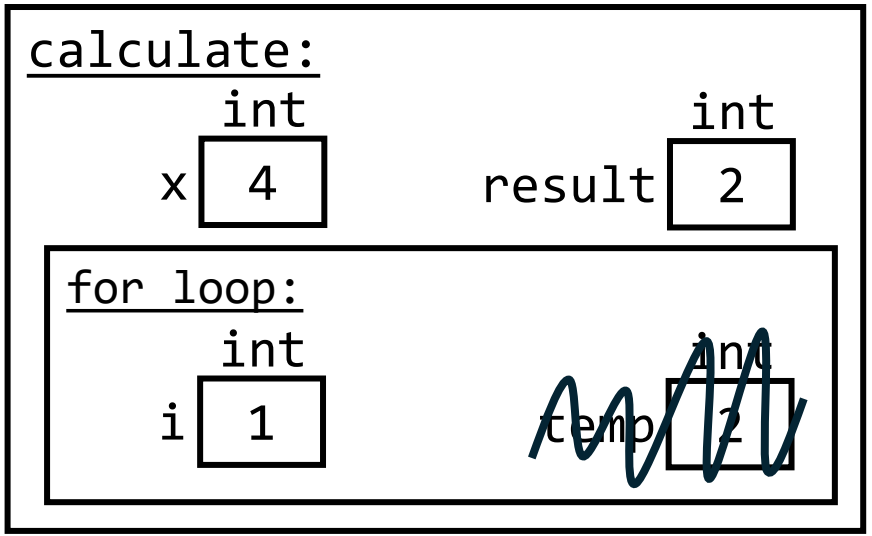




Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

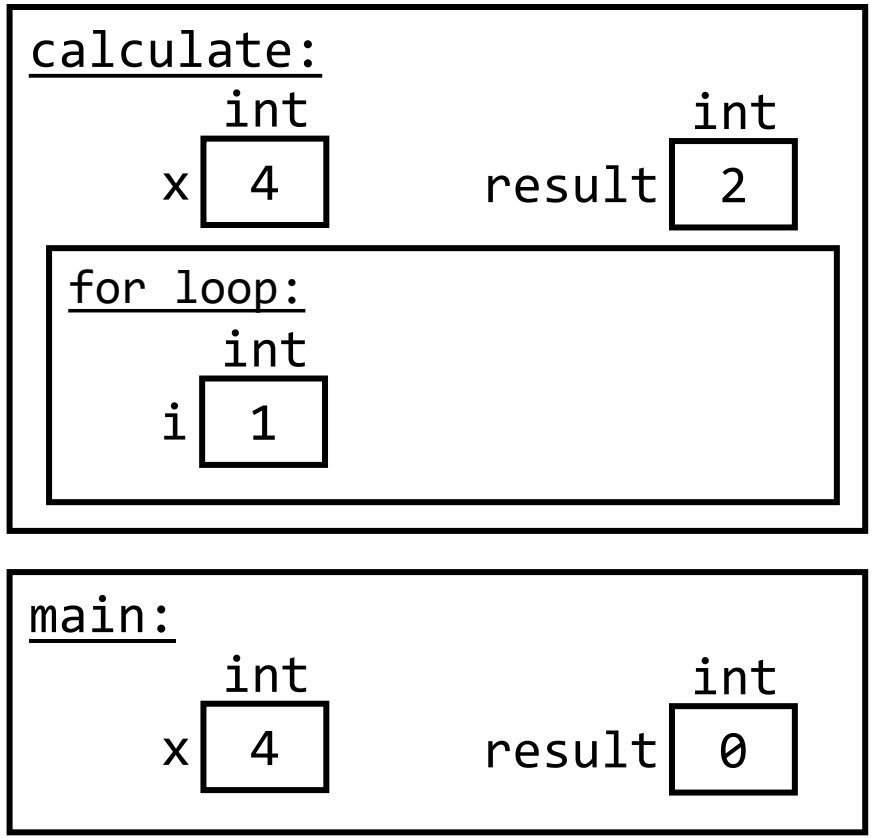
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```



Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

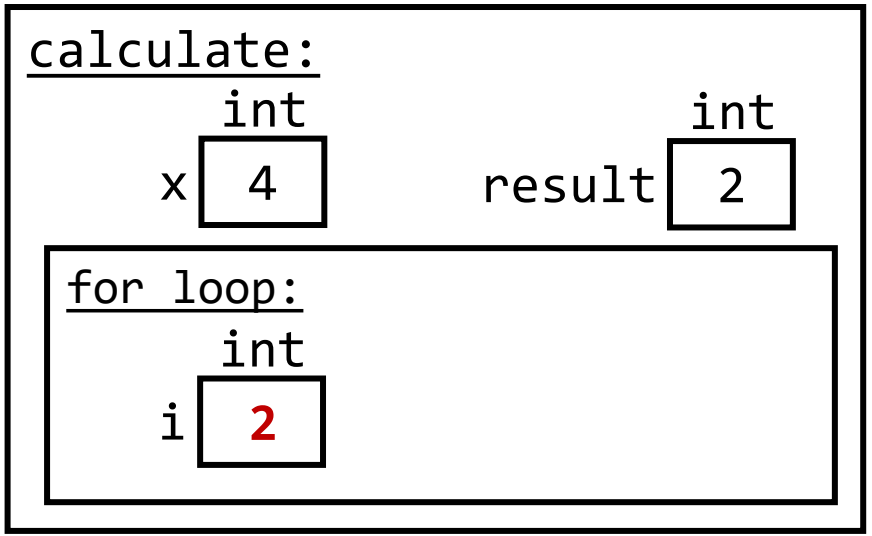
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

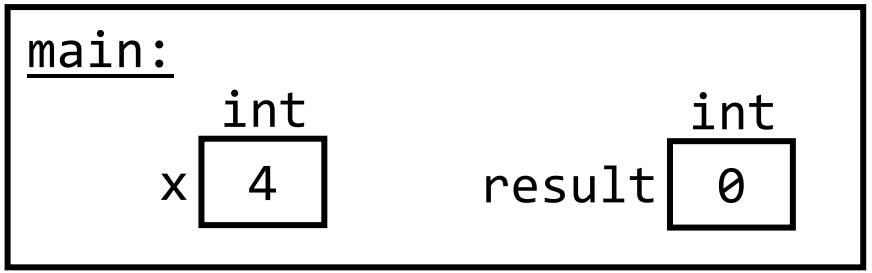


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

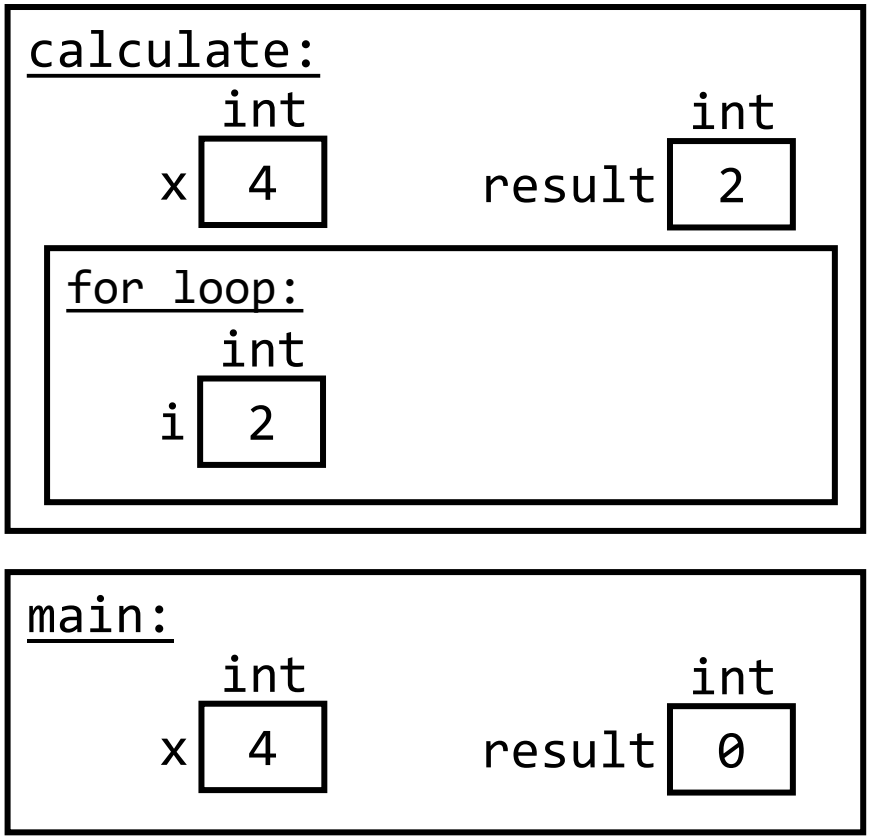




Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

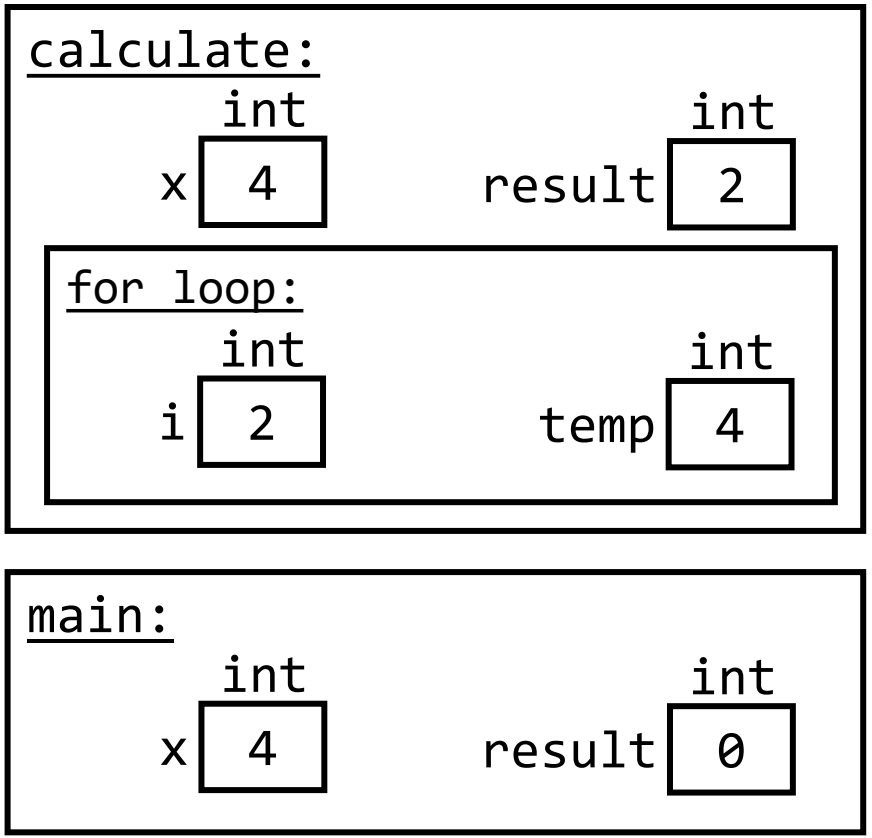
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```



Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

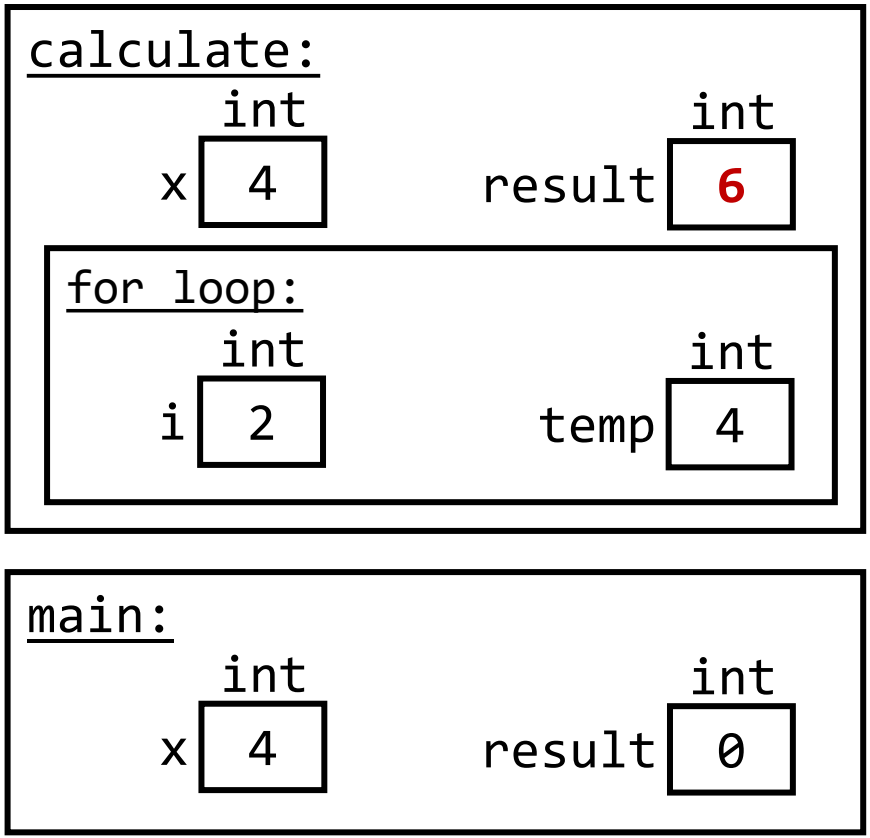
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```



Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

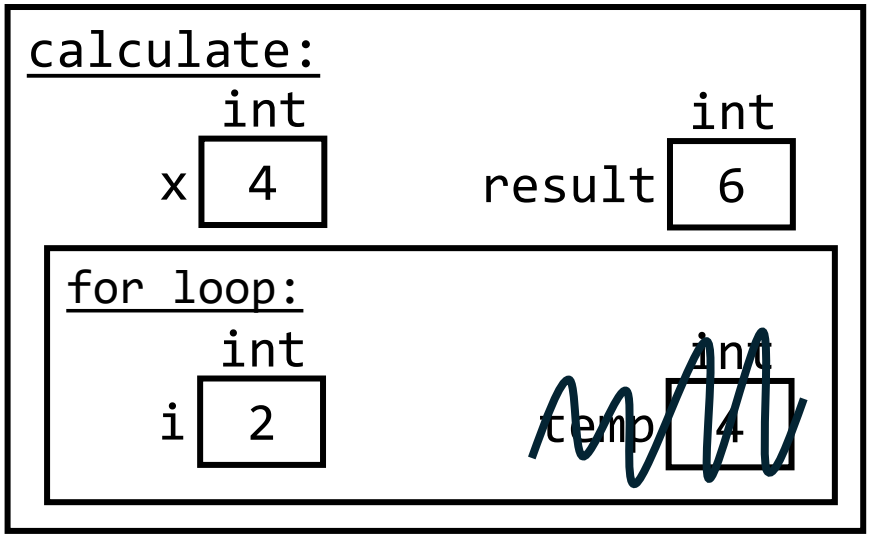
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```



Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

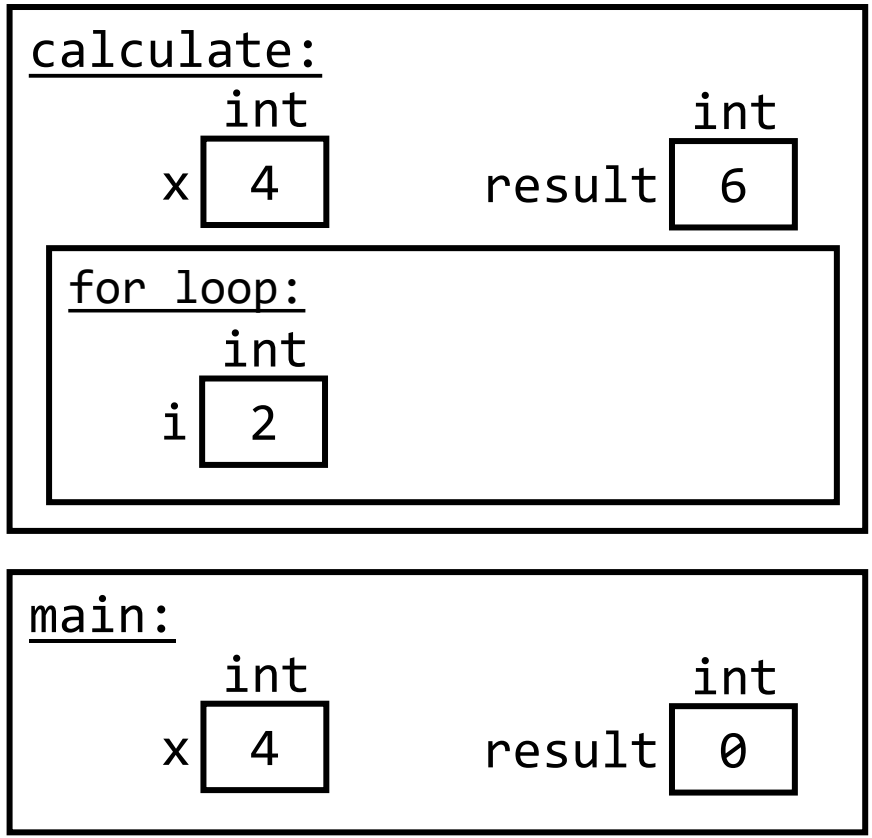
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```



Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

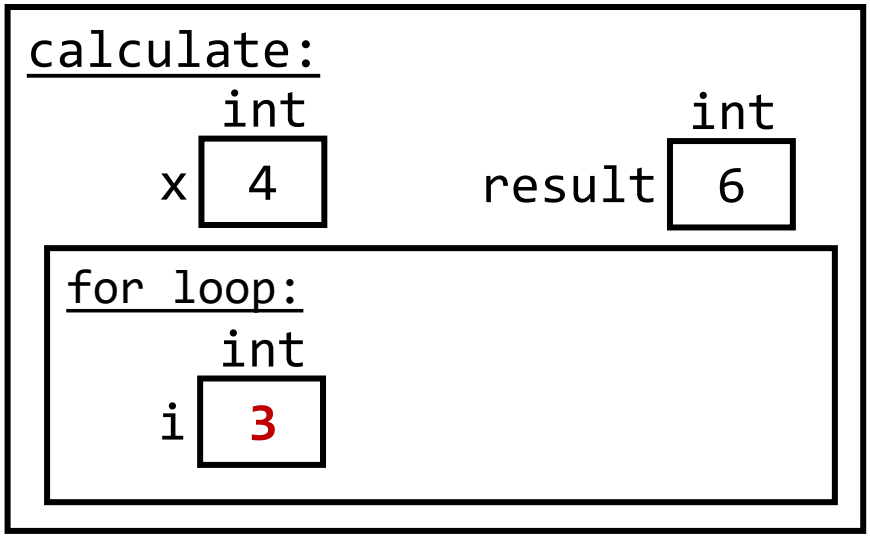
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

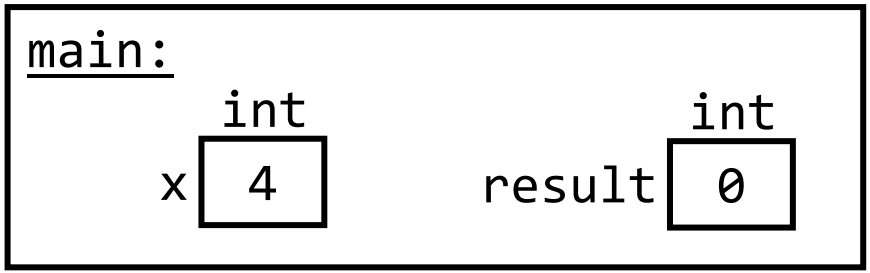


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

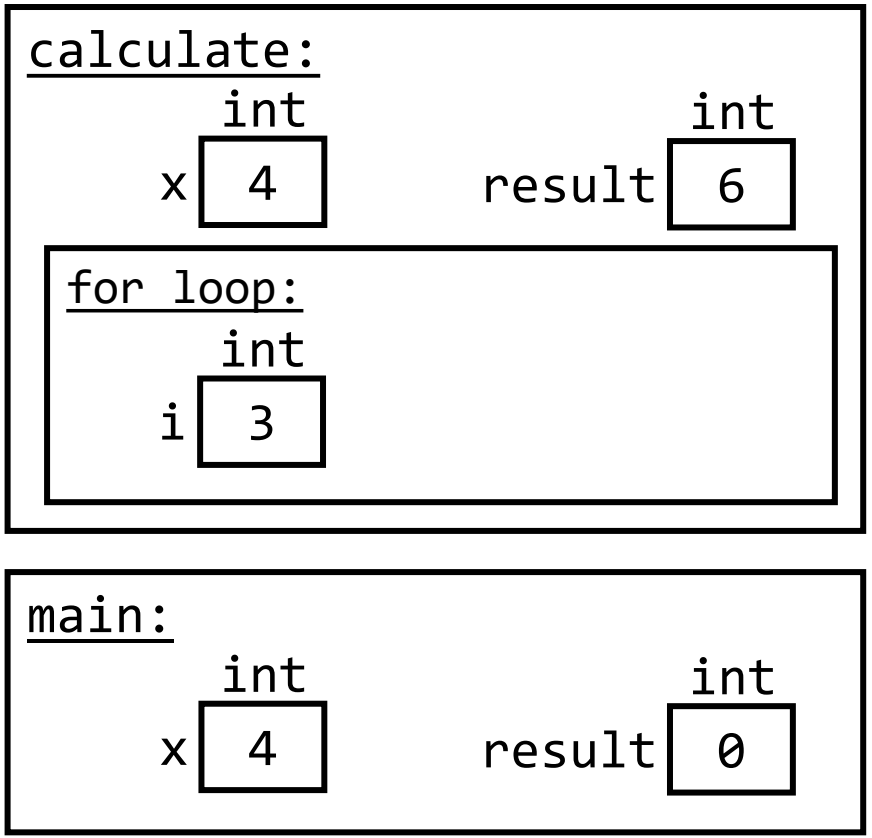




Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

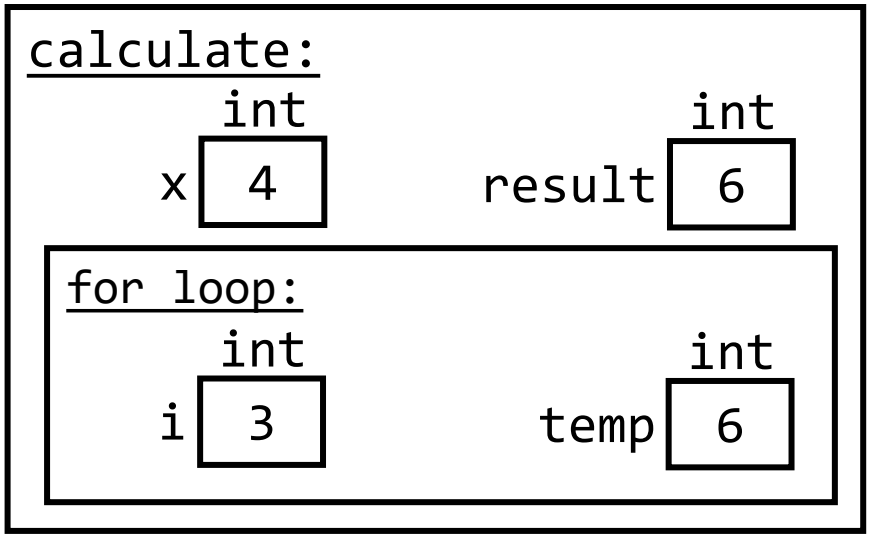
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

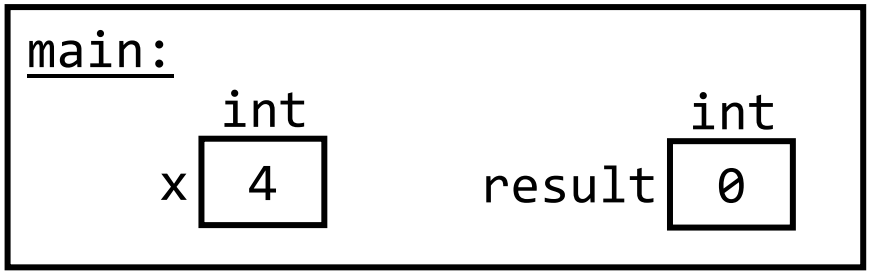


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

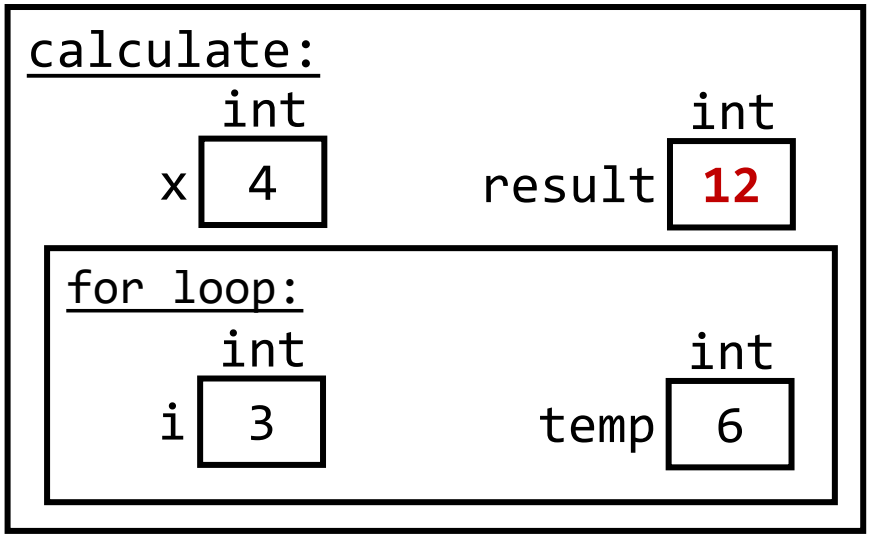


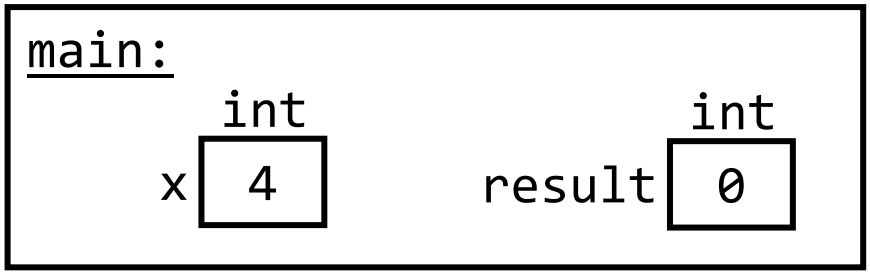


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

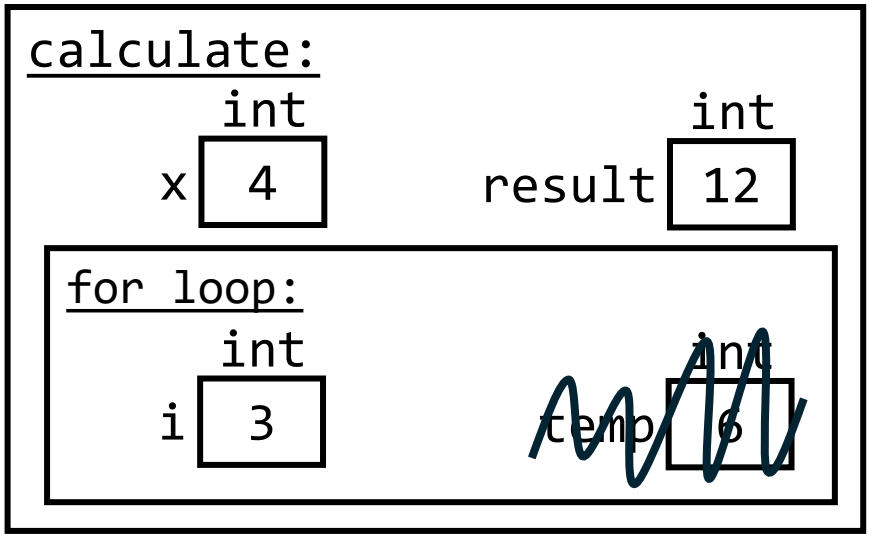




Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

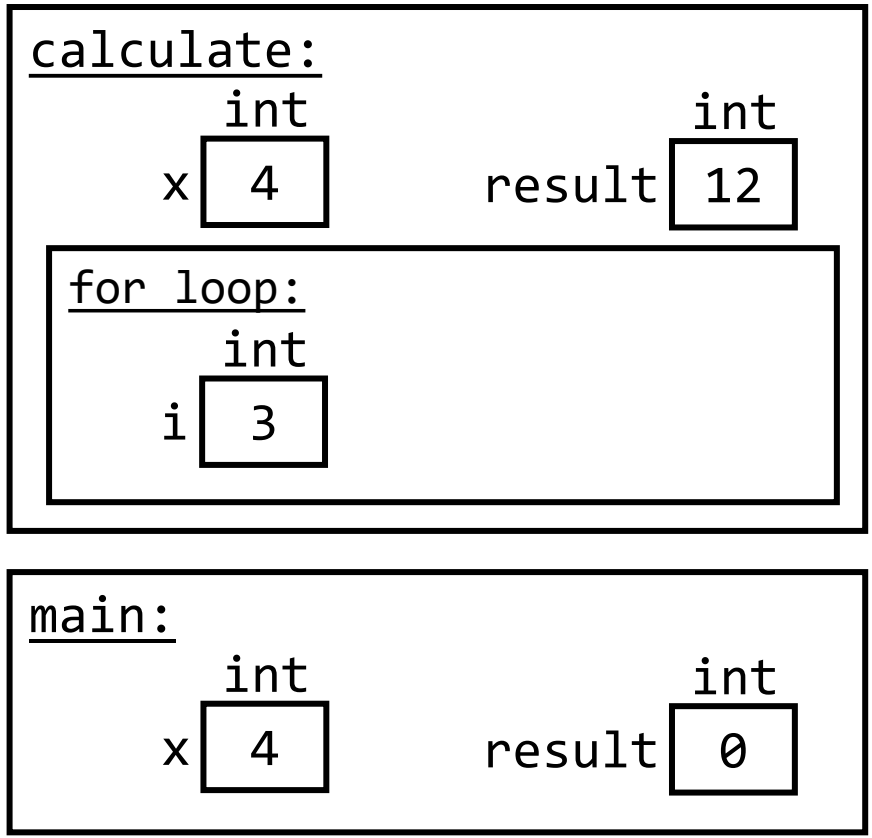
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```



Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

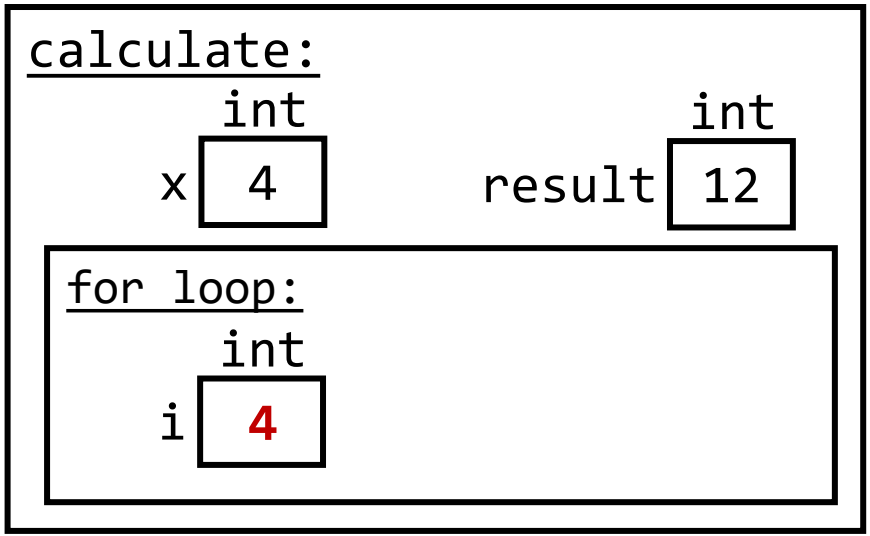
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

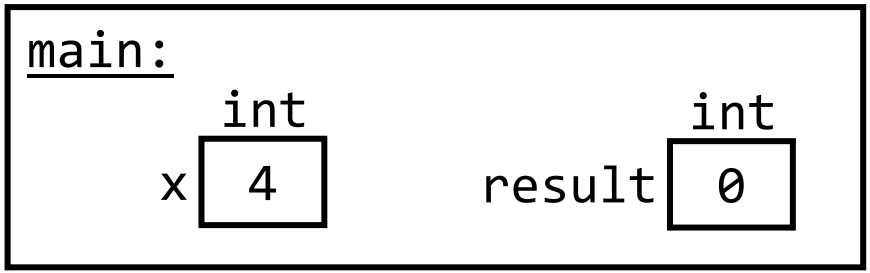


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

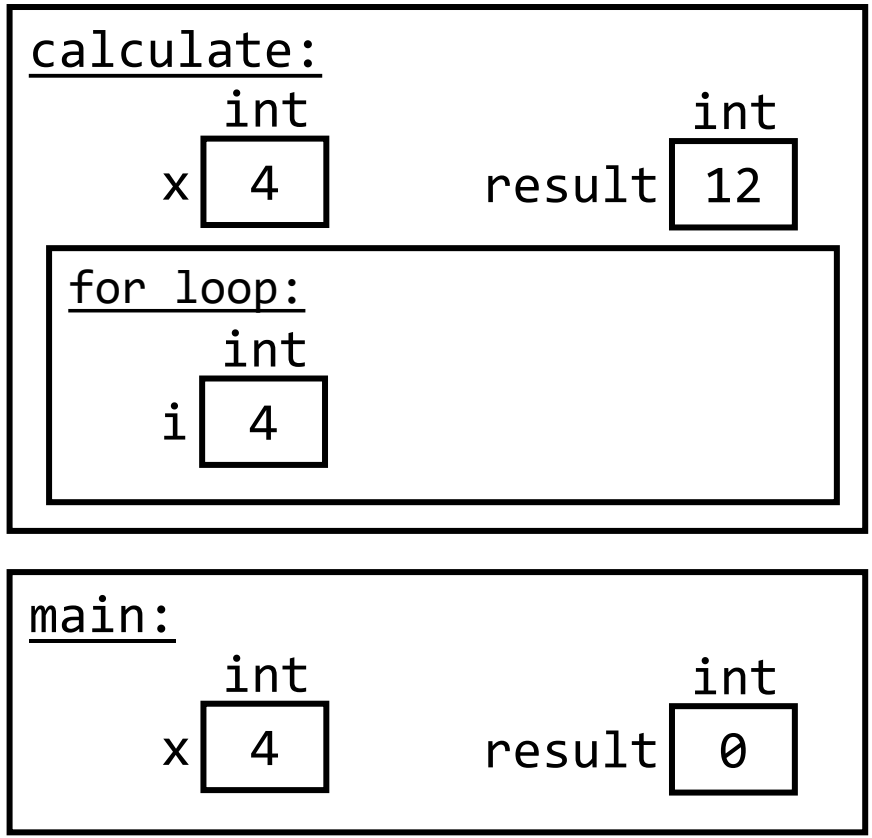




Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

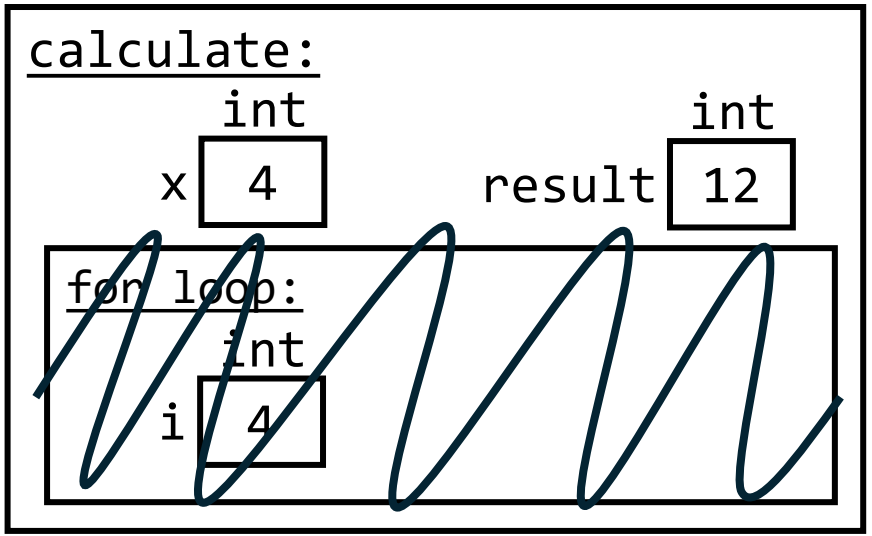
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

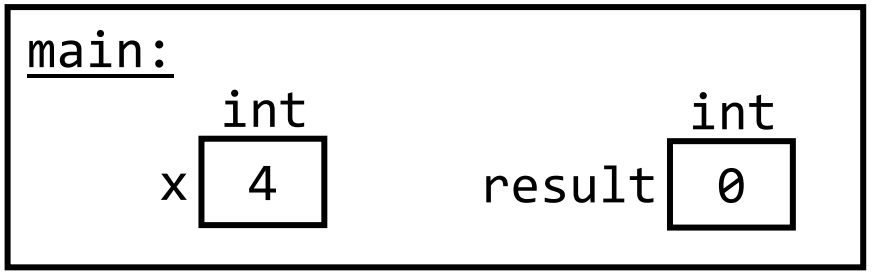


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

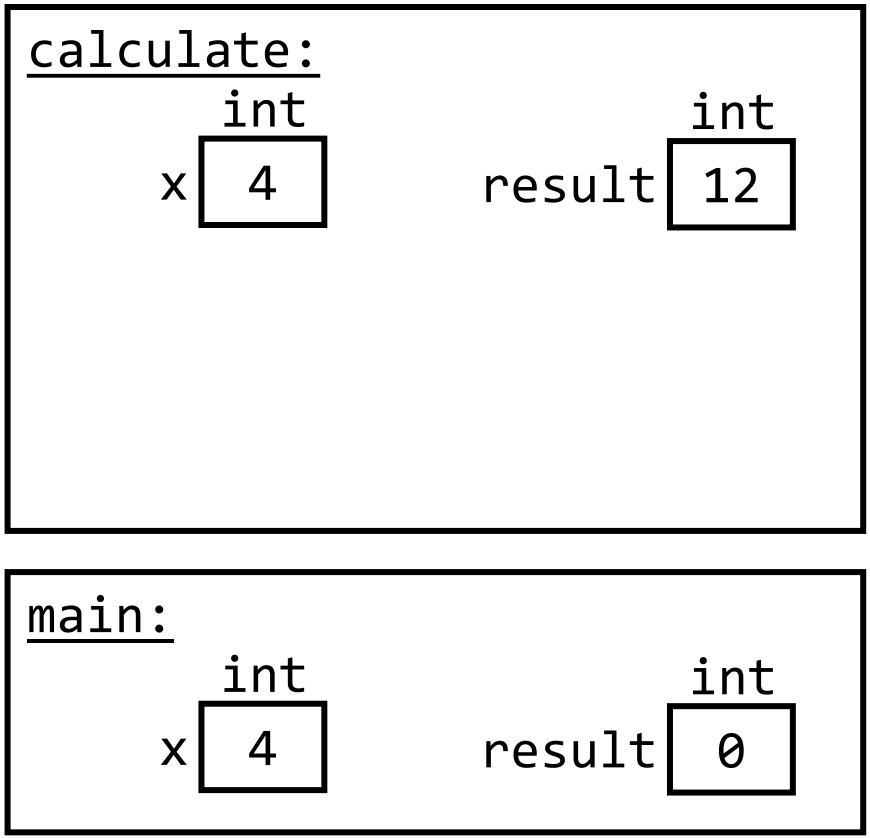




Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

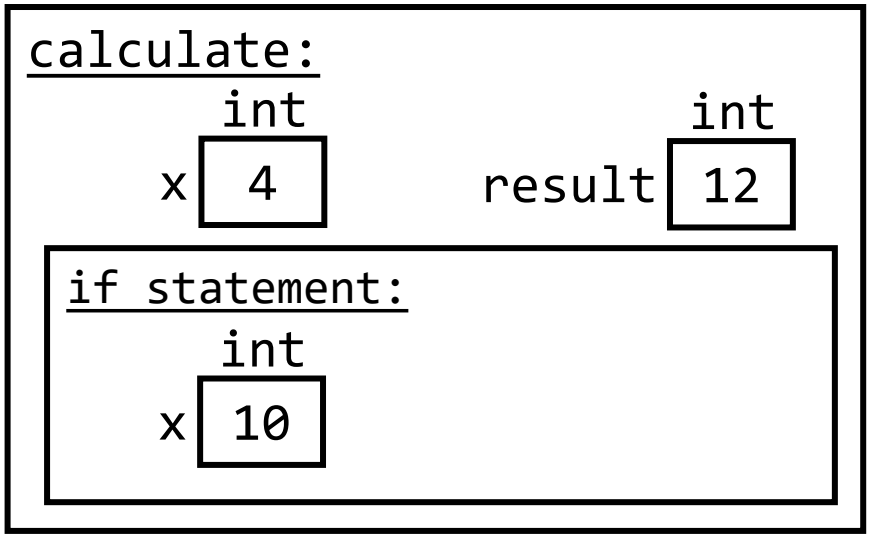
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

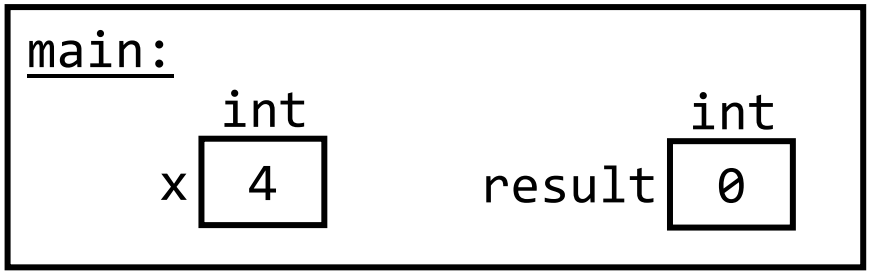


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

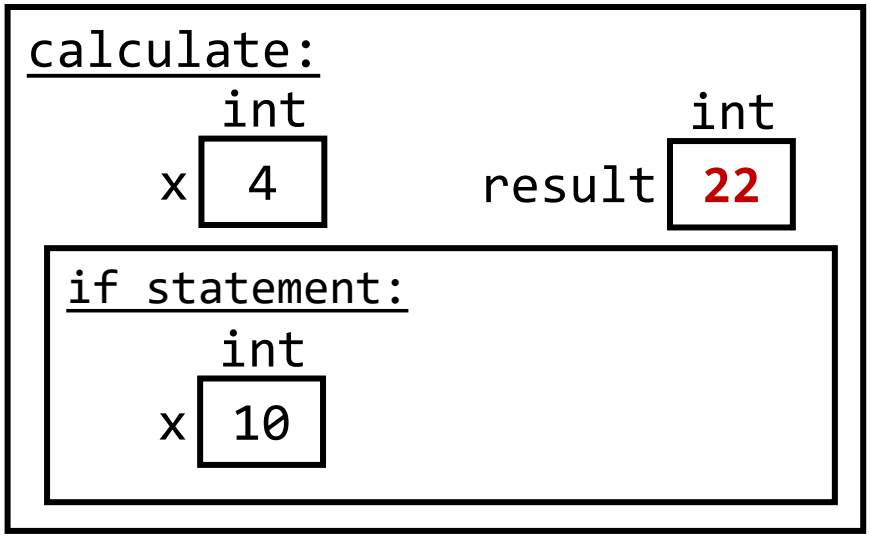


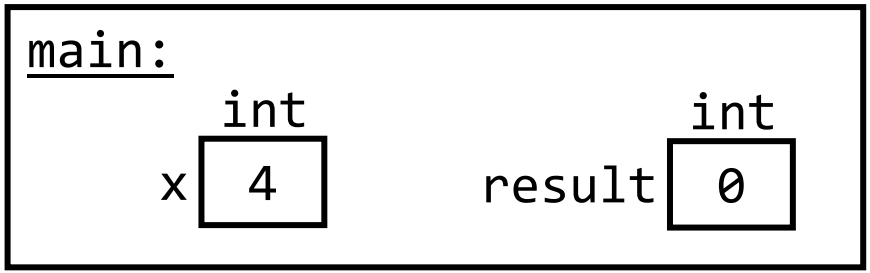


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

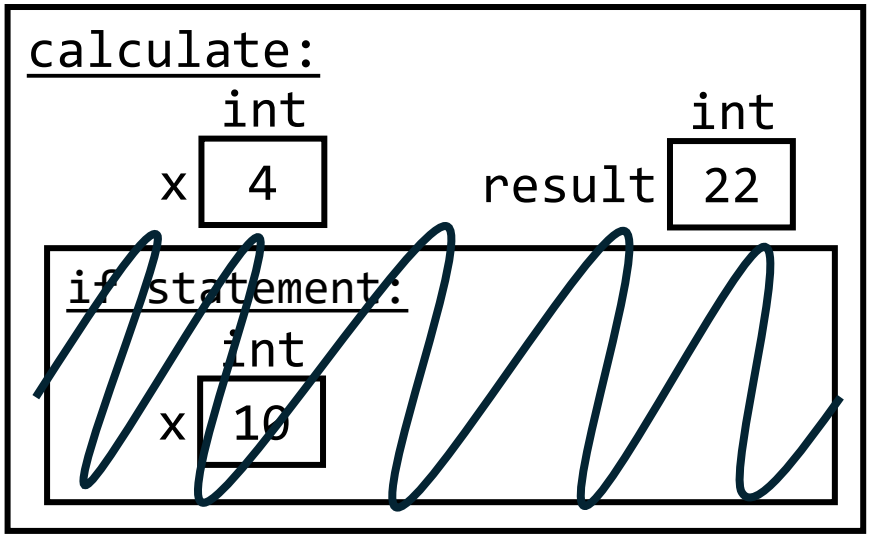


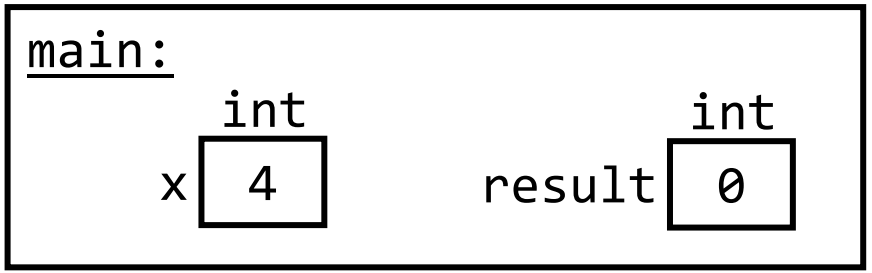


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

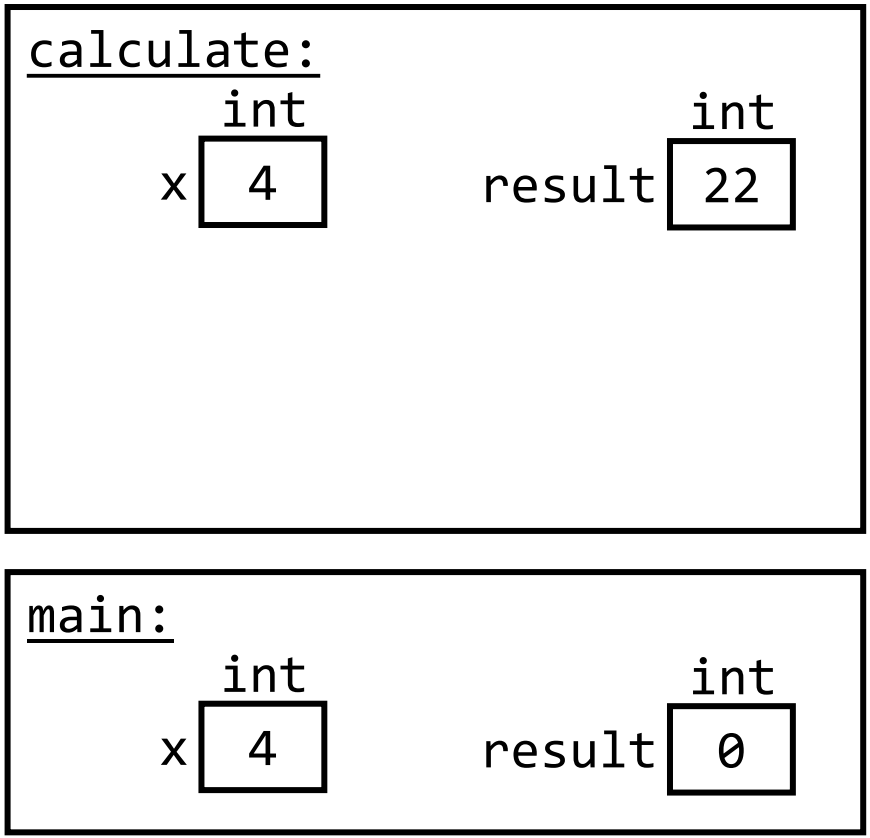




Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result; 22
}

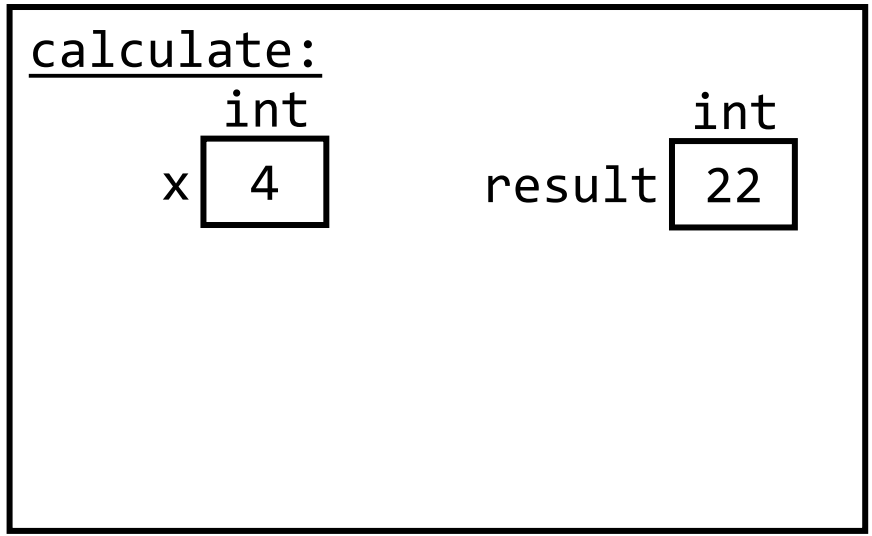
int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

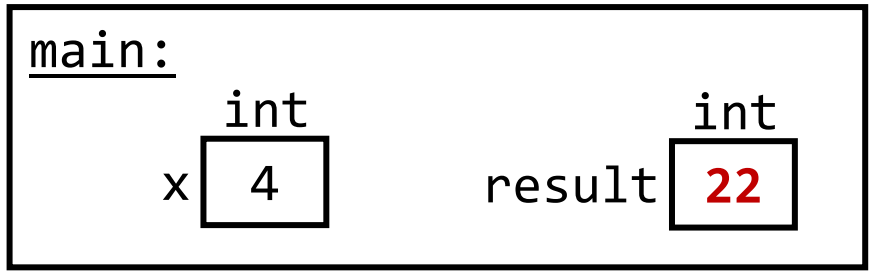


Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result; 22
}

int main() {
  int x{4};
  int result{0}; 22
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

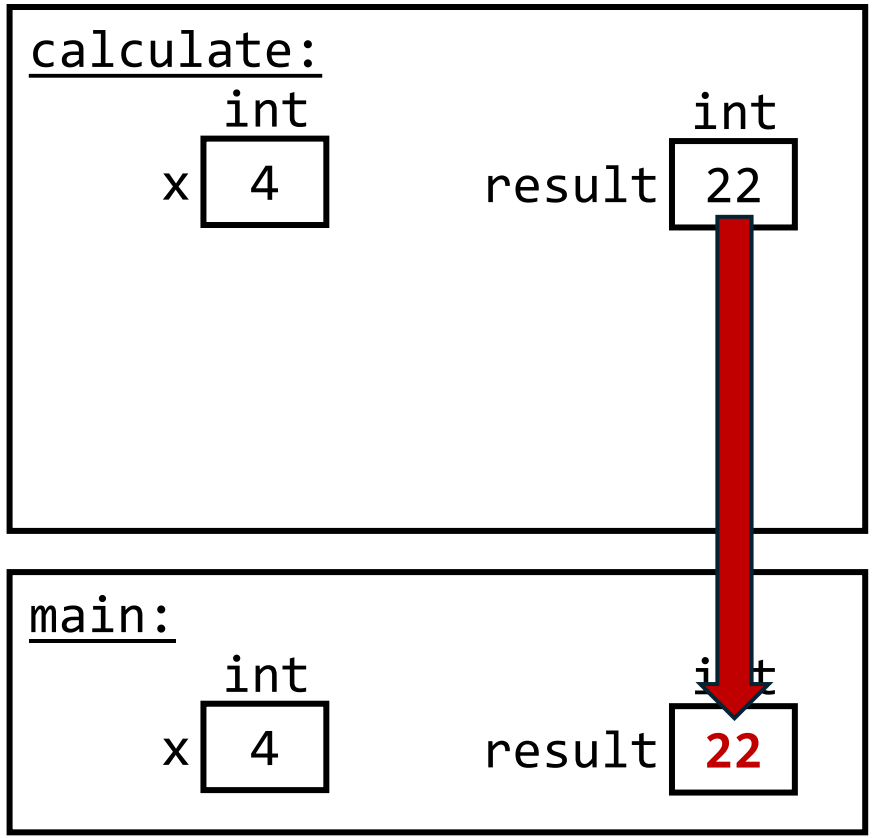




Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result; 22
}

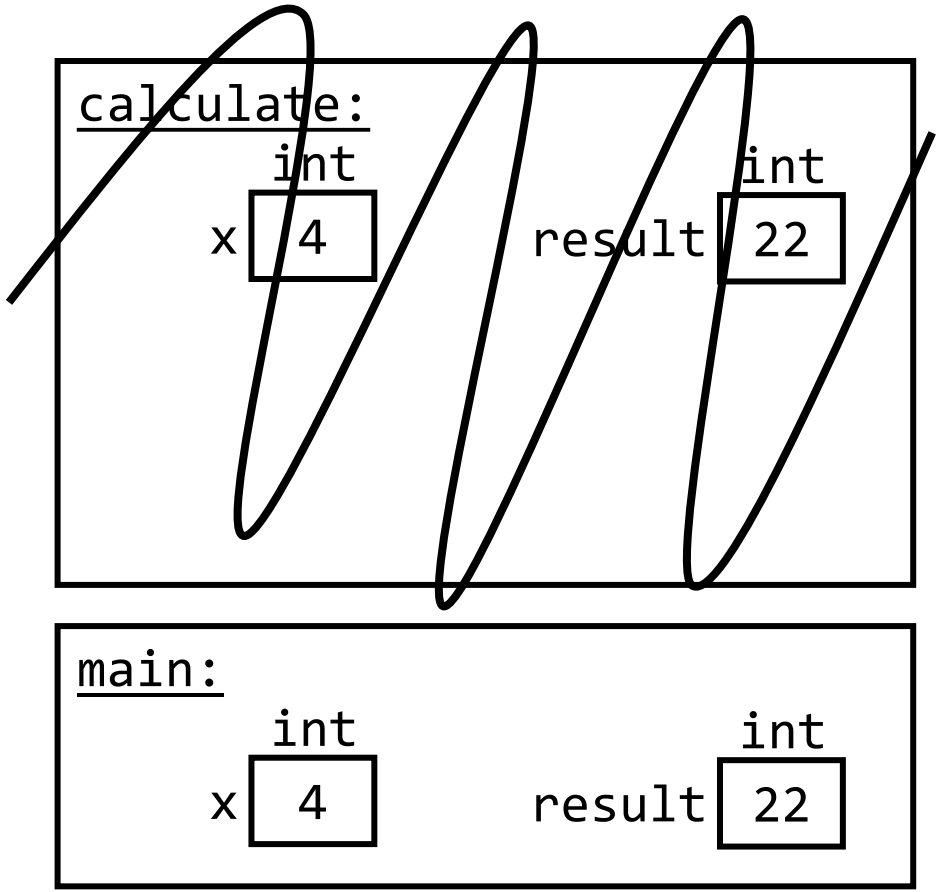
int main() {
  int x{4};
  int result{0}; 22
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```



Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```



Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

Output:

22

main:

	int
x	4

	int
result	22

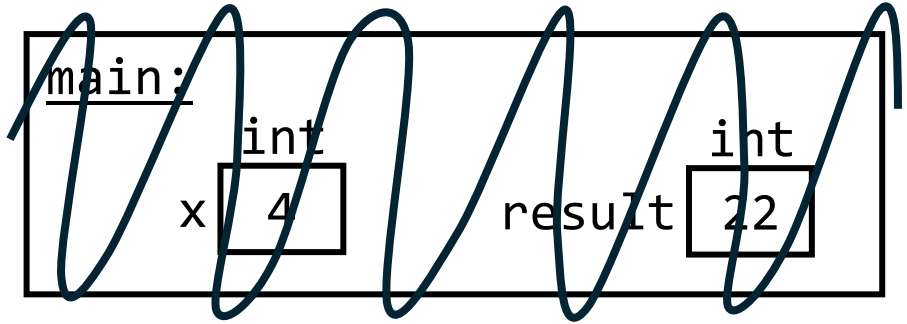
Functions code trace

```
int calculate(int x) {
  int result{0};
  for (int i{1}; i < x; i++) {
    int temp{i * 2};
    result += temp;
  }
  if (result > 5) {
    int x{10};
    result += x;
  }
  return result;
}

int main() {
  int x{4};
  int result{0};
  result = calculate(x);
  std::cout << result << std::endl;
  return 0;
}
```

Output:

22



14: Function Design

Anthony Estey
CSC 115: Fundamentals of Programming: II

Function Design

- In this course we will follow these 4 function design steps:
  1. **Prototype:** defines the function's signature for the compiler and other developers
  2. **Documentation:** describes what the function does and how to use it (written for anyone, ourselves or others, who use it)
  3. **Definition:** the implementation (the actual code to produce the desired behaviour)
  4. **Testing**: examples that verify the function works correctly, including typical, edge, and boundary cases

Considerations at each step

1. **Prototype**: consider the name, return type and parameter-list
2. **Documentation**: Purpose statement, parameters, return value
   - Purpose Statement: concisely describes what the function does
   - Parameters: Specify their types and describe what they do
   - Returns: Specify the return type and describe what the value represents
3. **Definition**: Write the code to implement the function
4. **Testing**: Call the function with different input conditions to ensure it has the correct behaviour under all possible scenarios
   - Test coverage: consider all possible input conditions and scenarios

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

1. **Prototype**: consider the name, return type and parameter-list

```
return-type function-name (parameter-list);
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

1. **Prototype**: consider the **name**, return type and parameter-list

```
return-type function-name (parameter-list);
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

1. **Prototype**: consider the **name**, return type and parameter-list

```
return-type is_between    (parameter-list);
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

1. **Prototype**: consider the **name**, return type and parameter-list

```
return-type is_between (parameter-list);
```

- Name must be descriptive: **`within_range`** also works

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

1. **Prototype**: consider the name, **return type** and parameter-list

```
return-type is_between (parameter-list);
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

1. **Prototype**: consider the name, **return type** and parameter-list

```
bool        is_between (parameter-list);
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

1. **Prototype**: consider the name, return type and **parameter-list**

```
bool is_between (parameter-list);
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

1. **Prototype**: consider the name, return type and **parameter-list**

```
bool is_between (int value, int min, int max);
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

1. **Prototype**: consider the name, return type and parameter-list

```
bool is_between (int value, int min, int max);
```

The prototype gives us everything we need to know to correctly *use* the function: it's name, return type, and parameter list.

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 *
 *
 *
 *
 *
 *
 *
 *
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 * @brief
 *
 *
 *
 *
 *
 *
 *
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 * @brief Determines if the given value is between the
 *        minimum and maximum threshold values
 *
 *
 *
 *
 *
 *
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 * @brief Determines if the given value is between the
 *        minimum and maximum threshold values
 * @param
 * @param
 * @param
 *
 *
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 * @brief Determines if the given value is between the
 *        minimum and maximum threshold values
 * @param value The integer value to check
 * @param min The lower threshold (exclusive)
 * @param max The upper threshold (exclusive)
 *
 *
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 * @brief Determines if the given value is between the
 *        minimum and maximum threshold values
 * @param value The integer value to check
 * @param min The lower threshold (exclusive)
 * @param max The upper threshold (exclusive)
 * @return
 *
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 * @brief Determines if the given value is between the
 *        minimum and maximum threshold values
 * @param value The integer value to check
 * @param min The lower threshold (exclusive)
 * @param max The upper threshold (exclusive)
 * @return true if min < value < max, false otherwise
 *
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 * @brief Determines if the given value is between the
 *        minimum and maximum threshold values
 * @param value The integer value to check
 * @param min The lower threshold (exclusive)
 * @param max The upper threshold (exclusive)
 * @return true if min < value < max, false otherwise
 *
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum ≤ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 * @brief Determines if the given value is between the
 *        minimum and maximum threshold values
 * @param value The integer value to check
 * @param min The lower threshold (exclusive)
 * @param max The upper threshold (exclusive)
 * @return true if min < value < max, false otherwise
 * @pre min <= max
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

2. **Documentation**: Purpose statement, parameters, return value

```
/**
 * @brief Determines if the given value is between the
 *        minimum and maximum threshold values.
 * @param value The integer value to check
 * @param min The lower threshold (exclusive)
 * @param max The upper threshold (exclusive)
 * @return true if min < value < max, false otherwise
 * @pre min <= max
 */
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

3. **Definition**: Write the code to implement the function

```
bool is_between(int value, int min, int max) {



}
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

3. **Definition**: Write the code to implement the function

```
bool is_between(int value, int min, int max) {
   if (value > min && value < max) {
      return true;
   } else {
      return false;
   }
}
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

3. **Definition**: Write the code to implement the function

```
bool is_between(int value, int min, int max) {
    return value > min && value < max;
}
```

Function Design Example

- Design a function that determines if the value of an integer is strictly in between two other given integers, a minimum and maximum threshold, where minimum $\leq$ maximum.

4. **Testing**: Call the function with different input conditions to ensure it has the correct behaviour under all possible scenarios

- What we must include to fulfill adequate test coverage:
  - A value that is within the threshold range
  - A value equal to and below the minimum threshold
  - A value equal to and above the maximum threshold
  - Multiple different minimum and maximum threshold values.

15: Arrays

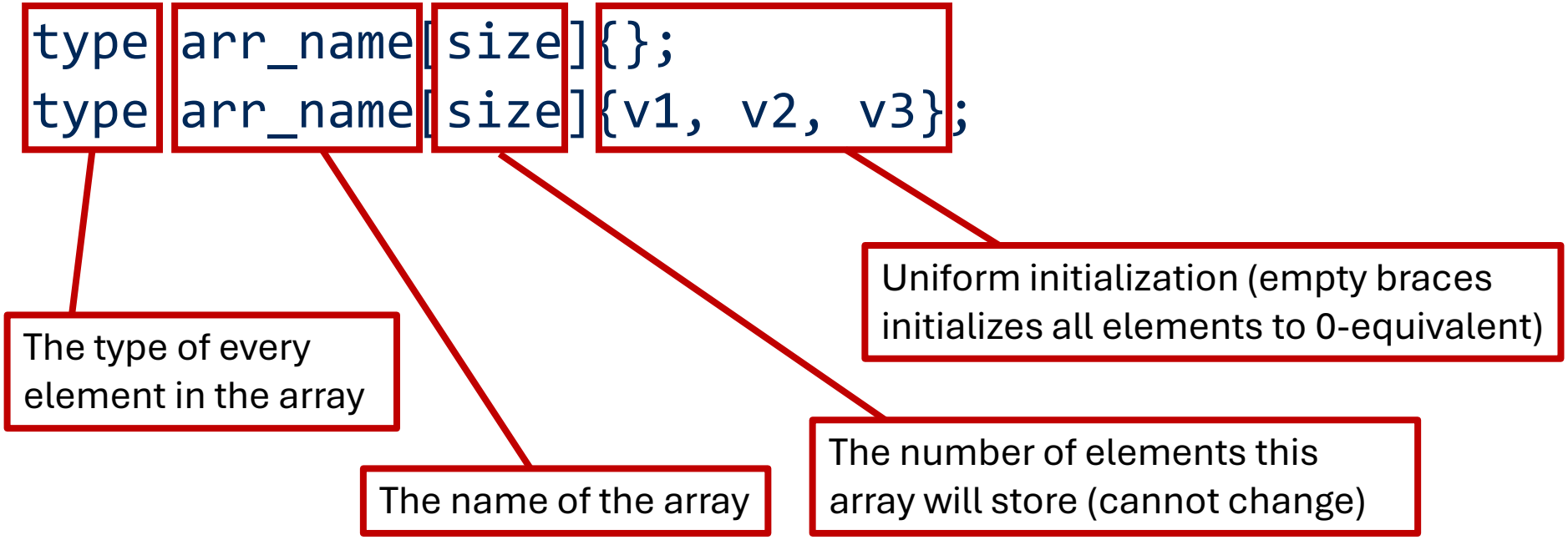
Anthony Estey
CSC 115: Fundamentals of Programming: II

Arrays in C++

- An array is a **fixed-size** collection that stores elements of which are all the **same type**
- Similar to Python lists, with some important differences:
  - Fixed size: cannot grow or shrink after declaration
  - All elements must be the same type
  - No built-in methods (no `.append()` or `.len()`)
- Arrays are stored in contiguous memory: memory for the whole arrays is allocated when it is created, and all elements sit side-by-side

Arrays in C++

- General form:



Arrays in C++

- General form:

```
type arr_name[size]{};
type arr_name[size]{v1, v2, v3};
```

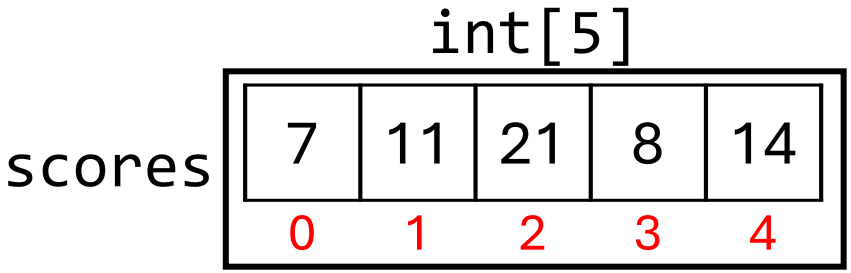
- Specific examples:

```
int scores[5]{}; // 5 ints all initialized to 0
int primes[5]{2, 3, 5, 7, 11}; // 5 ints with values
double temps[3]{}; // 3 doubles initialized to 0.0
```

Arrays in C++

- Elements are accessed by index, starting at 0:

```
int scores[5]{7, 11, 21, 8, 14};
```

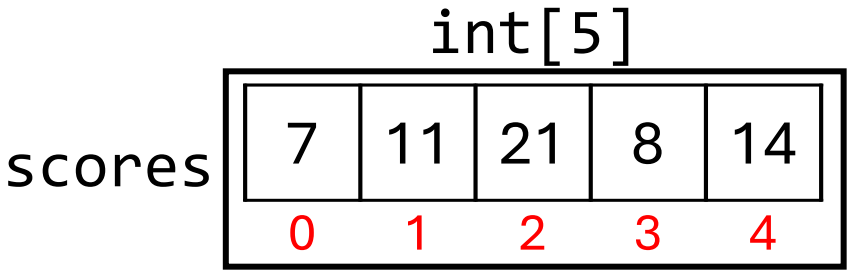


- We can access and update values using square bracket notation $[i]$, where i is the index of the element:
  - `int cur{scores[2]}; // initializes cur to 21`
  - `scores[3] = scores[0]+5; // changes scores[3]`
    `//  from 8 to 12`

Arrays in C++

- Elements are accessed by index, starting at 0:

```
int scores[5]{7, 11, 21, 8, 14};
```



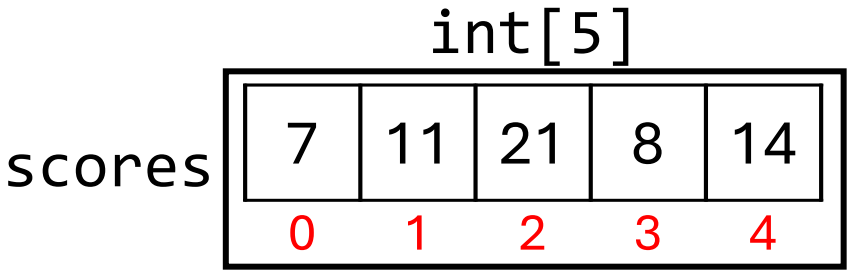
- Use a for loop to traverse all elements:

```
for (int i{0}; i < 5; i++) {
  std::cout << scores[i] << std::endl;
}
```

Arrays in C++

- Elements are accessed by index, starting at 0:

```
int scores[5]{7, 11, 21, 8, 14};
```



- Use a range-based for loop if we don't need the indexes:

```
for (int score : scores) {
  std::cout << score << std::endl;
}
```

Key Differences from Python

- There are a few key differences between C++ and Python when using arrays:

	Python	C++
Size	Grows dynamically	Fixed at declaration
Types	Mixed types allowed	All elements are the same type
Methods	`.append()`, `.len()`, etc.	None built-in
Out of bounds	An exception is thrown: **IndexError**	Undefined behaviour (more information on next slide)

Out of Bounds

- In Python, accessing an invalid index raises an IndexError
- In C++, there is no such protection:
  - the program reads or writes whatever is in that memory location
  - this can cause unpredictable behavior or a crash with no helpful error message
  - We will do some memory tracing and memory diagrams with arrays in the next unit
- In C++ it is our responsibility to ensure we never access an index outside the valid range (from index 0 to size-1)

16: Strings

Anthony Estey
CSC 115: Fundamentals of Programming: II

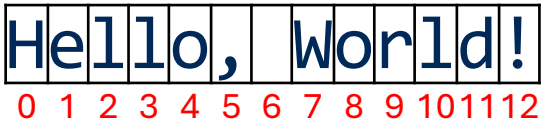
Strings in C++

- We are already familiar with the concept of a string from Python: it is a data type used to represent text
- In C++, we use **`std::string`** from the **`<string>`** library
- **`std::string`** is an object, meaning it has both data (the characters) and behaviours (methods) associated with it
  - This is our first look at what an object is, but objects are the main focus up an upcoming unit

String Operations

- Assume we have declared the following string:

```
std::string message{"Hello, World!"};
```

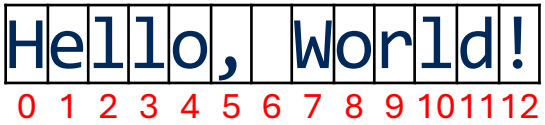


- Similar to arrays, each character can be accessed by its index number, which start from 0.

String Operations

- Assume we have declared the following string:

```
std::string message{"Hello, World!"};
```



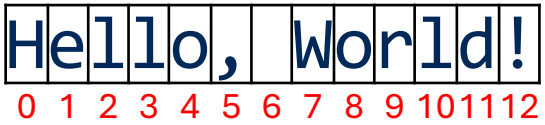
- Similar to arrays, each character can be accessed by its index number, which start from 0.

- We can access a single character in a string the same way we access an element in an array:
  - Therefore, `message[0]` is `'H'`, and `message[9]` is `'r'`.

String Operations

- Assume we have declared the following string:

```
std::string message{"Hello, World!"};
```



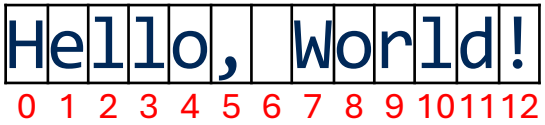
- Similar to arrays, each character can be accessed by its index number, which start from 0.

- The **length** function gets the length of the string (how many characters is has - including punctuation and spaces)
  - Therefore, `message.length()` returns `13`.

String Operations

- Assume we have declared the following string:

```
std::string message{"Hello, World!"};
```



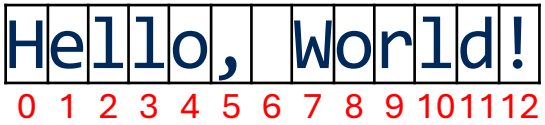
- Similar to arrays, each character can be accessed by its index number, which start from 0.

- The **substr** function allows us to access a range of text from the string, given an **initial index** and the **size** of the range:
  - Therefore, `message.substr(7, 5)` returns `"World"`.

String Operations

- Assume we have declared the following string:

```
std::string message{"Hello, World!"};
```



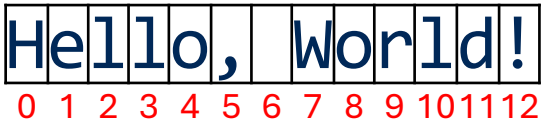
- Similar to arrays, each character can be accessed by its index number, which start from 0.

- The **find** function allows us to get the index of the first occurrence of something in a string:
  - Therefore, `message.find("World")` returns 7.

String Operations

- Assume we have declared the following string:

```
std::string message{"Hello, World!"};
```



- Similar to arrays, each character can be accessed by its index number, which start from 0.

- The **+** operator allows us to concatenate, effectively gluing something onto the side of a string:
  - `message + "Goodbye"` is `"Hello, World!Goodbye"`

    (we could manually add a space too)

Review of String Operations

- Assume we have declared the following string:

```
std::string message{"Hello, World!"};
```

Operation	Code	Result
Length	`message.length()`	`13`
Access by index	`message[0]`	`'H'`
Substring	`message.substr(7, 5)`	`"World"`
Find	`message.find("World")`	`7`
Concatenation	`message + " Goodbye"`	`"Hello, World! Goodbye"`

17: Structs

Anthony Estey
CSC 115: Fundamentals of Programming: II

Structs Motivation

- So far, our variables have stored a single piece of data, but often we want to represent something more complex.
- For example, let's say we wanted to write a program to do some analysis of how well students are doing in the course. How would we represent each student?

Structs Motivation

- What student information do we care about in our program?
  - Each student could have a **name**, an **ID**, and a **GPA**
  - We could declare three separate variables to store this information:

```
std::string student_name{"Ali Coder"};
std::string student_id{"V01234567"};
double student_gpa{7.8};
```

  - But this gets messy, fast! Especially as we work with more students
  - We need to be able to group this related data together

Structs Motivation

- **The problem**: no *single* data value adequately represents a student. Student information is composed of multiple attributes that all belong together!
  - Structs allow us to group related data together under a single name
  - In this case, a struct allows us to represent each student as a single unit, where each attribute (name, ID, GPA) has its own data member

Name: Ali Coder **ID**: V01234567 **GPA**: 7.8	**Name**: Sam Vikes **ID**: V01020304 **GPA**: 6.2	**Name**: Kim Lee **ID**: V00224466 **GPA**: 8.1

Syntax for structs

- Below we will create a struct to represent a student named "Ali Coder", with id "V01234567" and a GPA value of 7.8.

- General syntax (definition):

```
struct StructName {
    type member_name_1;
    type member_name_2;
    type member_name_3;
};
```

- Specific example:

```
struct Student {
    std::string full_name;
    std::string id;
    double gpa;
};
```

This resolves our motivating example: we have bundled **name**, **id**, and **gpa** into a single type that we can use throughout our program: a Student.

Declaration and initialization

```
struct Student {
    std::string full_name;
    std::string id;
    double gpa;
};
```

Note: The struct definition does not create a student in memory. Think of it as a *blueprint* or a *template* that describes the new data type we are creating.

- With the struct definition, we can now create an **instance** of a student based on its blueprint:

```
Student s1{"Ali Coder", "V01234567", 7.8};
Student s2{"Sam Vikes", "V01020304", 6.2};
Student s3{"Kim Lee",   "V00998877", 8.1};
```

Member access: the Dot Operator

- When working with student instances, we often want to work with one of the student attributes, like gpa
  - We use the **Dot operator** (.) to access a member of an instance:
- General syntax:

```
variable_name.member_name;
```

- Specific examples:

```
Student s1{"Ali Coder", "V01234567", 7.8}; // creates s1
std::cout << s1.gpa << std::endl;          // outputs 7.8
s1.gpa += 0.6;                             // updates s1's gpa to 8.4
std::cout << s1.gpa << std::endl;          // outputs 8.4
```